

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



HCMUTE

KHÓA LUẬN TỐT NGHIỆP

**XÂY DỰNG HỆ THỐNG TẠO WEBSITE PORTFOLIO
TỰ ĐỘNG DỰA TRÊN CÔNG NGHỆ
BROWSER-SIDE SANDBOX VÀ ĐỊNH TUYẾN MÔ
HÌNH NGÔN NGỮ LỚN ĐA PHƯƠNG THỨC**

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

Sinh viên: NGÔ KHẢI HOÀNG

MSSV: 22139024

TP. Hồ Chí Minh - Tháng 06, 2026

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG

KHÓA LUẬN TỐT NGHIỆP

XÂY DỰNG HỆ THỐNG TẠO WEBSITE PORTFOLIO
TỰ ĐỘNG DỰA TRÊN CÔNG NGHỆ
BROWSER-SIDE SANDBOX VÀ ĐỊNH TUYẾN MÔ
HÌNH NGÔN NGỮ LỚN ĐA PHƯƠNG THỨC

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

Sinh viên thực hiện: NGÔ KHẢI HOÀNG
MSSV: 22139024
Người hướng dẫn khoa học: TS. Huỳnh Thế Thiện

TP. Hồ Chí Minh - Tháng 06, 2026

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: XÂY DỰNG HỆ THỐNG TẠO WEBSITE PORTFOLIO TỰ ĐỘNG DỰA TRÊN CÔNG NGHỆ BROWSER-SIDE SANDBOX VÀ ĐỊNH TUYẾN MÔ HÌNH NGÔN NGỮ LỚN ĐA PHƯƠNG THỨC

Sinh viên: Ngô Khải Hoàng

MSSV: 22139024

Hướng dẫn: TS. Huỳnh Thế Thiệp

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn: Đề tài giải quyết bài toán tạo website tự động thông qua AI-Agentic Software Engineering. Việc chuyển dịch môi trường biên dịch về phía máy khách (Client-side computing) để bảo mật và giảm tải hạ tầng máy chủ là một hướng tiếp cận có tính ứng dụng thực tiễn.

2. Phương pháp thực hiện/ phân tích/ thiết kế: Kiến trúc hệ thống toàn diện và khá phức tạp. Sinh viên đã kết hợp các công nghệ hiện đại: WebContainer (ảo hóa), Convex DB (đồng bộ thời gian thực), Inngest (điều phối Saga Pattern) và Multimodal LLM Routing (DeepSeek/Gemini). Giải pháp sử dụng AST Patching để vá mã nguồn trực quan là một điểm sáng về tư duy thuật toán.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Hệ thống vận hành khá tốt, hiện thực hóa được luồng sinh mã, đồng bộ VFS thời gian thực và cập nhật giao diện HMR với độ trễ thấp. Tuy nhiên, tồn tại sự bất nhất trong việc báo cáo số liệu thực nghiệm thời gian đáp ứng của mô hình LLM.

4. Kết luận và đề xuất: Các kết luận bám sát kết quả đạt được. Các đề xuất mở rộng như tích hợp quản lý phiên bản Git cục bộ và ứng dụng mô hình đa tác nhân (Multi-agent System) để kiểm duyệt thiết kế (Design Auditor).

5. Hình thức trình bày và bố cục báo cáo: Báo cáo có cấu trúc chương mục hợp lý, văn phong học thuật mạch lạc. Các lưu đồ thuật toán và sơ đồ tuần tự UML được thiết kế rất trực quan, công phu. Tuy nhiên, vẫn còn mắc một số lỗi cầu thủ trong đánh máy mục lục và bỏ sót ký hiệu ở tiêu đề bảng biểu.

6. Kỹ năng chuyên nghiệp và tính sáng tạo: Sinh viên sở hữu năng lực lập trình Full-stack. Thể hiện kỹ năng giải quyết vấn đề, thiết kế kiến trúc vi dịch vụ và xử lý đồng thời (concurrency control) khá tốt.

7. Tài liệu trích dẫn: Nguồn tài liệu tham khảo phong phú, mang tính cập nhật cao với các công nghệ Web và AI mới nhất, được trích dẫn chuẩn xác.

8. Đánh giá về sự trùng lặp của đề tài: Đề tài mang tính độc lập hoàn toàn. Các cơ chế như Double-Loop Synchronization hay Origin Validation Protocol qua iframe là nỗ lực nghiên cứu tự thân của sinh viên.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa: Khuyết điểm lớn nhất nằm ở sự mâu thuẫn số liệu (Bảng 4.1 và nội dung phân tích). Sinh viên đã được yêu cầu rà soát, đo đạc lại và hiệu đính triệt để sự cố dữ liệu này trước khi nộp bản cuối.

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên: Thái độ học tập nghiêm túc và luôn chủ động nghiên cứu các giải pháp kỹ thuật khó để giải quyết vấn đề.

Đề nghị của giảng viên hướng dẫn:

Báo cáo hoàn toàn đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép bảo vệ khóa luận tốt nghiệp trước Hội đồng.

Tp. HCM, ngày 23 tháng 06 năm 2026

Người nhận xét
(Ký và ghi rõ họ tên)



TS. Huỳnh Thế Thiệp

Lời Cam Đoan

Với tư cách là người thực hiện khóa luận tốt nghiệp này, tôi là Ngô Khải Hoàng, mã số sinh viên 22139024, đang theo học ngành Hệ thống nhúng và IoT tại Khoa Điện - Điện tử, Trường Đại học Sư phạm Kỹ thuật TP.HCM. Tôi xin khẳng định đây hoàn toàn là công trình nghiên cứu do chính tôi sáng tạo dưới sự hướng dẫn của giáo viên hướng dẫn khoa học. Nội dung và kết quả của khóa luận phản ánh năng lực chuyên môn, kỹ năng nghiên cứu và sự nỗ lực tự thân của tôi, không hề vay mượn hay sao chép từ bất kỳ đồ án, bài báo hay tài liệu nào đã được công bố trước đây mà không trích dẫn nguồn. Tôi cam đoan mọi tài liệu tham khảo được sử dụng đều đã được ghi nhận đầy đủ và chính xác, tuân thủ nghiêm ngặt quy định về trích dẫn của Nhà trường và các chuẩn mực học thuật quốc tế. Tôi đảm bảo tính xác thực, khách quan của thông tin trình bày và khẳng định không có hành vi học thuật không trung thực. Tôi hoàn toàn chịu trách nhiệm về tính nguyên bản của công trình này và chấp nhận mọi hình thức xử lý kỷ luật nếu phát hiện bất kỳ vi phạm nào đối với bản cam kết này.

Người thực hiện khóa luận
(Ký và ghi rõ họ tên)

Ngô Khải Hoàng

Lời Cảm Tạ

Tôi nhận thức sâu sắc rằng việc hoàn thành khóa luận tốt nghiệp này không thể thực hiện được nếu thiếu đi sự đồng hành, tư vấn và hỗ trợ quý báu từ quý Thầy Cô cùng các bạn bè trong ngành Hệ thống nhúng và IoT, Khoa Điện - Điện tử, Trường Đại Học Sư Phạm Kỹ Thuật Thành phố Hồ Chí Minh. Tôi xin bày tỏ lòng biết ơn chân thành nhất đến tất cả những người đã dành thời gian, công sức góp ý và giúp đỡ tôi trong suốt quá trình thực hiện công trình này. Đặc biệt, sự dẫn dắt của Thầy Huỳnh Thế Thiện đóng vai trò then chốt. Những định hướng chuyên môn, lời khuyên tận tình và lộ trình nghiên cứu Thầy vạch ra ngay từ những bước đi đầu tiên đã ảnh hưởng sâu sắc đến tư duy và cách tiếp cận đề tài của tôi. Dù đã rất cố gắng, khóa luận chắc chắn vẫn còn tồn tại những điểm chưa hoàn thiện. Tôi rất mong nhận được những góp ý thẳng thắn và đánh giá khách quan để có thể nâng cao kiến thức, khắc phục hạn chế và tạo ra những sản phẩm chất lượng hơn trong tương lai. Xin chân thành cảm ơn!

Tóm Tắt

Trong thời đại chuyển đổi số và sự bùng nổ của trí tuệ nhân tạo (AI), việc sở hữu một trang web hồ sơ năng lực (portfolio) cá nhân đã trở nên thiết yếu đối với các lập trình viên và nhà thiết kế. Tuy nhiên, các giải pháp hiện tại đòi hỏi kỹ năng lập trình cao hoặc quá gò bó trong các khuôn mẫu cứng nhắc của các công cụ no-code. Luận văn này đề xuất hệ thống **Nexastu** — một nền tảng xây dựng website portfolio tự động thông minh bằng AI-Agent, hoạt động dưới dạng một môi trường phát triển tích hợp trên nền web (Web-based IDE) nhờ tích hợp công nghệ ảo hóa trình duyệt WebContainer và cơ chế định tuyến mô hình ngôn ngữ lớn đa phương thức. Hệ thống cho phép người dùng chỉ cần tải lên tệp lý lịch cá nhân (CV) PDF hoặc nhập mô tả văn bản để tạo ra một website Next.js hoàn chỉnh chạy trực tiếp trên trình duyệt.

Để đạt được mục tiêu đó, nghiên cứu này triển khai ba đóng góp kỹ thuật cốt lõi. *Thứ nhất*, hệ thống phát triển luồng sinh mã nguồn tự động (CodeGen Pipeline) sử dụng Gemini 2.5 Flash và DeepSeek V4 thông qua bộ điều phối Orchestrator, kết hợp hai cổng kiểm soát Smoke Gate và Generation Health Gate để đảm bảo mã sinh ra tối ưu và an toàn. *Thứ hai*, tích hợp công nghệ ảo hóa WebContainer API giúp chạy trực tiếp môi trường Node.js và máy chủ phát triển Next.js ngay trong trình duyệt của client mà không phụ thuộc vào hạ tầng backend. *Thứ ba*, hệ thống thiết kế cầu nối Inspector Bridge an toàn qua API `postMessage`, hỗ trợ tính năng chỉnh sửa trực quan (Visual Text Edit) dựa trên phân tích và vá cây cú pháp AST, và cơ chế tự động chuyển đổi thông minh (Multimodal Routing Fallback) từ DeepSeek sang Gemini khi có tệp hình ảnh đi kèm.

Kết quả thực nghiệm thực tế cho thấy hệ thống Nexastu hoạt động ổn định, rút ngắn thời gian xây dựng website từ hàng giờ xuống dưới 2 phút, đảm bảo mã nguồn sinh ra đạt độ chính xác cú pháp 100% qua các cổng kiểm soát và mang lại trải nghiệm chỉnh sửa trực quan mượt mà cho người dùng cuối. Những kết quả này chứng minh tính khả thi và hiệu quả vượt trội của việc áp dụng mô hình AI-Agent kết hợp ảo hóa client-side trong các nền tảng phát triển phần mềm tự động thế hệ mới.

Mục Lục

1	TỔNG QUAN	1
1.1	GIỚI THIỆU	1
1.1.1	Bối cảnh nghiên cứu	1
1.1.2	Tính cấp thiết của đề tài và Động lực nghiên cứu	1
1.1.3	Giải pháp đề xuất	4
1.2	MỤC TIÊU	4
1.3	PHƯƠNG PHÁP NGHIÊN CỨU	6
1.4	PHẠM VI VÀ GIỚI HẠN NGHIÊN CỨU	7
1.5	BỐ CỤC BÁO CÁO	8
2	CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ LIÊN QUAN	9
2.1	Kiến trúc ứng dụng Web hiện đại và Framework Next.js	9
2.1.1	Tổng quan về React và cơ chế Rendering	10
2.1.2	Framework Next.js và kiến trúc App Router	11
2.2	Cơ sở dữ liệu thời gian thực Convex	12
2.2.1	Luận lý lựa chọn công nghệ và phân tích đối chiếu	12
2.2.2	Cơ chế đăng ký và truy vấn phản ứng (Reactive Queries)	14
2.2.3	Đảm bảo tính nhất quán giao dịch qua Mutations và Actions	14
2.2.4	Cấu trúc Schema và an toàn kiểu dữ liệu (Type Safety)	16

2.3	Kiến trúc xử lý tác vụ ngầm hướng sự kiện Inngest	16
2.3.1	Nguyên lý Step Functions và Ứng dụng Saga Pattern trong điều phối tác vụ	17
2.3.2	Kiểm soát đồng thời và Đồng bộ hóa trạng thái bất đồng bộ	18
2.4	Công nghệ ảo hóa trình duyệt WebContainer API	19
2.4.1	Nguyên lý ảo hóa Node.js trên WebAssembly Sandbox và Cơ chế bảo mật đa luồng	19
2.4.2	Hệ thống tệp ảo (VFS) và Định tuyến mạng ảo hóa trình duyệt	20
2.4.3	So sánh WebContainer với các giải pháp ảo hóa truyền thống	21
2.5	Mô hình Ngôn ngữ Lớn và Kỹ nghệ Định tuyến	22
2.5.1	Đặc tính kỹ thuật của Gemini 2.5 Flash và DeepSeek V4	22
2.5.2	Kỹ nghệ định tuyến đa phương thức dự phòng (Multimodal Routing Fallback) . .	23
2.5.3	Định tuyến động đa mô hình và Cơ chế chốt chặn Vòng lặp tác nhân (Agent Loop Guard)	23
2.6	Phân tích cú pháp AST (Abstract Syntax Tree) và kỹ thuật vá mã nguồn	25
2.6.1	Cây cú pháp trừu tượng AST	25
2.6.2	Kỹ thuật định vị và vá mã nguồn (AST Patching)	26
2.7	Cơ chế đồng bộ hóa VFS-Convex và Đảm bảo Nhất quán Trạng thái	27
2.7.1	Thuật toán đồng bộ hóa trạng thái VFS lên Convex DB	27
2.7.2	Cơ chế đảm bảo nhất quán giao dịch và phục hồi qua bộ kiểm tra AST	28
2.8	Cơ chế chỉnh sửa trực tiếp contentEditable và Mô hình đồng bộ hai luồng	29
2.8.1	Thách thức của API HTML5 contentEditable trong soạn thảo trực quan	29
2.8.2	Mô hình đồng bộ trạng thái hai luồng (Double-Loop Synchronization)	30
3	PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	31
3.1	KIẾN TRÚC HỆ THỐNG TỔNG QUAN	31
3.1.1	Các thành phần cốt lõi của hệ thống	31
3.1.2	Các luồng truyền thông tin chủ đạo	33

3.2	THIẾT KẾ LUỒNG XỬ LÝ SINH MÃ TỰ ĐỘNG (RESUME-TO-CODE PIPELINE) . .	34
3.3	THIẾT KẾ CƠ CHẾ INSPECTOR VÀ CẦU NỐI TRUYỀN TIN CORS-SAFE	38
3.3.1	Cơ chế tương tác trực quan và đồng bộ hóa văn bản (Visual Text Edit)	39
3.3.2	Cơ chế quản lý tài nguyên nhị phân và thuật toán vá ảnh (AST Image Patching) .	41
3.3.3	Cơ chế phục hồi khối giao diện bảo toàn dữ liệu (Non-Destructive Block Restore) .	43
3.4	THIẾT KẾ CƠ CHẾ ĐỊNH TUYẾN ĐA PHƯƠNG THỨC THÔNG MINH	44
3.4.1	Cơ chế định tuyến đa phương thức dự phòng (Multimodal Routing Fallback) . . .	44
3.4.2	Giải thuật tự động thử lại ngăn chặn hiện tượng báo cáo thành công ảo (Retry Loop Guard)	46
3.5	THIẾT KẾ CƠ CHẾ ĐỒNG BỘ HÓA VFS VÀ PHẢN HỒI THỜI GIAN THỰC	47
3.5.1	Thiết kế thuật toán đồng bộ hóa trạng thái VFS lên Convex DB	47
3.5.2	Cơ chế tự động kích hoạt Hot Module Replacement (HMR) cập nhật Live Preview	48
3.6	THIẾT KẾ CƠ SỞ DỮ LIỆU HỆ THỐNG	49
3.6.1	Bảng dự án (projects)	50
3.6.2	Bảng lưu trữ tệp tin (files)	51
3.6.3	Bảng quản lý hội thoại và tin nhắn (conversations & messages)	51
3.6.4	Bảng sản phẩm và đơn hàng thương mại điện tử (products & orders)	53
3.6.5	Đánh giá thiết kế cơ sở dữ liệu và tối ưu truy vấn	55
3.7	CẤU TRÚC THÔNG ĐIỆP TRAO ĐỔI DỮ LIỆU	56
3.7.1	Cấu trúc JSON bóc tách thông tin CV từ Gemini 2.5 Flash	56
3.7.2	Cấu trúc thông điệp tạo tệp tin XML File Actions từ LLM	57
3.7.3	Cấu trúc thông điệp truyền tin bất đồng bộ postMessage qua Iframe Bridge	58
3.7.4	Cơ chế xác thực Origin bảo mật (Origin Validation Protocol)	59
3.8	THIẾT KẾ GIAO DIỆN THƯƠNG MẠI ĐIỆN TỬ VÀ TÍCH HỢP THANH TOÁN VIETQR	60

3.8.1	Thiết kế giao diện và mẫu giao diện Cửa hàng Công nghệ cao (E-commerce Hi-Tech Template)	61
3.8.2	Cơ chế thanh toán VietQR và giải thuật Live Polling trạng thái đơn hàng	62
4	KẾT QUẢ VÀ THẢO LUẬN	64
4.1	Phương pháp luận thực nghiệm và Thiết lập môi trường	64
4.2	Đo lường thời gian đáp ứng và Hiệu năng hệ thống	65
4.3	Đánh giá định lượng chất lượng sinh mã nguồn của LLM	68
4.4	So sánh đối chiếu kiến trúc hệ thống với các giải pháp công nghiệp	69
4.5	Kiến trúc triển khai thực tế đám mây Serverless	71
4.6	Minh chứng giao diện người dùng	71
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	75
5.1	Kết luận	75
5.2	Hạn chế của hệ thống	76
5.3	Hướng phát triển trong tương lai	77
	TÀI LIỆU THAM KHẢO	79

Danh sách hình

2.1	Cơ chế so khớp và cập nhật giao diện thông qua Virtual DOM trong React.	11
2.2	So sánh quy trình kết xuất và tải trang giữa CSR và SSR.	12
2.3	Sơ đồ kiến trúc xử lý dữ liệu và đồng bộ hóa thời gian thực của Convex Database.	15
2.4	Sơ đồ kiến trúc điều phối giao dịch phân tán Saga của Inngest.	17
2.5	Biểu diễn cây cú pháp trừu tượng AST chi tiết của phần tử JSX trong Nexastu.	25
3.1	Sơ đồ kiến trúc tổng quan hệ thống Nexastu.	32
3.2	Sơ đồ tuần tự Pipeline sinh mã Resume-to-Code.	36
3.3	Sơ đồ tuần tự tương tác cầu nối Inspector Bridge.	38
3.4	Sơ đồ hoạt động luồng tải ảnh và vá mã nguồn AST trong Nexastu.	42
3.5	Sơ đồ thuật toán định tuyến đa phương thức dự phòng.	45
3.6	Lưu đồ thuật toán đồng bộ hóa trạng thái VFS sang Convex.	47
4.1	Nhật ký thực thi tiến trình sinh mã Resume-to-Code trên giao diện Inngest.	67
4.2	Giao diện bảng điều khiển quản lý danh sách dự án (Dashboard) của Nexastu.	72
4.3	Giao diện bảng điều khiển làm việc chính (Web-IDE) trong chế độ phát triển.	72
4.4	Giao diện xem trước trực tiếp trang web portfolio Next.js của người dùng.	73
4.5	Tính năng mở rộng giao diện xem trước của WebContainer sang tab trình duyệt độc lập.	73
4.6	Minh họa cơ chế di chuột hover trong chế độ Inspector.	74
4.7	Chế độ chỉnh sửa chữ đúp chuột trực tiếp contentEditable.	74

Danh sách bảng

2.1	So sánh các giải pháp hạ tầng cơ sở dữ liệu cho hệ thống Web-IDE	13
2.2	Các giai đoạn thực thi trong quy trình điều phối Saga bằng Inngest Step Functions. . . .	18
2.3	So sánh các giải pháp ảo hóa môi trường thực thi mã nguồn	21
2.4	So sánh đặc tính kỹ thuật giữa Gemini 2.5 Flash và DeepSeek V4	23
3.1	Chú giải ký hiệu và đường liên kết trong sơ đồ kiến trúc hệ thống	33
3.2	Đặc tả cấu trúc dữ liệu bảng projects	50
3.3	Đặc tả cấu trúc dữ liệu bảng files	51
3.4	Đặc tả cấu trúc dữ liệu bảng conversations	52
3.5	Đặc tả cấu trúc dữ liệu bảng messages	52
3.6	Đặc tả cấu trúc dữ liệu bảng products	53
3.7	Đặc tả cấu trúc dữ liệu bảng orders	54
3.8	Đặc tả cấu trúc dữ liệu JSON bóc tách thông tin CV từ Gemini	57
3.9	Đặc tả cấu trúc thẻ XML File Actions trong kết quả sinh mã	58
3.10	Đặc tả thông điệp giao tiếp postMessage qua Iframe Bridge	59
4.1	Các chỉ số thời gian đáp ứng trung bình của hệ thống Nexastu	66
4.2	Đặc tả so sánh định lượng chất lượng sinh mã nguồn giữa hai mô hình LLM	68
4.3	Bảng so sánh đối chiếu kiến trúc hệ thống Nexastu với các giải pháp công nghiệp	70

Danh sách các từ viết tắt

Các từ viết tắt	Định nghĩa
API	Application Programming Interface
AST	Abstract Syntax Tree
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
DB	Database
DOM	Document Object Model
HMR	Hot-Module Replacement
HSL	Hue, Saturation, Lightness
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
JSX	JavaScript XML
LLM	Large Language Model
SOTA	State-of-the-Art
TSX	TypeScript XML
UI	User Interface
UX	User Experience
Wasm	WebAssembly
WC	WebContainer

Chương 1

TỔNG QUAN

1.1 GIỚI THIỆU

1.1.1 Bối cảnh nghiên cứu

Trong kỷ nguyên cách mạng công nghiệp lần thứ tư, sự giao thoa giữa trí tuệ nhân tạo (AI) và công nghệ phát triển phần mềm đang tái định hình cách thức thiết kế, triển khai và vận hành các ứng dụng công nghệ thông tin. Trong đó, việc xây dựng và khẳng định thương hiệu số cá nhân thông qua các trang web hồ sơ năng lực (portfolio) trực tuyến đã trở thành một nhu cầu thiết yếu đối với cộng đồng kỹ sư phần mềm, nhà phát triển hệ thống và nhà thiết kế. Một website portfolio chuyên nghiệp không đơn thuần là một trang thông tin tĩnh, mà đóng vai trò là một hệ thống phần mềm front-end tích hợp, thể hiện các dự án thực tế, kỹ năng chuyên môn, khả năng tương tác và năng lực kỹ thuật của cá nhân.

Sự tiến hóa vượt bậc của các framework phát triển web hiện đại (đặc biệt là Next.js [1] và React [2]) kết hợp với các thư viện CSS tối ưu hóa như Tailwind CSS đã thiết lập nên các tiêu chuẩn mới về hiệu năng tải trang, khả năng đáp ứng (responsiveness) và tối ưu hóa công cụ tìm kiếm (SEO). Song hành với đó, sự bùng nổ của các mô hình ngôn ngữ lớn (LLM) có năng lực suy luận và lập trình xuất sắc đã mở ra một hướng nghiên cứu mới đầy tiềm năng trong ngành khoa học máy tính: tự động hóa quy trình sản xuất phần mềm dựa trên tác tử thông minh (AI-Agent Software Engineering).

1.1.2 Tính cấp thiết của đề tài và Động lực nghiên cứu

Mặc dù nhu cầu sở hữu một trang web portfolio cá nhân chất lượng cao là rất lớn, việc hiện thực hóa mục tiêu này trong thực tế hiện nay đang đối mặt với một sự đánh đổi gay gắt (architectural

trade-off) giữa tính dễ sử dụng (Usability) và khả năng tùy biến linh hoạt cùng hiệu năng vận hành (Flexibility/Performance). Rào cản kỹ thuật và chi phí hạ tầng này tạo ra các điểm nghẽn lớn sau:

1. **Sự hạn chế của các giải pháp No-Code truyền thống:** Các nền tảng xây dựng website dạng kéo thả như Wix, Squarespace hay Webflow mang lại sự tiện lợi cho người dùng không chuyên, nhưng mã nguồn do các hệ thống này sinh ra thường cồng kềnh, không tối ưu hóa về mặt ngữ nghĩa (semantic HTML), dẫn đến tốc độ tải trang chậm và điểm SEO thấp. Hơn nữa, các giải pháp này thường gây ra sự lệ thuộc hoàn toàn vào nhà cung cấp (vendor lock-in) khi mã nguồn không thể tách rời khỏi hạ tầng lưu trữ độc quyền của nền tảng, buộc người dùng phải chi trả chi phí duy trì định kỳ ở mức cao.
2. **Rào cản kỹ thuật lớn của phương pháp lập trình thủ công:** Việc tự lập trình một website portfolio tối ưu bằng Next.js, React và Tailwind CSS mang lại quyền kiểm soát tuyệt đối và hiệu năng vượt trội, nhưng đòi hỏi kiến thức chuyên sâu về công nghệ web, quy trình cấu hình môi trường phát triển cục bộ, quản lý gói phụ thuộc, và quản trị hạ tầng máy chủ lưu trữ (hosting/deployment). Đây là một rào cản quá lớn đối với đa số người dùng không chuyên về phát triển web.
3. **Khoảng cách giữa mô hình sinh mã và môi trường triển khai thực tế:** Các trợ lý lập trình trí tuệ nhân tạo (AI) phổ biến hiện nay chủ yếu tập trung giải quyết bài toán sinh mã nguồn ở mức độ các đoạn mã (code snippets) rời rạc, đòi hỏi người dùng phải tự tích hợp và cấu hình thủ công. Mặc dù các môi trường phát triển tích hợp tác tử (Agentic IDE như Cursor) hoặc các nền tảng xem trước trực quan (Visual Preview như v0.dev) đã ra đời để tự động hóa quy trình này, chúng vẫn đối mặt với những giới hạn lớn đối với người dùng không chuyên. Cụ thể, các Agentic IDE yêu cầu quy trình thiết lập môi trường cục bộ (local environment) phức tạp; trong khi đó, các nền tảng xem trước trực quan lại bị giới hạn ở các ứng dụng tĩnh phía Frontend, chưa thể khởi chạy một hệ thống Full-stack hoàn chỉnh (ví dụ: dự án Next.js App Router với các gói phụ thuộc npm thực tế). Sự đứt gãy trong chu trình phản hồi (Feedback Loop) này làm giảm đáng kể hiệu quả ứng dụng thực tiễn của công nghệ AI sinh mã trong việc tiếp cận nhóm người dùng đại chúng.
4. **Thử thách về bảo mật và tài nguyên của cơ chế biên dịch phía Server (Server-side compilation):** Đối với các hệ thống AI tự động sinh và chạy thử nghiệm mã nguồn web cho hàng ngàn người dùng đồng thời, việc biên dịch và chạy runtime Next.js trên server backend tiêu tốn nguồn tài nguyên phần cứng cực kỳ lớn (RAM, CPU). Quan trọng hơn, do mã nguồn được sinh ra động từ các yêu cầu tự do của người dùng, hệ thống backend phải đối mặt trực tiếp với các lỗ hổng bảo mật nghiêm trọng như tấn công tiêm lệnh (Prompt Injection) dẫn đến thực thi mã từ xa độc hại (Remote Code Execution - RCE), đe dọa sự an toàn của toàn bộ máy chủ trung tâm.

Nghiên cứu thử nghiệm tiền đề và Bài học kinh nghiệm: Trước khi định hình kiến trúc hiện tại của Nexastu, chúng tôi đã tiến hành một khảo sát thực nghiệm tiền đề (pilot study) về hướng tiếp cận chuyển đổi trực tiếp giao diện từ bản vẽ thiết kế hoặc mã nguồn HTML/CSS tĩnh sang dự án Next.js (mô hình *Design-to-Code*). Kết quả đánh giá thực nghiệm cho thấy phương thức này vấp phải hai rào cản khoa học lớn:

Sự thiếu hụt dữ liệu ngữ cảnh thực tế (Semantic Context Scarcity): Bản vẽ thiết kế chỉ chứa thông tin phân bổ giao diện hiển thị mà hoàn toàn trống rỗng về dữ liệu nghiệp vụ của chủ thể (thông tin cá nhân, dự án thực tế, kỹ năng chuyên môn). Điều này buộc mô hình AI phải sinh dữ liệu giả lập (Lorem Ipsum), làm giảm đáng kể tính cá nhân hóa và giá trị thực tiễn của sản phẩm.

Sự suy giảm cấu trúc component (Component Degradation): Việc ánh xạ cấu trúc giao diện phẳng (flat layout) sang các component React/Next.js có tính phân cấp và tái sử dụng cao dẫn đến mã nguồn sinh ra bị phân mảnh, trùng lặp và cực kỳ khó refactor hoặc bảo trì.

Bài học kinh nghiệm từ nghiên cứu tiền đề này đã dịch chuyển tư duy thiết kế sang kiến trúc cốt lõi của Nexastu: **Lấy nội dung thực tế làm gốc (Content-First)** thông qua trích xuất dữ liệu lý lịch CV có cấu trúc chặt chẽ, kết hợp với ảo hóa client-side và áp dụng cơ chế vá mã nguồn động dựa trên cây cú pháp trừu tượng AST để tinh chỉnh giao diện trực quan, thay vì cố gắng chuyển đổi từ các bản thiết kế tĩnh.

Nhằm giải quyết triệt để các hạn chế đã nêu, luận văn tập trung vào việc khảo sát và thực thi giải pháp cho hai bài toán công nghệ trọng tâm. Nghiên cứu này đề xuất sự kết hợp đồng bộ giữa công nghệ ảo hóa phía máy khách (client-side) và trí tuệ nhân tạo, từ đó kiến tạo một môi trường thực thi vừa đảm bảo tính an toàn hệ thống, vừa tối ưu hóa chi phí tài nguyên vận hành.

Thứ nhất, dịch chuyển mô hình bảo mật và tính toán về phía máy khách (Client-side Computing): Đề tài tập trung nghiên cứu giải pháp ứng dụng công nghệ ảo hóa trình duyệt dựa trên nền tảng WebAssembly nhằm khởi chạy Node.js runtime trực tiếp trên trình duyệt của người dùng cuối qua WebContainer API [3]. Cơ chế này không chỉ chuyển giao toàn bộ áp lực tính toán từ hệ thống máy chủ về phía client nhằm tối giản hóa chi phí hạ tầng, mà còn thiết lập một hàng rào bảo mật nghiêm ngặt. Bằng cách cô lập mỗi môi trường thực thi mã nguồn framework động tại trình duyệt máy khách, hệ thống hạn chế tối đa các hệ lụy từ tấn công tiêm lệnh (Prompt Injection) dẫn đến thực thi mã từ xa độc hại (Remote Code Execution - RCE) đe dọa đến hệ thống máy chủ trung tâm.

Thứ hai, tối ưu hóa hiệu năng suy luận và đảm bảo tính khả thi kinh tế (Multimodal LLM Routing): Nghiên cứu xây dựng thuật toán định tuyến thông minh để tự động nhận diện tính chất của dữ liệu đầu vào. Hệ thống sẽ tự động phân tích và điều phối linh hoạt các cuộc gọi API giữa mô hình chuyên dụng tối ưu mã nguồn (DeepSeek V4 [4]) và mô hình đa phương thức có năng lực xử lý thị giác (Gemini 2.5 Flash [5]). Cơ chế định tuyến dự phòng này giải quyết hài hòa bài toán đánh đổi giữa năng lực xử lý hình ảnh thời gian thực và chi phí vận hành dịch vụ của hệ thống AI-Agent, đảm bảo tính khả thi khi triển khai hệ thống trong thực tế.

Sự giao thoa đồng bộ giữa hai hướng tiếp cận công nghệ trên chính là nền tảng thúc đẩy và hiện thực hóa hệ thống Nexastu. Đây cũng là tiền đề quan trọng giúp tối ưu hóa trải nghiệm người dùng cuối mà vẫn bảo vệ tính toàn vẹn của hệ thống ứng dụng.

1.1.3 Giải pháp đề xuất

Nhằm hiện thực hóa các định hướng nghiên cứu trên, đề tài đề xuất thiết kế và phát triển hệ thống **Nexastu** (AI-Agentic Hybrid Portfolio Builder) dưới dạng một môi trường phát triển tích hợp trên nền web (Web-based IDE) an toàn, tối ưu và thân thiện. Nexastu tích hợp sâu sắc công nghệ tác tử AI (AI-Agent) với môi trường ảo hóa trình duyệt nhằm tự động hóa toàn bộ vòng đời phát triển của một trang web portfolio cá nhân Next.js.

Hệ thống cho phép người dùng chỉ cần tải lên tệp lý lịch cá nhân (CV) PDF hoặc cung cấp mô tả bằng ngôn ngữ tự nhiên; từ đó, AI-Agent của Nexastu sẽ tự động phân tích cấu trúc thông tin, sinh mã nguồn Next.js/React/Tailwind chuẩn mực, cài đặt các gói phụ thuộc và khởi chạy máy chủ phát triển (development server) ngay trong sandbox của trình duyệt. Đồng thời, hệ thống tích hợp bộ định tuyến đa phương thức (Multimodal Router) nhằm phân phối linh hoạt các tác vụ lập trình và xử lý thị giác giữa hai mô hình ngôn ngữ lớn cốt lõi. Bên cạnh đó, Nexastu giới thiệu giải pháp cầu nối trực quan Inspector Bridge chạy ngầm, ứng dụng các thuật toán thao tác trên cây cú pháp trừu tượng AST (Abstract Syntax Tree) [6] để mang lại tính năng chỉnh sửa văn bản trực tiếp (Visual Text Edit) ngay trên giao diện đang thực thi thời gian thực mà không cần can thiệp mã nguồn thủ công. Đây là giải pháp mang tính đột phá giúp phổ cập hóa công nghệ lập trình hiện đại đến nhóm người dùng không chuyên.

1.2 MỤC TIÊU

Trước bối cảnh các giải pháp xây dựng website portfolio hiện tại đang phải đối mặt với sự đánh đổi gay gắt giữa tính dễ sử dụng và khả năng tùy biến linh hoạt, đồng thời các trợ lý lập trình AI hiện đại vẫn tồn tại khoảng cách lớn giữa khâu sinh mã nguồn và môi trường triển khai thực thi an toàn, luận văn này đặt ra mục tiêu tổng quát là nghiên cứu, thiết kế và hiện thực hóa hệ thống **Nexastu** — một môi trường phát triển tích hợp trên nền web (Web-based IDE) hỗ trợ tự động tạo website portfolio cá nhân thông minh từ lý lịch người dùng. Hệ thống hướng tới việc kết hợp đồng bộ giữa công nghệ ảo hóa sandbox phía máy khách (Client-side Sandbox) và cơ chế định tuyến mô hình ngôn ngữ lớn đa phương thức (Multimodal LLM Routing), nhằm tối ưu hóa hiệu năng, bảo mật tuyệt đối hạ tầng máy chủ và nâng cao trải nghiệm chỉnh sửa trực quan cho người dùng cuối. Để hiện thực hóa mục tiêu tổng quát trên, nghiên cứu tập trung giải quyết các mục tiêu cụ thể sau đây:

Xây dựng tiến trình sinh mã nguồn tự động từ lý lịch người dùng (Resume-to-Code Pipeline) — Mục tiêu đầu tiên là nghiên cứu giải pháp chuyển đổi dữ liệu bán cấu trúc từ tệp CV (PDF/Text) thành mã nguồn hệ thống hoàn chỉnh (Next.js/React/Tailwind). Tiến trình này tích hợp bộ điều phối Orchestrator cùng các cơ chế kiểm soát chất lượng tự động (Smoke Gate và Health Gate) nhằm đảm bảo mã nguồn sinh ra đạt chuẩn cú pháp và có khả năng biên dịch thực tế mà không cần sự

can thiệp thủ công của lập trình viên.

Ảo hóa môi trường thực thi và biên dịch phía máy khách (Client-side Sandbox) — Nhằm giải quyết triệt để gánh nặng tài nguyên máy chủ và nguy cơ an ninh mạng (RCE), nghiên cứu hướng tới ứng dụng giải pháp sandbox trình duyệt để khởi chạy máy chủ phát triển Next.js trực tiếp trong sandbox của trình duyệt người dùng. Đóng góp này giúp dịch chuyển toàn bộ tác vụ biên dịch và cài đặt gói phụ thuộc về phía máy khách, mở ra mô hình điện toán biên hiệu quả cho các ứng dụng web động.

Hiện thực hóa cầu nối tương tác trực quan thời gian thực (Interactive Visual Inspector) — Đây là giải pháp then chốt giúp xóa nhòa khoảng cách giữa giao diện thực thi và mã nguồn. Mục tiêu cụ thể bao gồm việc thiết lập kênh truyền tin hai chiều bảo mật giữa môi trường ứng dụng chạy trong iframe và giao diện IDE thông qua API `postMessage` (*Inspector Bridge*); nghiên cứu thuật toán phân tích và chỉnh sửa cấu trúc mã nguồn trực tiếp từ các thao tác trực quan của người dùng thông qua việc vá cây cú pháp trừu tượng AST (*Visual Text Edit*) nhằm cập nhật tức thời mã nguồn mà không làm ảnh hưởng đến cấu trúc logic của dự án.

Tối ưu hóa chi phí vận hành thông qua bộ định tuyến thông minh (Multimodal LLM Routing) — Mục tiêu là thiết kế một cơ chế định tuyến động tự động điều phối các yêu cầu xử lý từ người dùng đến các mô hình chuyên dụng phù hợp (mô hình tối ưu hóa sinh mã và mô hình xử lý thị giác). Sự kết hợp này nhằm tối ưu hóa sự đánh đổi giữa chất lượng mã nguồn sinh ra, tốc độ phản hồi và chi phí vận hành dịch vụ AI thực tế.

Đánh giá thực nghiệm và đo lường hiệu năng hệ thống — Tiến hành các thực nghiệm định lượng trên tập dữ liệu CV đa dạng để đánh giá tỷ lệ biên dịch thành công, thời gian suy luận (latency) và chi phí tài nguyên của hệ thống, từ đó khẳng định tính khoa học và độ bền vững của mô hình đề xuất trước các thách thức trong triển khai thực tế.

Để làm cơ sở khoa học và công cụ thực thi cho các mục tiêu trên, đối tượng nghiên cứu trọng tâm của luận văn tập trung vào ba thực thể công nghệ cốt lõi bao gồm: (1) Các mô hình ngôn ngữ lớn đa phương thức chuyên dụng cho lập trình và thị giác máy tính (*DeepSeek V4* [4], *Gemini 2.5 Flash* [5]); (2) Giải pháp ảo hóa hệ điều hành mini chạy runtime Node.js trực tiếp trên trình duyệt WebAssembly thông qua *StackBlitz WebContainer API* [3]; và (3) Các thuật toán thao tác dữ liệu cấu trúc dựa trên cây cú pháp trừu tượng (*Abstract Syntax Tree - AST* [6]) phục vụ cho việc vá mã nguồn động.

Thông qua việc hoàn thành các mục tiêu trên, luận văn hướng tới việc cung cấp một giải pháp toàn diện, không chỉ giải quyết các thách thức kỹ thuật hiện hữu trong lĩnh vực tự động hóa phát triển web, mà còn đóng góp một mô hình kiến trúc nơi công nghệ AI tạo sinh, sandbox ảo hóa phía máy khách và kỹ thuật thao tác mã nguồn trực quan hội tụ trong cùng một hệ thống nhất quán, phổ cập hóa năng lực lập trình hiện đại đến nhóm người dùng không chuyên.

1.3 PHƯƠNG PHÁP NGHIÊN CỨU

Để hoàn thành các mục tiêu đề ra, luận văn áp dụng các phương pháp nghiên cứu cụ thể sau:

Phương pháp nghiên cứu lý thuyết: Nghiên cứu các lý thuyết nền tảng và phân tích các tiêu chuẩn công nghệ tiên tiến liên quan. Các nội dung nghiên cứu bao gồm việc tìm hiểu nguyên lý kết xuất và quản lý trạng thái của framework Next.js 15 (App Router, React Server Components) kết hợp với cơ chế đồng bộ dữ liệu thời gian thực và mô hình lập lịch tác vụ hướng sự kiện; khảo sát công nghệ ảo hóa phía trình duyệt dựa trên WebAssembly, cơ chế khởi chạy máy chủ Node.js cục bộ ngay trong sandbox trình duyệt và các giao thức truyền tin an toàn Cross-Origin (API `postMessage`); nghiên cứu các mô hình ngôn ngữ lớn đa phương thức (Gemini 2.5 Flash [5], DeepSeek V4 [4]), cơ chế định tuyến (Routing) và kỹ nghệ viết prompt; nghiên cứu kỹ thuật phân tích và vá mã nguồn thông qua cây cú pháp trừu tượng (Abstract Syntax Tree - AST) [6] để thực hiện tính năng Visual Text Edit; đồng thời nghiên cứu lý thuyết giới hạn của các phương pháp tự động sinh mã giao diện trực tiếp từ bản thiết kế (Design-to-Code) làm cơ sở lý luận đối sánh.

Phương pháp thực nghiệm xây dựng hệ thống: Thiết kế và lập trình thực tế các cấu phần cốt lõi của hệ thống Nexastu. Quy trình thực nghiệm bao gồm việc khảo sát và lập trình mẫu thử nghiệm (prototyping) mô hình tiền đề Design-to-Code để nhận diện rào cản kỹ thuật của việc chuyển đổi giao diện từ bản vẽ hoặc mã nguồn tĩnh sang Next.js; lập trình bộ điều phối Orchestrator kết nối với LLM để bóc tách thông tin CV dựa trên Zod Schema, sinh mã nguồn Next.js/React và thiết lập các trạm lọc tự động Smoke Gate và Health Gate; tích hợp WebContainer để chạy máy chủ Next.js trực tiếp tại máy khách, đồng bộ hóa tệp tin thời gian thực qua Convex DB [7]; lập trình cầu nối Inspector Bridge an toàn chạy trong iframe của WebContainer để truyền nhận thông điệp qua API `postMessage`, hiện thực hóa tính năng visual highlighter và thuật toán sửa đổi mã nguồn trực tiếp tại client thông qua AST; và xây dựng cơ chế định tuyến thông minh Multimodal Router để phân phối linh hoạt giữa các mô hình ngôn ngữ lớn đích.

Phương pháp phân tích và đánh giá: Đo lường, phân tích và đánh giá hiệu năng của hệ thống Nexastu một cách khoa học. Các nội dung đánh giá bao gồm việc phân tích đối sánh các chỉ số về mức độ suy giảm cấu trúc component (Component Degradation) và sự thiếu hụt dữ liệu ngữ cảnh (Semantic Context Scarcity) của mô hình tiền đề Design-to-Code so với giải pháp Content-First đề xuất; đo lường định lượng tỷ lệ sinh mã chạy được thành công qua các vòng Smoke/Health Gate trên tập dữ liệu CV thử nghiệm, thống kê độ trễ phản hồi (Inference Latency) và tối ưu hóa chi phí gọi API; khảo sát định tính đối với tính năng chỉnh sửa trực quan Visual Edit qua cầu nối Inspector Bridge để tối ưu hóa độ mượt mà của giao diện Web-based IDE; và thực hiện so sánh đối chiếu hiệu năng sinh mã nguồn và khả năng hiểu hình ảnh giữa hai dòng mô hình dưới cùng điều kiện đầu vào.

1.4 PHẠM VI VÀ GIỚI HẠN NGHIÊN CỨU

Để đảm bảo tính tập trung và chiều sâu, các nghiên cứu và thực nghiệm được giới hạn trong một phạm vi xác định. Các giới hạn của luận văn được trình bày như sau:

Về phạm vi công nghệ sinh mã nguồn: Nền tảng Nexastu tập trung tối ưu hóa việc sinh mã nguồn sử dụng bộ ba công nghệ tiêu chuẩn công nghiệp hiện nay bao gồm framework Next.js 15 (App Router, sử dụng TypeScript), thư viện React 18 và giải pháp Tailwind CSS. Đề tài giới hạn không mở rộng sang các framework front-end khác (như Vue, Angular) hay các thư viện quản lý trạng thái phức tạp khác, nhằm tập trung giải quyết triệt để cấu trúc thành phần, tính năng tối ưu hóa SEO và tốc độ tải trang của giao diện portfolio Next.js. Đồng thời, việc sinh mã nguồn Next.js chỉ tập trung vào cấu trúc một trang web portfolio cá nhân (single-page portfolio) có tích hợp các phần giới thiệu bản thân, dự án, kỹ năng, và biểu mẫu liên hệ, không hỗ trợ sinh các hệ thống đa trang phức tạp có nghiệp vụ backend riêng biệt.

Về môi trường sandbox ảo hóa và tính tương thích trình duyệt: Giải pháp ảo hóa WebContainer chạy trên nền tảng WebAssembly đòi hỏi các tính năng bảo mật nâng cao từ trình duyệt máy khách (như `SharedArrayBuffer` và các giao thức bảo mật cao). Do đó, tính năng live preview và máy chủ Next.js ảo hóa chỉ hoạt động ổn định trên các trình duyệt máy tính cá nhân hiện đại (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari) chạy trên hệ điều hành máy tính (Windows, macOS, Linux), và không hỗ trợ các trình duyệt di động hoặc trình duyệt trên các thiết bị IoT có cấu hình thấp. Đồng thời, hệ thống yêu cầu cấu hình các HTTP Headers bảo mật thích hợp (`Cross-Origin-Opener-Policy` và `Cross-Origin-Embedder-Policy`) để kích hoạt sandbox WebAssembly.

Về định dạng tài liệu đầu vào và giới hạn xử lý ngôn ngữ: Hệ thống tập trung xử lý dữ liệu lý lịch cá nhân đầu vào dưới dạng tệp văn bản thô (txt, markdown) hoặc file PDF có chứa lớp văn bản (text layer) được viết bằng tiếng Anh hoặc tiếng Việt. Luận văn giới hạn không đi sâu xử lý các định dạng PDF quét dạng ảnh (scanned PDF), hình ảnh chụp CV bằng camera thông thường, hoặc các tài liệu phi cấu trúc phức tạp khác khi không có sự hỗ trợ của các API nhận dạng ký tự quang học (OCR) chuyên dụng từ bên thứ ba. Ngoài ra, việc bóc tách thông tin phụ thuộc hoàn toàn vào năng lực hiểu văn bản của mô hình ngôn ngữ lớn được chọn, do đó kết quả trích xuất có thể bị ảnh hưởng nếu CV đầu vào sử dụng các định dạng bảng biểu quá phức tạp hoặc định dạng cột không chuẩn.

Về giới hạn tương tác trực quan và đồng bộ mã nguồn: Tính năng chỉnh sửa trực quan (Visual Text Edit) thông qua cầu nối Inspector Bridge và giải pháp vá cây cú pháp trừu tượng AST chỉ áp dụng hiệu quả đối với các nút văn bản tĩnh (static text nodes) được khai báo trực tiếp trong các thẻ JSX của mã nguồn Next.js. Hệ thống chưa hỗ trợ chỉnh sửa trực tiếp các dữ liệu động được kết xuất từ các vòng lặp (như phương thức `map` trên một mảng dữ liệu) hoặc các dữ liệu được gọi động từ API bên ngoài tại thời điểm runtime. Đồng thời, tốc độ phản hồi giao diện xem trước phụ thuộc vào độ trễ đồng bộ mạng giữa cơ sở dữ liệu Convex DB và hệ thống tệp ảo WebContainer VFS.

1.5 BỐ CỤC BÁO CÁO

Luận văn được cấu trúc thành 5 chương chính như sau:

Chương 1. Tổng quan: Giới thiệu lý do lựa chọn đề tài (tính cấp thiết), mục tiêu, đối tượng và phạm vi nghiên cứu, phương pháp nghiên cứu, giới hạn và bố cục báo cáo.

Chương 2. Cơ sở lý thuyết và công nghệ liên quan: Trình bày chi tiết cơ sở khoa học của các công nghệ cốt lõi trong hệ thống như Next.js, cơ sở dữ liệu Convex, hệ thống hướng sự kiện Inngest, công nghệ ảo hóa WebContainer, lý thuyết về LLM, định tuyến mô hình và phân tích cú pháp AST.

Chương 3. Phân tích và thiết kế hệ thống: Chi tiết hóa kiến trúc hệ thống Nexastu, luồng xử lý dữ liệu của pipeline sinh mã nguồn, sơ đồ hoạt động của cầu nối Inspector và các thuật toán định tuyến mô hình.

Chương 4. Hiện thực hóa và kết quả thử nghiệm: Trình bày chi tiết mã nguồn hiện thực các module quan trọng, mô tả giao diện người dùng, kết quả kiểm thử tự động, các ca thử nghiệm thực tế và đánh giá hiệu năng hệ thống.

Chương 5. Kết luận và hướng phát triển: Đánh giá tổng kết các mục tiêu đã hoàn thành, nhận diện những điểm hạn chế còn tồn tại và đề xuất hướng mở rộng nghiên cứu trong tương lai.

Chương 2

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ LIÊN QUAN

Chương này trình bày nền tảng lý thuyết và các công nghệ cốt lõi làm tiền đề khoa học cho việc thiết kế và hiện thực hóa hệ thống Nexastu. Nội dung được tổ chức theo trình tự logic từ kiến trúc ứng dụng web nền tảng (Next.js, React), hạ tầng dữ liệu thời gian thực (Convex) và cơ chế xử lý tác vụ ngầm hướng sự kiện (Inngest), đến công nghệ ảo hóa trình duyệt mang tính đột phá (WebContainer API), lý thuyết về mô hình ngôn ngữ lớn và kỹ nghệ định tuyến đa phương thức, và cuối cùng là các kỹ thuật phân tích cú pháp trừu tượng (AST) phục vụ cho tính năng vá mã nguồn trực quan. Mỗi công nghệ được phân tích không chỉ ở khía cạnh nguyên lý hoạt động, mà còn được đặt trong bối cảnh vai trò cụ thể của nó trong kiến trúc tổng thể của hệ thống Nexastu, nhằm giúp người đọc hiểu rõ lý do lựa chọn và cách thức tích hợp các thành phần công nghệ này.

2.1 Kiến trúc ứng dụng Web hiện đại và Framework Next.js

Trong bối cảnh phát triển ứng dụng web hiện đại, việc lựa chọn kiến trúc và framework đóng vai trò quyết định đến hiệu năng, khả năng mở rộng cũng như trải nghiệm người dùng. Hệ thống Nexastu được xây dựng trên hệ sinh thái React và Next.js — giải pháp hàng đầu hiện nay cho việc phát triển frontend. Đây là nền tảng cốt lõi giúp hệ thống vận hành cơ chế App Router mượt mà, tạo tiền đề cho công cụ cấu trúc và chuẩn hóa mã nguồn tự động phía sau.

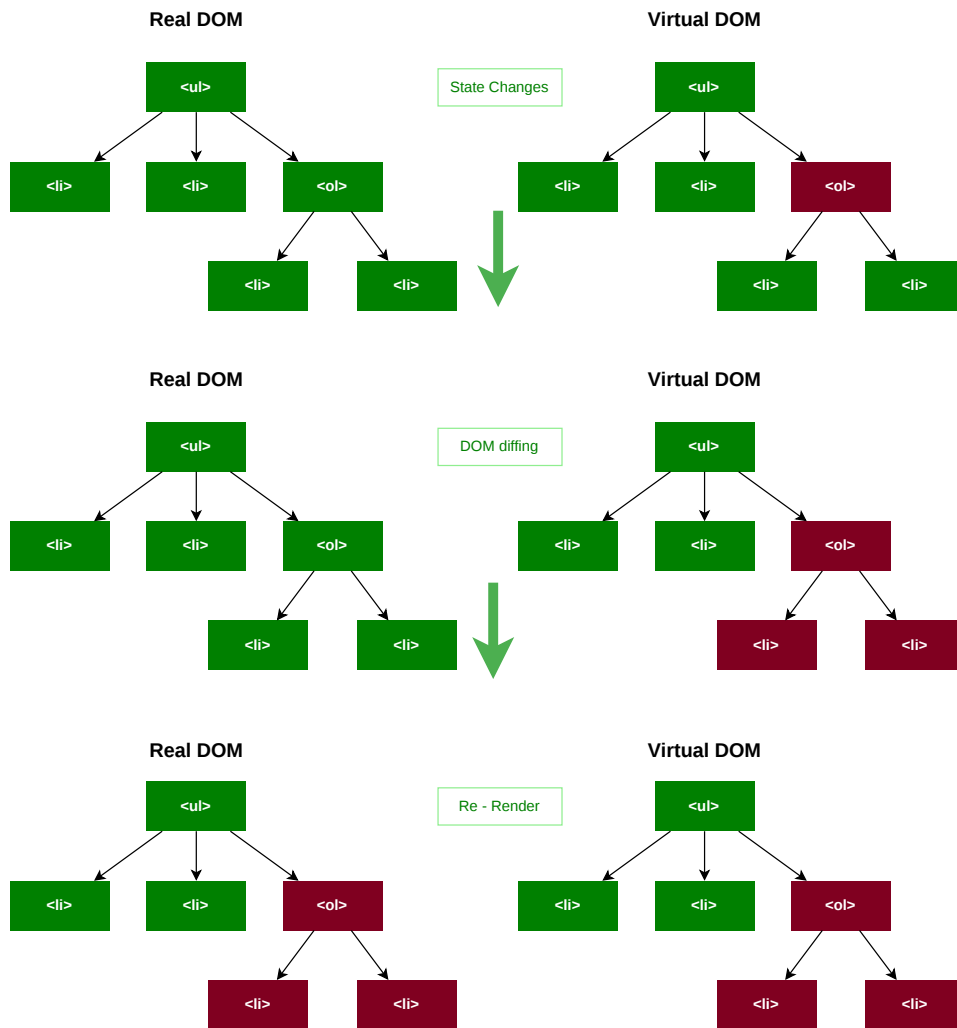
2.1.1 Tổng quan về React và cơ chế Rendering

React là một thư viện JavaScript mã nguồn mở được phát triển bởi Meta (Facebook) nhằm xây dựng giao diện người dùng (UI) dạng khai báo (declarative) và dựa trên các thành phần độc lập (component-based). Triết lý thiết kế cốt lõi của React xoay quanh khái niệm Virtual DOM — một biểu diễn bộ nhớ ảo nhẹ của cây DOM thực. Khi trạng thái (state) hoặc thuộc tính (props) của một thành phần thay đổi, React tự động tính toán và tạo ra một cây Virtual DOM mới.

Quá trình cập nhật giao diện sau đó được thực thi thông qua thuật toán so khớp gọi là **Reconciliation** (hay thuật toán Diffing). Thuật toán này duyệt qua cả hai cây Virtual DOM (cũ và mới) để tìm ra sự khác biệt tối thiểu với độ phức tạp thời gian đạt mức tối ưu $O(n)$ nhờ hai giả định cơ bản: (1) hai phần tử có kiểu khác nhau sẽ tạo ra hai cây khác nhau, và (2) lập trình viên có thể cung cấp thuộc tính **key** để chỉ ra phần tử nào ổn định qua các lần kết xuất. Sau khi xác định được các nút bị thay đổi (dirty nodes), React sẽ thực hiện gom lô (batching) các thay đổi này và chỉ cập nhật trực tiếp những phần tử thực sự thay đổi lên cây DOM thực (Real DOM) của trình duyệt. Cơ chế này giúp loại bỏ hoàn toàn các thao tác cập nhật DOM trực tiếp hoặc vẽ lại giao diện (repaint/reflow) không cần thiết trên trình duyệt, tối ưu hóa đáng kể hiệu năng kết xuất đồ họa đối với các ứng dụng web động.

Trong các phiên bản gần đây, đặc biệt là phiên bản React 19 tích hợp sâu với các framework hiện đại, một sự chuyển dịch mô hình lớn (paradigm shift) đã diễn ra với sự ra đời của **React Server Components (RSC)** và **Client Components** [2]:

- **React Server Components (RSC)**: Mặc định, các thành phần được thực thi trên máy chủ (server-side). Chúng cho phép truy cập trực tiếp vào cơ sở dữ liệu, hệ thống tệp tin và các tài nguyên backend mà không cần gửi thêm mã JavaScript của component đó xuống trình duyệt. Điều này giúp giảm thiểu đáng kể kích thước gói mã nguồn (bundle size) tải về phía client, tối ưu hiệu suất tải trang ban đầu. RSC đặc biệt phù hợp cho các thành phần hiển thị dữ liệu tĩnh hoặc nội dung được truy vấn từ cơ sở dữ liệu, bởi toàn bộ quá trình kết xuất diễn ra trên server và chỉ gửi kết quả HTML đã được kết xuất hoàn chỉnh về trình duyệt. Trong Nexastu, việc AI-Agent ưu tiên sinh mã nguồn portfolio của người dùng dưới dạng các RSC giúp giảm thiểu tối đa mã JavaScript tải về client, từ đó giải quyết triệt để điểm nghẽn về mã nguồn công kênh và tối ưu hóa điểm SEO/hiệu năng tải trang vượt trội so với các nền tảng No-Code truyền thống.
- **Client Components**: Được đánh dấu bằng chỉ thị `"use client"` ở đầu tệp tin. Các thành phần này chạy trực tiếp trên trình duyệt, hỗ trợ tương tác động của người dùng như các sự kiện nhấp chuột, quản lý trạng thái cục bộ thông qua `useState`, `useEffect`, và truy cập vào các API của trình duyệt (DOM). Trong hệ thống Nexastu, các Client Components đóng vai trò quan trọng trong việc xử lý tương tác thời gian thực của giao diện IDE, bao gồm trình soạn thảo mã nguồn, cửa sổ xem trước (preview) và bộ Inspector trực quan.



Hình 2.1: Cơ chế so khớp và cập nhật giao diện thông qua Virtual DOM trong React.

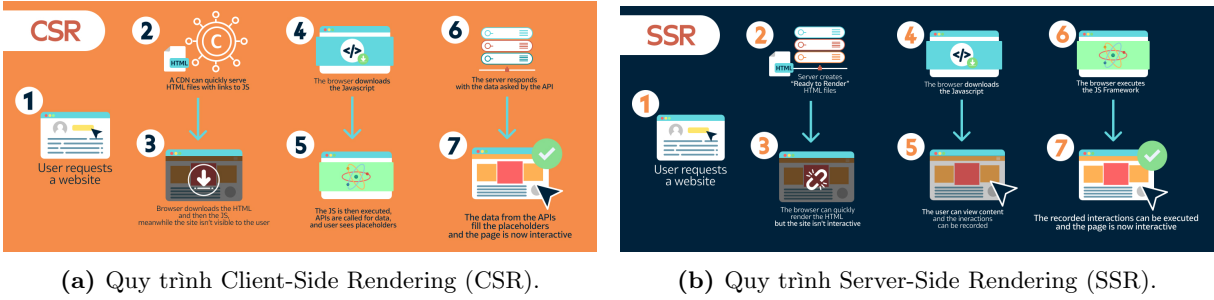
2.1.2 Framework Next.js và kiến trúc App Router

Next.js là một framework React dành cho môi trường sản xuất (production) được phát triển bởi Vercel [1]. Next.js cung cấp các giải pháp tối ưu hóa SEO, định tuyến tự động, và hỗ trợ các mô hình kết xuất (rendering) đa dạng bao gồm SSR (Server-Side Rendering), SSG (Static Site Generation), và ISR (Incremental Static Regeneration). Sự đa dạng này cho phép lập trình viên lựa chọn chiến lược kết xuất phù hợp nhất cho từng trang hoặc từng thành phần, tối ưu hóa cả trải nghiệm người dùng lẫn hiệu năng hệ thống.

Kiến trúc **App Router** (giới thiệu từ Next.js 13 và hoàn thiện ở Next.js 15) sử dụng thư mục `app/` để tự động ánh xạ cấu trúc thư mục thành các đường dẫn (routing) của ứng dụng. App Router tận dụng tối đa mô hình RSC của React nhằm tối ưu hóa hiệu năng bằng cách chia nhỏ trang web thành các phần kết xuất tĩnh và động. Hai cơ chế quan trọng của App Router bao gồm:

- **Streaming:** Cho phép máy chủ gửi từng phần của trang HTML về trình duyệt ngay khi chúng sẵn sàng, thay vì chờ toàn bộ trang được kết xuất xong. Điều này giúp người dùng nhìn thấy nội dung nhanh hơn, cải thiện đáng kể chỉ số Time to First Byte (TTFB) và First Contentful Paint (FCP).
- **Suspense:** Cung cấp cơ chế hiển thị trạng thái tải (loading state) cho các thành phần chưa sẵn sàng, cho phép giao diện phản hồi tức thì trong khi dữ liệu đang được truy vấn từ server.

Bên cạnh đó, **Server Actions** là một tính năng đột phá của Next.js cho phép định nghĩa các hàm chạy hoàn toàn trên server nhưng có thể được gọi trực tiếp từ giao diện Client Components giống như một hàm JavaScript thông thường. Cơ chế này loại bỏ nhu cầu tự viết các API Endpoint trung gian, mang lại tính an toàn kiểu dữ liệu cao (Type Safety) từ đầu cuối đến đầu cuối (end-to-end), và đơn giản hóa luồng giao tiếp dữ liệu nhờ cơ chế bảo mật tích hợp sẵn trên môi trường thực thi server. Trong Nexastu, Server Actions được sử dụng để xử lý các tác vụ như tải tệp CV lên hệ thống, kích hoạt pipeline sinh mã nguồn và thực hiện các thao tác và mã nguồn AST từ tương tác Inspector trên giao diện một cách nhanh chóng, an toàn mà không cần tạo các route API RESTful thủ công.



Hình 2.2: So sánh quy trình kết xuất và tải trang giữa CSR và SSR.

2.2 Cơ sở dữ liệu thời gian thực Convex

Đối với một môi trường phát triển tích hợp trực tuyến trên nền web (Web-based IDE) như Nexastu, nơi mã nguồn và trạng thái hệ thống liên tục được sinh ra, chỉnh sửa và đồng bộ hóa song song giữa tác nhân trí tuệ nhân tạo (AI-Agent) và người dùng cuối, việc thiết kế hạ tầng dữ liệu đáp ứng xử lý đồng thời theo thời gian thực (*real-time concurrency*) và đảm bảo tính nhất quán giao dịch (*transactional consistency*) là điều kiện tiên quyết. Convex [7] được lựa chọn làm cơ sở dữ liệu cốt lõi cho Nexastu nhằm giải quyết các giới hạn của kiến trúc cơ sở dữ liệu truyền thống.

2.2.1 Luận lý lựa chọn công nghệ và phân tích đối chiếu

Trong các kiến trúc hệ thống web truyền thống, để hiện thực hóa tính năng đồng bộ thời gian thực, các kỹ sư thường kết hợp hệ quản trị cơ sở dữ liệu quan hệ (như PostgreSQL qua Prisma ORM)

với một máy chủ kết nối WebSocket độc lập (như Socket.io) hoặc dịch vụ trung gian (như Pusher). Tuy nhiên, mô hình này phát sinh hai nhược điểm lớn trong ngữ cảnh hệ thống sinh mã:

- **Độ phức tạp hạ tầng:** Lập trình viên phải tự quản lý việc duy trì kết nối WebSocket, đồng bộ hóa trạng thái bộ nhớ đệm (caching), và thiết lập cơ chế phát tán sự kiện (event publish/subscribe) khi dữ liệu thay đổi. Điều này tạo ra nhiều điểm lỗi đơn lẻ (*single point of failure*) và làm tăng độ trễ mạng do dữ liệu phải đi qua nhiều lớp trung gian.
- **Xung đột dữ liệu (*Race Conditions*):** Khi tác nhân AI-Agent và người dùng cùng ghi đè lên các tệp tin dự án trong thời gian thực, các truy vấn ghi truyền thống không có cơ chế khóa tối ưu sẽ dẫn đến xung đột ghi và mất mát dữ liệu của cấu trúc thư mục hệ thống tệp ảo (Virtual File System – VFS).

Firebase Firestore là một giải pháp cơ sở dữ liệu phổ biến khác hỗ trợ tính năng đồng bộ thời gian thực, nhưng nó gặp hạn chế lớn trong việc xử lý các giao dịch nguyên tử (*atomic transactions*) phức tạp trên nhiều bảng dữ liệu và không hỗ trợ thực thi trực tiếp các luận lý phía máy chủ (backend logic) chuyên sâu (như tích hợp Inngest hay định tuyến mô hình ngôn ngữ lớn) trong một môi trường thực thi (runtime) JavaScript an toàn và có độ trễ thấp.

Để làm rõ tính ưu việt và sự phù hợp của hệ thống Convex DB trong kiến trúc Nexastu, một phân tích đối sánh chi tiết giữa các mô hình hạ tầng dữ liệu đã được thực hiện và được trình bày trong Bảng 2.1.

Bảng 2.1: So sánh các giải pháp hạ tầng cơ sở dữ liệu cho hệ thống Web-IDE

Tiêu chí	Convex Database	PostgreSQL + Socket.io	Firebase Firestore
Cơ chế đồng bộ	Đẩy tự động (Reactive Subscriptions)	Thủ công qua WebSocket độc lập	Đẩy tự động (Snapshot Listeners)
Tính giao dịch	Serializable (Mức cô lập ACID cao nhất tự động)	Mặc định Read Committed (Có thể cấu hình Serializable thủ công)	Hỗ trợ ACID đa tài liệu (Giới hạn tần suất ghi trên một tài liệu)
Quản lý trạng thái	Tự động đồng bộ trạng thái giữa client và server	Tích hợp thủ công thư viện cache và đồng bộ (React Query, Redis)	Tự động qua Client SDK
Môi trường chạy	Serverless Runtime (JavaScript/TypeScript)	Máy chủ Node.js liên tục (Always-on)	Dịch vụ Cloud NoSQL (Trễ Cold Start nếu gọi qua Cloud Functions)
An toàn kiểu dữ liệu	Đầu đến cuối (End-to-end) nhờ tự động sinh kiểu từ Schema	Từng phần (Cấu hình thủ công qua ORM và DTO)	Hạn chế (NoSQL động, không ràng buộc schema ở DB)

2.2.2 Cơ chế đăng ký và truy vấn phản ứng (Reactive Queries)

Thay vì sử dụng mô hình kéo dữ liệu truyền thống (pull-based) thông qua REST API hoặc truy vấn SQL, Convex hoạt động dựa trên cơ chế đăng ký phản ứng (Reactive Subscriptions) thông qua giao thức WebSocket dài hạn. Khi một thành phần phía máy khách (Client Component) gọi truy vấn dữ liệu bằng hook `useQuery`, Convex thiết lập một kết nối WebSocket liên kết trực tiếp trình duyệt với cơ chế truy vấn điện toán đám mây (Cloud Query Engine) [7].

Bên dưới tầng cơ sở dữ liệu, Convex xây dựng một đồ thị phụ thuộc (dependency graph) để quản lý các truy vấn đang hoạt động. Cụ thể, khi một hàm truy vấn (Query function) được thực thi trên máy chủ Convex, hệ thống tự động ghi lại tất cả các bản ghi hoặc tập hợp dữ liệu mà hàm đó đã truy cập (gọi là tập hợp đọc – read-set). Khi một giao dịch ghi (Mutation) diễn ra và thay đổi dữ liệu, nó tạo ra một tập hợp ghi (write-set) chứa danh sách các bản ghi bị sửa đổi. Bộ điều phối Convex sẽ đối chiếu hai tập hợp này; nếu phát hiện sự giao thoa giữa write-set of mutation và read-set của một truy vấn đang hoạt động, hệ thống sẽ tự động lập lịch tính toán lại (re-compute) kết quả của truy vấn đó trên server, sau đó chỉ đẩy phần dữ liệu chênh lệch (delta) hoặc kết quả mới về client qua kết nối WebSocket. Cơ chế này giúp giao diện người dùng tự động phản ánh trạng thái mới nhất mà không cần kỹ thuật thăm dò (polling) định kỳ hay tải lại trang, tối ưu hóa độ trễ phản hồi và tiết kiệm băng thông mạng.

Trong Nexastu, cơ chế này đóng vai trò quyết định tính mượt mà của môi trường phát triển: ngay khi tác nhân AI-Agent hoàn thành việc sinh hoặc vá mã nguồn và cập nhật vào cơ sở dữ liệu Convex (Convex DB), giao diện IDE của người dùng sẽ nhận được bản cập nhật tức thì, kích hoạt luồng đồng bộ sang hệ thống tệp ảo WebContainer VFS và cập nhật trực tiếp lên khung xem trước trực tiếp (Live Preview) mà không cần bất kỳ thao tác làm mới thủ công nào từ phía người dùng.

2.2.3 Đảm bảo tính nhất quán giao dịch qua Mutations và Actions

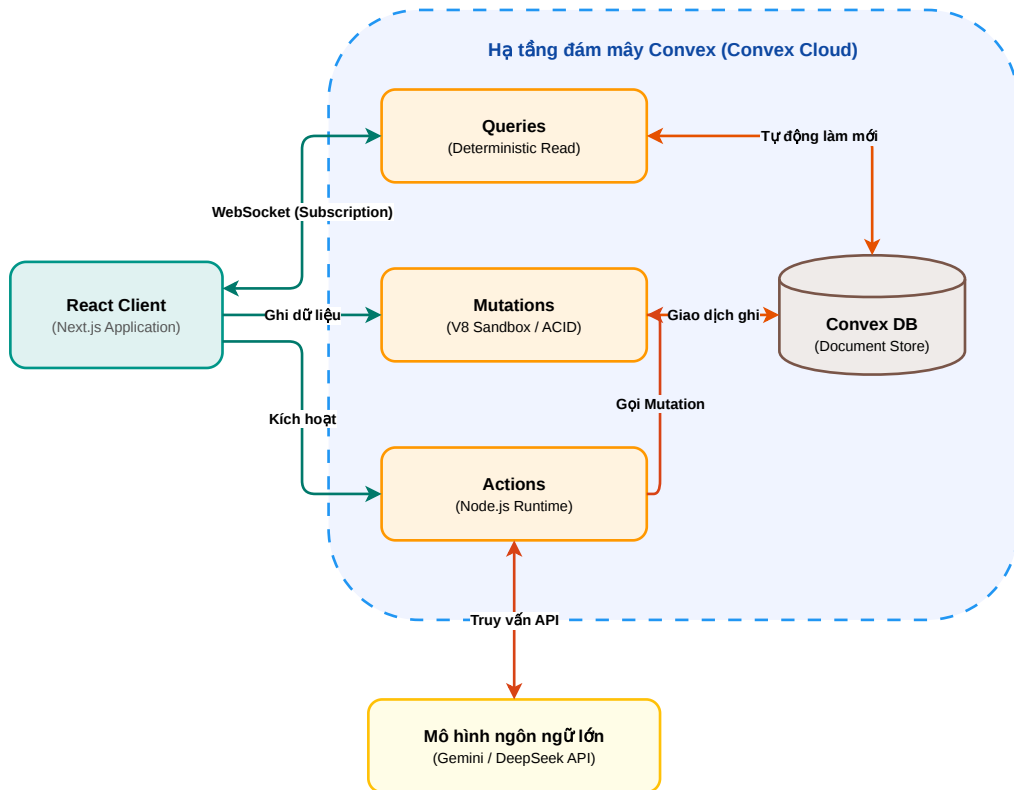
Để xử lý các thao tác ghi đồng thời từ nhiều nguồn mà không xảy ra xung đột dữ liệu (race conditions), Convex phân tách nghiêm ngặt hai môi trường thực thi:

- **Mutations:** Chạy trên một môi trường hộp cát JavaScript (JavaScript Sandbox) sử dụng động cơ V8 (V8 Engine) biệt lập và hoàn toàn mang tính xác định (*deterministic*). Để đảm bảo tính nhất quán giao dịch ở cấp độ cao nhất (*Serializable Isolation*), Convex cấm tất cả các hành vi bất định (*non-deterministic*) bên trong Mutations, bao gồm việc gọi các yêu cầu mạng bên ngoài (*HTTP fetch*), tạo số ngẫu nhiên (`Math.random()`), hay lấy thời gian hệ thống trực tiếp. Khi thực

thì, nếu một *Mutation* bị lỗi giữa chừng, toàn bộ các thay đổi trước đó sẽ được tự động hoàn tác (*rollback*) để tránh dữ liệu mờ côi. Điều này đảm bảo rằng các cấu trúc thư mục phức tạp của website portfolio (gồm hàng chục tệp tin liên kết với nhau) luôn được ghi thành công trọn vẹn hoặc không ghi gì cả.

- **Actions:** Chạy trên môi trường Node.js đầy đủ, cho phép thực hiện các thao tác bất định (*non-deterministic*) như gọi các mô hình ngôn ngữ lớn (LLM) (Gemini, DeepSeek), thanh toán qua cổng PayOS hoặc gọi các liên kết phản hồi (webhook) bên ngoài. Tuy nhiên, để bảo vệ tính toàn vẹn của dữ liệu, Actions không có quyền tương tác trực tiếp với cơ sở dữ liệu; muốn cập nhật trạng thái dữ liệu, Actions bắt buộc phải gọi đến một Mutation tương ứng để ghi dữ liệu một cách an toàn.

Sự phân tách này được minh họa qua sơ đồ kiến trúc truyền thông và xử lý dữ liệu của Convex trong hệ thống Nexastu (Hình 2.3). Cụ thể, sơ đồ chỉ ra rằng React Client gửi yêu cầu đọc dữ liệu trực tiếp tới Queries qua kết nối WebSocket, trong khi các yêu cầu thay đổi dữ liệu hoặc kích hoạt luồng AI được chuyển tương ứng tới Mutations và Actions. Đặc biệt, Actions khi cần cập nhật trạng thái dữ liệu bắt buộc phải gọi sang Mutation (đường "Gọi Mutation") thay vì tương tác trực tiếp với Convex DB, đảm bảo tính toàn vẹn và mức cô lập giao dịch cao nhất.



Hình 2.3: Sơ đồ kiến trúc xử lý dữ liệu và đồng bộ hóa thời gian thực của Convex Database.

2.2.4 Cấu trúc Schema và an toàn kiểu dữ liệu (Type Safety)

Khác với các cơ sở dữ liệu NoSQL thông thường cho phép ghi dữ liệu tự do dễ dẫn đến sai cấu trúc, Convex hỗ trợ kiểm soát tính toàn vẹn của dữ liệu thông qua cơ chế kết hợp hai lớp an toàn:

- **Xác thực động (Runtime Validation):** Convex sử dụng thư viện xác thực tích hợp để định nghĩa sơ đồ dữ liệu (Schema) nghiêm ngặt cho từng bảng bằng các hàm xác thực động (như `v.string()`, `v.number()`, `v.id("table")`). Bất kỳ thao tác ghi nào vi phạm Schema sẽ lập tức bị chặn ngay ở mức nhân cơ sở dữ liệu trước khi được lưu xuống đĩa.
- **An toàn kiểu lúc biên dịch (Compile-time Type Safety):** Từ sơ đồ dữ liệu (Schema) đã định nghĩa, Convex tự động tạo kiểu (codegen) các kiểu TypeScript tương ứng. Các kiểu dữ liệu này được chia sẻ đồng bộ giữa frontend (Next.js) và backend (Convex functions). Nếu có bất kỳ sự thay đổi Schema nào ở database, trình biên dịch TypeScript sẽ báo lỗi ngay lập tức tại các dòng mã client hoặc server không khớp kiểu, giúp phát hiện sớm các lỗi cú pháp dữ liệu từ giai đoạn phát triển và giảm thiểu tối đa các lỗi sự cố dừng (crash) ứng dụng lúc thực thi (runtime).

Ví dụ, thực thể `files` lưu trữ tệp tin dự án được ràng buộc chặt chẽ với Schema gồm: `projectId` đại diện cho khóa ngoại liên kết bảng dự án, `path` lưu đường dẫn tệp (chuỗi văn bản chuẩn), và `content` lưu mã nguồn tệp tin. Cơ chế này đảm bảo tác nhân AI-Agent không thể ghi đè các cấu trúc dữ liệu không hợp lệ hoặc thiếu trường bắt buộc vào hệ thống tệp tin của người dùng. Thiết kế sơ đồ chi tiết và cấu trúc cụ thể của các thực thể dữ liệu này được đặc tả chi tiết tại Mục 3.6 (Chương 3).

2.3 Kiến trúc xử lý tác vụ ngầm hướng sự kiện Inngest

Một trong những thách thức kỹ thuật lớn nhất của hệ thống Nexastu là quản lý quy trình sinh mã nguồn tự động — một *pipeline* phức tạp bao gồm nhiều bước tuần tự (bóc tách CV, sinh mã, kiểm tra cú pháp, sửa lỗi, đồng bộ tệp tin), mỗi bước có thể mất từ vài giây đến vài phút. Nếu chạy trực tiếp trên luồng *HTTP request* thông thường, quy trình này sẽ nhanh chóng vượt quá giới hạn thời gian phản hồi (*timeout*) và làm nghẽn giao diện người dùng. Inngest được lựa chọn để giải quyết triệt để thách thức này.

Inngest là một nền tảng xử lý tác vụ ngầm (*background jobs*) hướng sự kiện (*event-driven*) không máy chủ (*serverless*) [8]. Khác với các hệ thống hàng đợi truyền thống (như *Redis Queue*, *BullMQ*) đòi

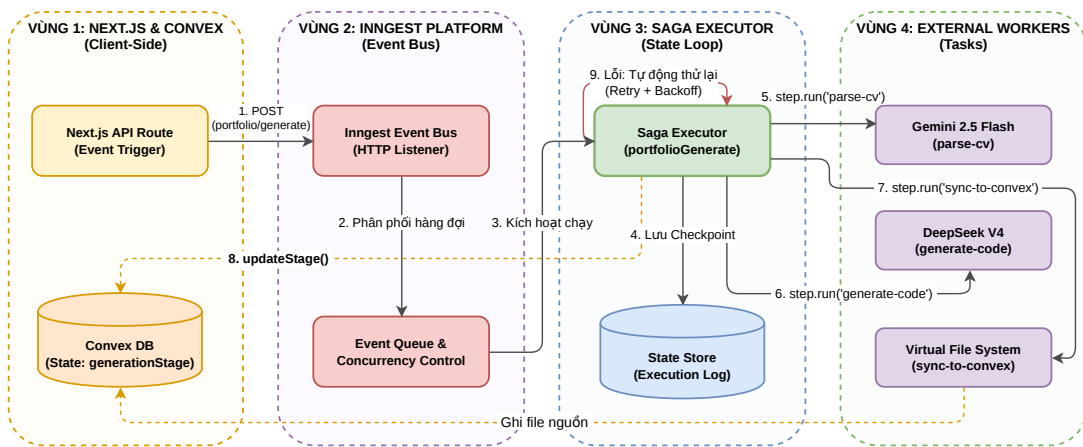
hỏi tự quản lý hạ tầng, Inngest hoạt động hoàn toàn *serverless* và tích hợp liền mạch với kiến trúc Next.js App Router.

2.3.1 Nguyên lý Step Functions và Ứng dụng Saga Pattern trong điều phối tác vụ

Inngest cho phép chia nhỏ một quy trình làm việc phức tạp thành các bước nhỏ độc lập được thực thi tuần tự hoặc song song (thông qua `step.run`, `step.wait`). Đặc tính nổi bật của *Step Functions* là khả năng lưu giữ trạng thái (*state preservation*) giữa các bước: mỗi bước hoàn thành sẽ tự động lưu kết quả vào hệ thống, và nếu một bước tiếp theo bị lỗi (ví dụ *API LLM* bị quá tải hoặc quá hạn ngạch cuộc gọi), Inngest sẽ tự động thực hiện cơ chế thử lại (*retry*) thông minh cho riêng bước đó mà không phải chạy lại toàn bộ quy trình sinh mã từ đầu.

Về mặt kiến trúc phần mềm, cơ chế điều phối này hiện thực hóa mô hình *Saga Pattern* dạng *Orchestrator-based Saga* [9] trong hệ thống phân tán. Khi một quy trình sinh mã phức tạp cần gọi nhiều dịch vụ bất đồng bộ (Gemini API, DeepSeek API, kiểm thử cú pháp), Inngest đóng vai trò là bộ điều phối trung tâm (*Orchestrator*). Bộ điều phối này ghi lại lịch sử thực thi của từng bước vào nhật ký trạng thái (*execution log*). Nếu bước sinh mã gặp sự cố mạng, Inngest sẽ tạm dừng và kích hoạt cơ chế thử lại theo các chiến lược: thử lại tuyến tính (*linear backoff*), hoặc theo cấp số nhân (*exponential backoff*) có nhiễu ngẫu nhiên (*jitter*) nhằm tránh hiện tượng bão yêu cầu mạng (*thundering herd*). Nhờ đó, các bước đã thực hiện thành công phía trước (như bóc tách lý lịch CV hoàn thành ở bước 1) sẽ không bị chạy lại, giúp tiết kiệm tối đa tài nguyên tính toán và chi phí gọi các API LLM đắt đỏ.

Để trực quan hóa nguyên lý này, sơ đồ kiến trúc điều phối giao dịch phân tán của Inngest được mô tả chi tiết tại Hình 2.4.



Hình 2.4: Sơ đồ kiến trúc điều phối giao dịch phân tán (Saga Orchestrator) của Inngest trong hệ thống Nexastu.

Nhằm đảm bảo tính độc lập với chi tiết triển khai thực tế (sẽ được làm rõ tại Chương 4), cấu trúc các bước thực thi và cơ chế chuyển đổi trạng thái của Saga Orchestrator thông qua Inngest Step Functions được mô tả chi tiết trong Bảng 2.2.

Bảng 2.2: Các giai đoạn thực thi trong quy trình điều phối Saga bằng Inngest Step Functions.

Bước thực thi	Tác nhân / Công nghệ	Nội dung xử lý và Vai trò	Trạng thái lưu trữ
1. Phân tách lý lịch (<code>parse-cv</code>)	Gemini API	Tiếp nhận tệp tin CV định dạng PDF từ yêu cầu của người dùng, phân tích ngữ nghĩa và trích xuất dữ liệu bán cấu trúc sang đối tượng JSON chuẩn hóa.	<code>cvData</code> (Hồ sơ cá nhân, học vấn, kinh nghiệm làm việc)
2. Sinh mã nguồn (<code>generate-code</code>)	DeepSeek API	Dựa trên dữ liệu JSON đã cấu trúc, sinh ra mã nguồn các thành phần giao diện React và Next.js tương thích với kiến trúc portfolio.	<code>files</code> (Cây thư mục chứa mã nguồn thô ban đầu)
3. Kiểm thử cú pháp (<code>smoke-gate</code>)	Trình phân tích cú pháp AST (<code>ts-morph</code>) & Trình biên dịch ảo	Phân tích tính cú pháp mã nguồn sinh ra, phát hiện các lỗi cú pháp TypeScript, JSX, hoặc CSS phổ biến, tự động sửa đổi và hoàn thiện.	<code>checkedFiles</code> (Mã nguồn sạch, sẵn sàng nạp vào Web-Container)

2.3.2 Kiểm soát đồng thời và Đồng bộ hóa trạng thái bất đồng bộ

Hệ thống Nexastu cho phép cấu hình giới hạn số lượng tác vụ xử lý đồng thời (*concurrency limit*) cho từng hàm hoặc nhóm người dùng. Điều này cực kỳ quan trọng đối với các tác vụ tích hợp AI, giúp tránh làm quá tải máy chủ Sandbox và ngăn chặn hiện tượng vi phạm giới hạn tần suất gọi (*rate limit*) của các nhà cung cấp mô hình ngôn ngữ lớn (Google/DeepSeek). Khi số lượng người dùng tăng đột biến, Inngest sẽ tự động xếp hàng các sự kiện vượt ngưỡng và phân phối tuần tự ngay khi có tài nguyên trống.

Inngest hoạt động bất đồng bộ bên dưới nền nhưng liên tục gửi các tín hiệu cập nhật trạng thái của tiến trình (ví dụ: *parsing_input*, *generating_code*, *compiling*, *finalized*) về Convex DB. Nhờ cơ chế *Reactive Queries* của Convex, giao diện người dùng sẽ cập nhật trạng thái tiến trình sinh mã theo thời gian thực tương ứng mà không cần phải thực hiện các cơ chế thăm dò (*polling*) thủ công từ *client*. Sự kết hợp giữa Inngest và Convex tạo nên một kiến trúc hướng sự kiện hoàn chỉnh, trong đó mỗi bước hoàn thành của *pipeline* đều được phản ánh tức thì trên giao diện *IDE*. Chi tiết về sơ đồ tuần tự mô tả luồng xử lý và tương tác của pipeline sinh mã nguồn này được thiết kế và trình bày tại Hình 3.2 (Chương 3).

2.4 Công nghệ ảo hóa trình duyệt WebContainer API

Một trong những đóng góp kỹ thuật quan trọng nhất của Nexastu là việc dịch chuyển toàn bộ môi trường biên dịch và thực thi mã nguồn web từ máy chủ trung tâm về phía trình duyệt của người dùng cuối. Quyết định kiến trúc này được thúc đẩy bởi hai yếu tố: (1) giảm thiểu chi phí tài nguyên máy chủ khi phục vụ hàng ngàn người dùng đồng thời, và (2) loại bỏ hoàn toàn nguy cơ thực thi mã từ xa độc hại (RCE) trên hạ tầng trung tâm. WebContainer API của StackBlitz [3] là công nghệ cho phép hiện thực hóa giải pháp này.

2.4.1 Nguyên lý ảo hóa Node.js trên WebAssembly Sandbox và Cơ chế bảo mật đa luồng

WebContainer hoạt động dựa trên việc biên dịch nhân hệ điều hành tối giản và nhân V8 JavaScript sang định dạng nhị phân WebAssembly (Wasm). Trình duyệt client sẽ thực thi trực tiếp mã này trong một môi trường sandbox cô lập hoàn toàn. Để đạt được hiệu năng thực thi gần tương đương với phần cứng vật lý, WebContainer sử dụng kiến trúc đa luồng (multi-threading) trong WebAssembly, cho phép phân phối tải công việc của môi trường Node.js (như biên dịch mã nguồn Next.js, giải nén gói tin từ npm) lên nhiều lõi CPU của thiết bị người dùng.

Để thực hiện chia sẻ bộ nhớ bất đồng bộ với độ trễ cực thấp giữa các Web Workers đại diện cho các luồng hệ thống, WebContainer bắt buộc phải sử dụng đối tượng bộ nhớ dùng chung SharedArrayBuffer của trình duyệt. Tuy nhiên, sau sự kiện phát hiện lỗ hổng bảo mật kênh bên Spectre vào năm 2018 — lỗ hổng cho phép kẻ tấn công đọc dữ liệu từ các vùng nhớ khác trong cùng quy trình của trình duyệt bằng cách sử dụng các bộ định thời độ phân giải cao — các nhà phát triển trình duyệt đã vô hiệu hóa mặc định SharedArrayBuffer để bảo vệ người dùng.

Để kích hoạt lại tính năng này một cách an toàn, hệ thống Nexastu bắt buộc phải thiết lập môi trường cô lập nguồn gốc chéo (Cross-Origin Isolation) thông qua việc cấu hình hai tiêu đề phản hồi HTTP (HTTP Response Headers) đặc biệt trên máy chủ lưu trữ chính:

`Cross-Origin-Opener-Policy: same-origin`

`Cross-Origin-Embedder-Policy: require-corp`

Tiêu đề Cross-Origin-Opener-Policy (COOP) với giá trị same-origin đảm bảo rằng tài liệu hiện tại sẽ được tải trong một nhóm ngữ cảnh duyệt web (browsing context group) hoàn toàn mới, cách ly với

các cửa sổ hoặc tab khác, từ đó ngăn chặn việc truy cập thông tin chéo luồng. Trong khi đó, tiêu đề Cross-Origin-Embedder-Policy (COEP) với giá trị `require-corp` bắt buộc trình duyệt chỉ được phép tải các tài nguyên từ nguồn gốc khác (cross-origin) khi các tài nguyên đó có khai báo đồng ý tường minh thông qua tiêu đề Cross-Origin Resource Sharing (CORS) hoặc Cross-Origin Resource Policy (CORP). Cơ chế bảo mật kép này thiết lập ranh giới cô lập không gian địa chỉ bộ nhớ, ngăn chặn triệt để các cuộc tấn công kênh bên như Spectre truy cập trái phép vào `SharedArrayBuffer` của WebContainer Sandbox, đảm bảo tính toàn vẹn và an toàn thông tin tối đa cho người dùng cuối.

2.4.2 Hệ thống tệp ảo (VFS) và Định tuyến mạng ảo hóa trình duyệt

Để quản lý cấu trúc mã nguồn Next.js trong môi trường trình duyệt cô lập, WebContainer duy trì Hệ thống tệp ảo (Virtual File System - VFS) trên bộ nhớ RAM [10]. VFS mô phỏng cấu trúc cây thư mục Unix và hỗ trợ các thao tác đọc, ghi qua việc ánh xạ trực tiếp các lời gọi từ thư viện `fs` của Node.js chuẩn. Việc thực thi hoàn toàn trên bộ nhớ RAM giúp loại bỏ độ trễ truy xuất đĩa vật lý, đồng thời hỗ trợ cơ chế thay thế mô-đun nóng (HMR) của Next.js phản hồi tức thì trước thay đổi từ giao diện soạn thảo.

Bên cạnh lưu trữ, thách thức tiếp theo là ảo hóa mạng. Khi các tiến trình CLI như `npm run dev` khởi tạo máy chủ phát triển Next.js trong Wasm Sandbox để lắng nghe trên một cổng mạng ảo (ví dụ: cổng 3000), do trình duyệt không thể mở kết nối TCP vật lý, WebContainer sử dụng cơ chế lắng nghe cổng ảo (`on('port', ...)`) kết hợp Service Worker để đánh chặn và định tuyến lưu lượng.

Cụ thể, WebContainer đăng ký một lớp chặn mạng ở tầng Service Worker trình duyệt. Khi khung xem trước (Live Preview iframe) gửi yêu cầu truy cập đến địa chỉ cục bộ ảo (ví dụ: cổng 3000), Service Worker sẽ đánh chặn yêu cầu HTTP trước khi nó được gửi ra card mạng vật lý. Sau đó, Service Worker chuyển dữ liệu yêu cầu thành thông điệp bất đồng bộ gửi đến luồng xử lý chính của WebContainer qua kênh `postMessage` hoặc bộ nhớ dùng chung. Máy chủ web ảo chạy trong hộp cát Node.js Sandbox tiếp nhận, xử lý, tạo nội dung HTML/CSS/JS tương ứng và gửi phản hồi ngược lại cho Service Worker. Cuối cùng, Service Worker định tuyến lại phản hồi HTTP chuẩn trả về cho iframe hiển thị. Cơ chế định tuyến khép kín này giúp bảo vệ toàn bộ luồng truyền thông và tối ưu tốc độ tải trang xem trước do không phát sinh dữ liệu ra Internet bên ngoài.

2.4.3 So sánh WebContainer với các giải pháp ảo hóa truyền thống

Để lựa chọn được giải pháp thiết kế môi trường biên dịch tối ưu cho hệ thống Nexastu, việc thực hiện phân tích và đối sánh kỹ thuật giữa các giải pháp ảo hóa là cực kỳ quan trọng. Hiện nay, các hệ thống phát triển trực tuyến thường được xây dựng dựa trên ba mô hình cốt lõi: công nghệ ảo hóa trình duyệt (WebContainer API), công nghệ đóng gói container phía máy chủ (Docker Containers), và cơ chế biên dịch trực tiếp trên hệ điều hành máy chủ (Server-side Compilation). Các giải pháp này được so sánh và đánh giá dựa trên các tiêu chí về vị trí thực thi, chi phí hạ tầng, mức độ bảo mật chống thực thi mã độc (RCE), khả năng mở rộng hệ thống, độ trễ và khả năng hỗ trợ các thư viện phát triển, như được tổng hợp chi tiết trong Bảng 2.3.

Bảng 2.3: So sánh các giải pháp ảo hóa môi trường thực thi mã nguồn

Tiêu chí	WebContainer API	Docker Container	Server-side Compilation
Vị trí thực thi	Trình duyệt client (WebAssembly)	Máy chủ (Linux kernel)	Máy chủ (Native OS)
Chi phí hạ tầng	Không (sử dụng tài nguyên client)	Cao (yêu cầu máy chủ riêng cho mỗi phiên)	Rất cao (yêu cầu cụm máy chủ lớn)
Bảo mật RCE	Rất cao (Cỗ lập hoàn toàn trong hộp cát WebAssembly phía Client)	Trung bình (cần cấu hình chính sách nghiêm ngặt)	Nguy cơ cao (mã nguồn chạy trực tiếp trên server)
Khả năng mở rộng	Tự động mở rộng theo số lượng thiết bị người dùng (phần tập trung)	Hạn chế (phụ thuộc vào năng lực máy chủ trung tâm)	Hạn chế (phụ thuộc vào năng lực máy chủ trung tâm)
Độ trễ khởi tạo	Trung bình (phụ thuộc vào tốc độ tải tài nguyên WebAssembly lần đầu)	Cao (thời gian khởi tạo container)	Thấp (môi trường luôn sẵn sàng)
Hỗ trợ Framework	Node.js, Next.js	Đầy đủ (mọi ngôn ngữ và framework)	Đầy đủ (mọi ngôn ngữ và framework)

Dựa trên kết quả đối sánh tại Bảng 2.3, có thể rút ra các phân tích kiến trúc quan trọng sau:

- **Hiệu quả hạ tầng và khả năng mở rộng:** WebContainer API thể hiện ưu thế vượt trội khi dịch chuyển toàn bộ gánh nặng tính toán (Next.js compilation, npm package extraction) về thiết bị của người dùng cuối. Điều này giúp chi phí hạ tầng máy chủ của hệ thống giảm về mức tiệm cận không, đồng thời mang lại khả năng mở rộng tuyến tính vô hạn theo số lượng người dùng đồng thời mà không bị giới hạn bởi năng lực chịu tải của cụm máy chủ trung tâm.
- **Giải quyết bài toán an toàn thông tin (RCE):** Trong các giải pháp chạy code trên server (Docker hoặc Server-side), việc cho phép người dùng hoặc tác nhân AI thực thi mã nguồn bất kỳ luôn tiềm ẩn nguy cơ tấn công chiếm quyền điều khiển hệ thống (Remote Code Execution - RCE). Bằng việc thực thi khép kín trong hộp cát WebAssembly phía client, WebContainer triệt tiêu hoàn toàn nguy cơ tấn công này đối với hạ tầng backend của Nexastu.

- **Đánh đổi về độ trễ khởi tạo:** Mặc dù WebContainer gặp nhược điểm là độ trễ tải gói tin WebAssembly ban đầu phụ thuộc vào tốc độ mạng của client, nhưng khi môi trường sandbox đã khởi chạy thành công, tốc độ phản hồi của hệ thống tệp ảo (VFS) và định tuyến mạng nội bộ qua Service Worker đạt mức gần như tức thời (dưới 10ms), vượt trội so với việc gửi yêu cầu qua lại giữa client và container chạy từ xa trên đám mây.

Từ những phân tích trên, WebContainer API là lựa chọn tối ưu và mang tính quyết định để xây dựng môi trường phát triển trực quan tích hợp AI cho hệ thống Nexastu.

2.5 Mô hình Ngôn ngữ Lớn và Kỹ nghệ Định tuyến

Công nghệ AI tạo sinh (Generative AI), đặc biệt là các mô hình ngôn ngữ lớn (Large Language Model - LLM), đóng vai trò là bộ não trung tâm của hệ thống Nexastu. Từ việc bóc tách thông tin lý lịch cá nhân, sinh mã nguồn Next.js/React hoàn chỉnh, đến việc hỗ trợ người dùng chỉnh sửa giao diện qua hội thoại tự nhiên, mọi tác vụ sáng tạo đều được điều phối bởi LLM. Tuy nhiên, không một mô hình đơn lẻ nào có thể tối ưu cho tất cả các tác vụ đa dạng này. Phần này trình bày đặc tính kỹ thuật của hai mô hình được tích hợp trong Nexastu và cơ chế định tuyến thông minh giữa chúng.

2.5.1 Đặc tính kỹ thuật của Gemini 2.5 Flash và DeepSeek V4

Hệ thống tận dụng thế mạnh riêng biệt của từng mô hình:

- **Gemini 2.5 Flash:** Là mô hình đa phương thức thế hệ mới của Google [5], nổi bật với khả năng xử lý ngữ cảnh siêu lớn (context window lên đến 1 triệu token) và phân tích các tệp tin phức tạp (PDF, hình ảnh). Mô hình này được sử dụng làm lõi xử lý dữ liệu lý lịch CV PDF để trích xuất sang định dạng cấu trúc JSON thông qua Zod Schema và xử lý các phản hồi có đính kèm hình ảnh. Gemini 2.5 Flash có ưu thế về tốc độ phản hồi nhanh và chi phí thấp cho đa tác vụ.
- **DeepSeek V4 (deepseek-v4-pro):** Là mô hình lý luận (reasoning) và lập trình chuyên sâu của DeepSeek [4], sử dụng cơ chế chuỗi suy nghĩ (Chain of Thought) trước khi trả lời. DeepSeek V4 vượt trội trong việc sinh mã nguồn Next.js/React phức tạp, viết CSS Tailwind chuẩn và sửa các lỗi logic mã nguồn phức tạp. Mô hình này thường tạo ra mã nguồn có cấu trúc tốt hơn và ít lỗi biên dịch hơn so với các mô hình đa phương thức tổng quát.

Bảng 2.4 trình bày so sánh chi tiết giữa hai mô hình trên các tiêu chí quan trọng cho ứng dụng trong hệ thống Nexastu.

Bảng 2.4: So sánh đặc tính kỹ thuật giữa Gemini 2.5 Flash và DeepSeek V4

Tiêu chí	Gemini 2.5 Flash	DeepSeek V4
Nhà phát triển	Google DeepMind	DeepSeek AI
Kiểu mô hình	Đa phương thức (nhận diện hình ảnh)	Chuyên biệt lập trình (chỉ văn bản)
Context window	1.000.000 tokens	128.000 tokens
Cơ chế suy luận	Thinking mode (tùy chọn)	Chain of Thought (mặc định)
Thế mạnh chính	Bóc tách CV/PDF, xử lý hình ảnh, phản hồi nhanh	Sinh mã nguồn chất lượng cao, sửa lỗi logic phức tạp
Vai trò trong Nexastu	Bóc tách dữ liệu CV, xử lý phản hồi đa phương thức	Sinh mã nguồn chính, tự sửa lỗi, tối ưu cấu trúc (Refactor)

2.5.2 Kỹ nghệ định tuyến đa phương thức dự phòng (Multimodal Routing Fallback)

Để tối ưu hóa chi phí API và đạt hiệu năng sinh mã cao nhất, Nexastu phát triển bộ định tuyến thông minh hoạt động theo nguyên tắc phân luồng dựa trên kiểu dữ liệu đầu vào. Khi người dùng giao tiếp thuần văn bản, hệ thống định tuyến yêu cầu API đến DeepSeek V4 để tận dụng tư duy logic lập trình vượt trội. Tuy nhiên, nếu người dùng đính kèm hình ảnh (ví dụ phác thảo giao diện hoặc ảnh chụp màn hình bị lỗi), hệ thống tự động chuyển đổi dự phòng (fallback) sang Gemini 2.5 Flash để xử lý dữ liệu hình ảnh đa phương thức, đảm bảo luồng nghiệp vụ không bị gián đoạn.

Cơ chế định tuyến này còn đảm bảo tính chịu lỗi (fault tolerance): nếu một mô hình gặp sự cố (hết hạn ngạch API, lỗi mạng, thời gian phản hồi quá lâu), hệ thống sẽ tự động chuyển đổi dự phòng sang mô hình còn lại, đảm bảo trải nghiệm người dùng không bị gián đoạn. Toàn bộ logic định tuyến được tích hợp trong Inngest Step Functions, cho phép mỗi lần chuyển đổi mô hình đều được ghi nhận trạng thái và có khả năng thử lại thông minh. Sơ đồ lưu đồ chi tiết của thuật toán định tuyến đa phương thức dự phòng này được trình bày tại Hình 3.5 (Chương 3).

2.5.3 Định tuyến động đa mô hình và Cơ chế chốt chặn Vòng lặp tác nhân (Agent Loop Guard)

Trong các hệ thống lập trình tự động dựa trên tác nhân trí tuệ nhân tạo (AI-Agent), khả năng điều phối linh hoạt giữa các mô hình ngôn ngữ lớn (LLM) và đảm bảo tính ổn định trong chu kỳ thực

thi công cụ là hai yếu tố then chốt quyết định hiệu năng hệ thống. Nexastu giải quyết vấn đề này bằng việc phát triển bộ định tuyến động đa mô hình (Model Switcher) kết hợp với chốt chặn vòng lặp vô hạn (Agent Loop Guard).

1. **Cơ chế Định tuyến động đa mô hình (Model Switcher):** Hệ thống hỗ trợ cả mô hình Google Gemini và DeepSeek V4 thông qua bộ chọn trên giao diện người dùng. Trạng thái lựa chọn này được lưu trong thuộc tính `aiProvider` của bảng hội thoại trong cơ sở dữ liệu Convex.

- Khi AI-Agent khởi chạy thông qua Inngest workflow, hệ thống truy vấn thuộc tính `aiProvider` để quyết định khởi tạo mô hình tương ứng.
- Đối với DeepSeek V4, hệ thống sử dụng thư viện OpenAI Adapter để cấu hình địa chỉ endpoint của DeepSeek API, cho phép thực thi chế độ suy luận lập trình chuyên sâu.
- Đối với Gemini, hệ thống khởi tạo thông qua Google Gen AI SDK.
- **Định tuyến cưỡng bức khi có đa phương tiện:** Trong trường hợp người dùng chọn DeepSeek (mô hình thuần văn bản) nhưng tin nhắn đính kèm hình ảnh (như phác thảo, ảnh chụp màn hình lỗi), bộ định tuyến sẽ tự động ghi đè quyết định và chuyển hướng yêu cầu sang Gemini 2.5 Flash để xử lý dữ liệu đa phương thức, kèm theo nhãn thông báo ghi chú (Banner Warning) hiển thị trên giao diện người dùng.

2. **Cơ chế chốt chặn Vòng lặp tác nhân (Agent Loop Guard):** Khi AI-Agent tự động gọi các công cụ trong môi trường mạng của Inngest, có khả năng Agent rơi vào vòng lặp vô hạn (infinite loop) do gọi lặp đi lặp lại cùng một công cụ với cùng một tham số nhưng không tạo ra tiến triển mới. Để tránh lãng phí tài nguyên và chi phí gọi API, Nexastu triển khai cơ chế Loop Guard ngay trong bộ định tuyến của mạng tác nhân (Agent Network):

- Hệ thống liên tục duy trì một bộ nhớ đệm ghi nhận lịch sử kết quả của các lượt chạy gần nhất.
- Tại mỗi chu kỳ định tuyến (router iteration), hệ thống lấy ra 3 lượt gọi công cụ gần nhất (tool calls) và so sánh nội dung của chúng (bao gồm tên công cụ và toàn bộ danh sách đối số truyền vào).
- Nếu phát hiện 3 lượt gọi liên tiếp có cấu trúc đối số hoàn toàn trùng khớp:

$$\text{Calls}_{t-2} = \text{Calls}_{t-1} = \text{Calls}_t \quad (2.1)$$

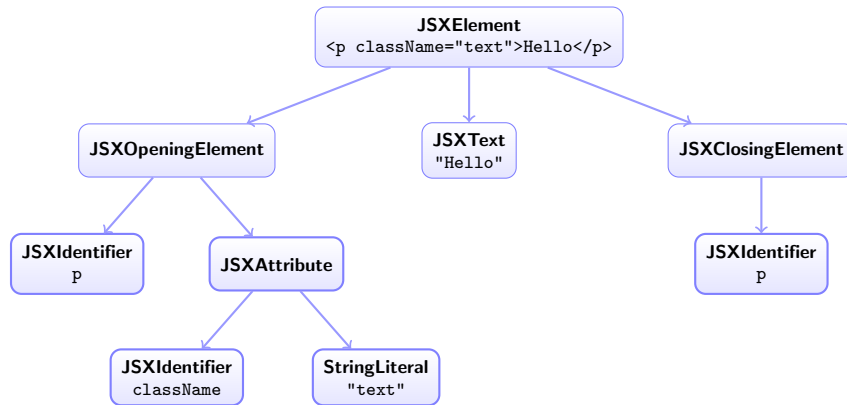
hệ thống sẽ lập tức kích hoạt cơ chế ngắt sớm (early termination) và dừng luồng chạy của Inngest Agent, trả về cảnh báo để tránh việc chạy lặp vô tận gây cạn kiệt tài nguyên.

2.6 Phân tích cú pháp AST (Abstract Syntax Tree) và kỹ thuật vá mã nguồn

Một trong những tính năng khác biệt của Nexastu so với các công cụ AI sinh mã hiện có là khả năng cho phép người dùng chỉnh sửa trực tiếp nội dung văn bản trên giao diện xem trước trực tiếp (Live Preview) mà kết quả được ánh xạ chính xác về mã nguồn tương ứng. Tính năng này đòi hỏi một kỹ thuật phức tạp: ánh xạ chính xác vị trí một phần tử HTML hiển thị trên giao diện sang đúng dòng và cột trong tệp tin mã nguồn `.tsx`. Cơ sở lý thuyết cho giải pháp này là Cây cú pháp trừu tượng (Abstract Syntax Tree - AST).

2.6.1 Cây cú pháp trừu tượng AST

AST là một biểu diễn cấu trúc dạng cây của mã nguồn của một chương trình máy tính [6]. Khi một tệp tin `.tsx` được nạp vào, trình phân tích cú pháp (parser) sẽ chuyển đổi chuỗi văn bản mã nguồn thành các nút (nodes) của cây AST. Mỗi nút đại diện cho một cấu trúc ngôn ngữ lập trình như định nghĩa biến, lời gọi hàm, hoặc thẻ JSX (ví dụ: `JSXElement`, `JSXAttribute`, `JSXText`). Để trực quan hóa, cấu trúc cây AST tương ứng của một phần tử JSX cơ bản trong hệ thống được minh họa chi tiết tại Hình 2.5.



Hình 2.5: Biểu diễn cây cú pháp trừu tượng AST chi tiết của phần tử JSX trong Nexastu.

Quá trình xây dựng cây AST từ mã nguồn bao gồm ba giai đoạn chính:

1. **Phân tích từ vựng (Lexical Analysis):** Mã nguồn được chia thành các đơn vị từ vựng nhỏ nhất gọi là token (ví dụ: từ khóa, định danh, toán tử, chuỗi ký tự). Giai đoạn này loại bỏ các khoảng trắng và chú thích không cần thiết.
2. **Phân tích cú pháp (Syntactic Analysis):** Các token được tổ chức thành cấu trúc cây phân cấp dựa trên ngữ pháp của ngôn ngữ lập trình. Mỗi nút trong cây chứa thông tin về loại nút (node type), vị trí trong mã nguồn (dòng - line, cột - column), và các nút con (children).
3. **Biểu diễn cây AST:** Kết quả cuối cùng là một cấu trúc dữ liệu dạng cây, trong đó nút gốc (root node) đại diện cho toàn bộ chương trình, và các nút lá (leaf nodes) đại diện cho các phần tử nguyên tử như giá trị hằng số, tên biến, hoặc chuỗi văn bản JSX.

2.6.2 Kỹ thuật định vị và vá mã nguồn (AST Patching)

Nexastu sử dụng thư viện `ts-morph` (dựa trên TypeScript Compiler API) để thực hiện các thao tác vá mã nguồn một cách chính xác và an toàn. Quy trình vá mã nguồn trong hệ thống bao gồm:

1. **Gắn định vị (locator) vào DOM:** Trong quá trình sinh mã ban đầu, AI-Agent tự động chèn thêm một thuộc tính định vị đặc biệt `data-nexastu-source=<filePath>:<line>:<col>` vào các thẻ JSX quan trọng. Thuộc tính này đóng vai trò như một bản đồ liên kết giữa phần tử DOM hiển thị và vị trí tương ứng trong mã nguồn.
2. **Bắt sự kiện và gửi tọa độ:** Khi người dùng lựa chọn hoặc chỉnh sửa văn bản trên giao diện Live Preview, Inspector Bridge sẽ đọc giá trị `data-nexastu-source` từ phần tử DOM đó và gửi ngược lại cho ứng dụng cha thông qua API `postMessage`.
3. **Phân tích và vá lỗi (Patching):**
 - Hệ thống sử dụng `ts-morph` để dựng cây AST của tệp tin đích từ nội dung mã nguồn lưu trong Convex DB.
 - Sử dụng thuật toán duyệt cây (tree traversal) để tìm chính xác nút `JSXElement` khớp với tọa độ dòng (`line`) và cột (`col`) đã nhận.
 - Để tránh việc lệch dòng khi mã nguồn bị chỉnh sửa nhiều lần, hệ thống sử dụng thuật toán đối khớp văn bản gốc (`originalText`) để xác định đúng vị trí cần thay thế, thay thế giá trị bằng văn bản mới (`newText`) hoặc thuộc tính CSS mới.
4. **Đồng bộ và kích hoạt HMR:** Chuyển đổi ngược cây AST đã cập nhật thành chuỗi văn bản mã nguồn, lưu vào Convex DB và tự động đồng bộ sang hệ thống tệp ảo WebContainer để kích hoạt cơ chế Hot Module Replacement (HMR), hiển thị kết quả chỉnh sửa mới lập tức trên giao diện Live Preview mà không cần tải lại trang. Quy trình tương tác và trao đổi thông điệp qua cổng `postMessage` của cơ chế này được đặc tả chi tiết tại sơ đồ Hình 3.3 (Chương 3).

2.7 Cơ chế đồng bộ hóa VFS-Convex và Đảm bảo Nhất quán Trạng thái

Trong một hệ thống nơi mã nguồn được sinh ra và chỉnh sửa liên tục bởi AI-Agent, việc đảm bảo khả năng duy trì trạng thái ổn định khi xảy ra lỗi và bảo đảm tính bền vững của dữ liệu là yêu cầu then chốt. Nexastu giải quyết hai thách thức này thông qua thuật toán đồng bộ hóa VFS-Convex hai chiều và cơ chế kiểm soát lỗi cú pháp thông qua phân tích tĩnh.

2.7.1 Thuật toán đồng bộ hóa trạng thái VFS lên Convex DB

Do VFS của WebContainer chỉ tồn tại trên bộ nhớ RAM, toàn bộ mã nguồn website portfolio của người dùng sẽ bị giải phóng khi trình duyệt thực hiện tải lại trang hoặc khi người dùng thay đổi thiết bị truy cập. Để đảm bảo tính bền vững của dữ liệu (*Durability*), Nexastu thiết kế một thuật toán đồng bộ hóa hai chiều thời gian thực giữa hệ thống tệp ảo cục bộ (*client-side VFS*) và cơ sở dữ liệu đám mây Convex:

1. **Đăng ký trình theo dõi sự thay đổi (File Watcher):** Hệ thống đăng ký lắng nghe sự kiện ghi (`writeFile`), xóa (`unlink`) hoặc đổi tên tệp tin trên đĩa ảo của WebContainer Sandbox. Mỗi sự kiện được ghi nhận kèm theo đường dẫn tệp, nội dung mới và dấu thời gian (*timestamp*).
2. **Bộ đệm và cơ chế gom lô (Debounce & Batching):** Nhằm tránh việc gửi hàng loạt yêu cầu mạng (*network requests*) liên tục gây quá tải băng thông và dẫn đến xung đột ghi hoặc tranh chấp dữ liệu (*write conflicts/race conditions*) trên cơ sở dữ liệu đám mây, các sự kiện thay đổi được đẩy vào một hàng đợi đệm (*buffer queue*). Khi người dùng dừng thao tác soạn thảo (sử dụng kỹ thuật trì hoãn – *debounce* với thời gian chờ 300ms) hoặc khi AI hoàn tất một đợt sinh file hàng loạt, hệ thống sẽ gom tất cả các tệp tin thay đổi lại thành một gói tin gộp duy nhất.
3. **Giao dịch nguyên tử đồng bộ (Atomic sync transaction):** Toàn bộ danh sách thay đổi được gửi lên Convex DB trong một giao dịch (*transaction*) duy nhất thông qua hàm `updateFilesBatch` (biến đổi gộp). Hàm này cập nhật đồng bộ toàn bộ cây thư mục trên cơ sở dữ liệu Convex trong cùng một giao dịch gộp, đảm bảo tính nhất quán cao nhất của dữ liệu. Nếu bất kỳ thao tác con nào trong giao dịch bị lỗi, toàn bộ giao dịch sẽ tự động hoàn tác (*rollback*), ngăn chặn tình trạng dữ liệu không đồng bộ. Thuật toán gom lô thay đổi (*batching*) bằng bộ định thời trì hoãn (*debounce timer*) trước khi đồng bộ được mô tả thông qua lưu đồ hoạt động tại Hình 3.6 (Chương 3).

2.7.2 Cơ chế đảm bảo nhất quán giao dịch và phục hồi qua bộ kiểm tra AST

Để đảm bảo tính nhất quán của mã nguồn dự án trong môi trường phát triển trực tuyến không sử dụng các lệnh hệ thống đĩa cứng phức tạp, Nexastu đề xuất giải pháp quản lý giao dịch ở mức phân tích cú pháp tĩnh thông qua bộ kiểm tra cấu trúc AST của TypeScript:

- **Kiểm tra tính hợp lệ trước khi ghi (Pre-write Validation):** Trước khi bất kỳ thay đổi nào được lưu xuống VFS, nội dung tệp tin mới biên dịch sẽ được bộ phân tích cú pháp `ts-morph` quét tĩnh để kiểm tra lỗi cú pháp JSX/TSX. Nếu phát hiện lỗi nghiêm trọng (như thiếu thẻ đóng hoặc lỗi khai báo), giao dịch sẽ bị từ chối ngay lập tức và giữ nguyên phiên bản cũ trên bộ nhớ.
- **Phục hồi khối an toàn (AST-based Non-Destructive Restore):** Khi người dùng thực hiện hoàn tác hoặc muốn phục hồi một phần giao diện về trạng thái gốc, hệ thống truy vấn siêu dữ liệu cấu trúc gốc từ tệp `.nexastu/debug-info.json` trên Convex DB, thực hiện thuật toán chồng chập (overlay) để lấy lại các khối thẻ mẫu sạch mà không làm ảnh hưởng đến các tệp tin khác trong không gian làm việc. Điều này giúp tránh hoàn toàn các thao tác chạy lệnh hệ thống Git gây rủi ro xóa nhầm tệp tin.
- **Mức độ cô lập giao dịch của Convex:** Tính nguyên tử của các thay đổi đa tệp tin được bảo đảm bởi mức cô lập Serializable Isolation của Convex DB. Khi một giao dịch Mutation ghi gộp `updateFilesBatch` thất bại ở bất kỳ tệp tin nào, toàn bộ giao dịch ghi của chu kỳ đó sẽ tự động được hoàn tác (rollback) trên cơ sở dữ liệu, ngăn ngừa hoàn toàn tình trạng dữ liệu mờ côi hoặc không đồng bộ.

Cơ chế đồng bộ hóa này đảm bảo tính bền vững (Durability) của dữ liệu mà vẫn duy trì được tốc độ xử lý tức thì của môi trường Sandbox phía client. Khi người dùng trở lại hệ thống từ một thiết bị khác hoặc sau khi tải lại trang, toàn bộ cây thư mục mã nguồn sẽ được khôi phục từ Convex DB và đồng bộ ngược vào WebContainer VFS, đảm bảo trải nghiệm liên tục và không mất dữ liệu.

2.8 Cơ chế chỉnh sửa trực tiếp contentEditable và Mô hình đồng bộ hai luồng

Trong các công cụ xây dựng giao diện trực quan hiện đại, trải nghiệm tương tác trực tiếp trên giao diện WYSIWYG (*What You See Is What You Get*) là một thành phần cốt lõi. Người dùng có nhu cầu nhấn trực tiếp vào một dòng chữ trên màn hình xem trước và chỉnh sửa nó mà không cần mở trình soạn thảo mã nguồn. Tuy nhiên, việc hiện thực hóa cơ chế này trên môi trường web động (Next.js/React) đối mặt với các thách thức kỹ thuật lớn về quản lý trạng thái DOM và độ trễ biên dịch.

2.8.1 Thách thức của API HTML5 contentEditable trong soạn thảo trực quan

Thuộc tính `contentEditable` là một tiêu chuẩn của HTML5 cho phép người dùng sửa đổi trực tiếp nội dung văn bản của bất kỳ phần tử DOM nào ngay trên trình duyệt. Tuy nhiên, đối với một hệ thống phát triển tích hợp dựa trên nền web (*Web-based IDE*) đồng bộ với mã nguồn như Nexastu, việc sử dụng `contentEditable` phát sinh các rào cản nghiêm trọng:

1. **Mất tiêu điểm và vị trí con trỏ (Caret Selection Loss):** Khi người dùng đang nhập văn bản, bất kỳ hành động kết xuất lại (*re-render*) nào của React hoặc cập nhật từ hệ thống tệp ảo sẽ tái tạo lại nút DOM. Quá trình này làm mất hoàn toàn tiêu điểm (*focus*) và con trỏ soạn thảo của người dùng bị nhảy về vị trí đầu dòng, gây gián đoạn quá trình nhập liệu.
2. **Sự phân mảnh cấu trúc DOM (DOM Fragmentation):** Các trình duyệt khác nhau xử lý các phím chức năng (như phím Enter hoặc Backspace) trên phần tử `contentEditable` theo các cách khác nhau. Ví dụ, Google Chrome tự động chèn các thẻ `<div>` hoặc `<br>` mới khi nhấn Enter, trong khi Firefox có thể chèn các thẻ `<p>`. Điều này làm biến đổi cấu trúc HTML ban đầu và tạo ra các thẻ JSX không hợp lệ khi được vá ngược lại mã nguồn.
3. **Xung đột lan truyền sự kiện (Event Propagation):** Các phần tử văn bản thường nằm lồng bên trong các thẻ liên kết (`<a>`) hoặc nút bấm (`button`). Khi người dùng nhấn chuột hoặc tương tác soạn thảo, trình duyệt sẽ tự động kích hoạt hành động mặc định (như chuyển hướng trang hoặc gửi – *submit* biểu mẫu), làm gián đoạn luồng chỉnh sửa trực quan.
4. **Bất đồng bộ giữa DOM thực và Virtual DOM:** Việc chỉnh sửa bằng `contentEditable` thay đổi trực tiếp DOM thực của trình duyệt. Tuy nhiên, React quản lý giao diện thông qua Virtual DOM và mô hình luồng dữ liệu một chiều. Sự bất đồng bộ này khiến React tự động ghi đè và làm mất các thay đổi của người dùng trong lần kết xuất tiếp theo.

2.8.2 Mô hình đồng bộ trạng thái hai luồng (Double-Loop Synchronization)

Để mang lại trải nghiệm chỉnh sửa tức thời với độ trễ 0ms (*Zero-Latency*), hệ thống không thể chờ đợi toàn bộ chu trình xử lý mã nguồn hoàn tất. Chu trình xử lý mã nguồn hoàn chỉnh bao gồm các bước: (1) nhận văn bản sửa đổi, (2) vá mã nguồn thông qua AST, (3) ghi vào Convex DB, (4) WebContainer VFS đồng bộ, (5) máy chủ phát triển Next.js (*Next.js development server*) biên dịch lại, và (6) HMR nạp lại giao diện. Chu trình này có độ trễ hệ thống dao động từ 500ms đến 1.500ms tùy thuộc vào cấu hình máy trạm. Nếu người dùng phải chờ khoảng thời gian này cho mỗi ký tự được gõ, hiện tượng giật lag cực kỳ nghiêm trọng sẽ xảy ra.

Nexastu đề xuất giải pháp kiến trúc dựa trên mô hình đồng bộ hai luồng (*Double-Loop Synchronization*) để tách biệt luồng hiển thị giao diện và luồng xử lý mã nguồn bền vững:

1. **Luồng lặp nhanh phía máy khách (Fast Loop – 0ms):** Hoạt động trực tiếp trên giao diện tương tác của IDE. Khi người dùng thực hiện chỉnh sửa trực quan trên phần tử `contentEditable`, hệ thống sẽ thực hiện cập nhật lạc quan (*optimistic update*) trạng thái giao diện phía máy khách (*Client-Side State*). DOM thực hiện thị văn bản mới ngay lập tức mà không chờ phản hồi từ server.
2. **Luồng lặp chậm chạy ngầm (Slow Loop – Background):** Hoạt động bất đồng bộ ở phía sau. Gói tin chỉnh sửa chứa tọa độ mã nguồn và văn bản mới được gửi lên API để vá cây AST và cập nhật cơ sở dữ liệu Convex DB. Tiến trình WebContainer VFS phát hiện thay đổi, Next.js tự động biên dịch ngầm và thực hiện HMR.

Khi vòng lặp chậm (*Slow Loop*) hoàn thành, trạng thái biên dịch mới của mã nguồn Next.js sẽ khớp hoàn toàn với trạng thái hiển thị hiện tại của vòng lặp nhanh (*Fast Loop*), tạo nên một trải nghiệm chỉnh sửa trơn tru, liền mạch mà vẫn đảm bảo tính bền vững của mã nguồn.

Chương 3

PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 KIẾN TRÚC HỆ THỐNG TỔNG QUAN

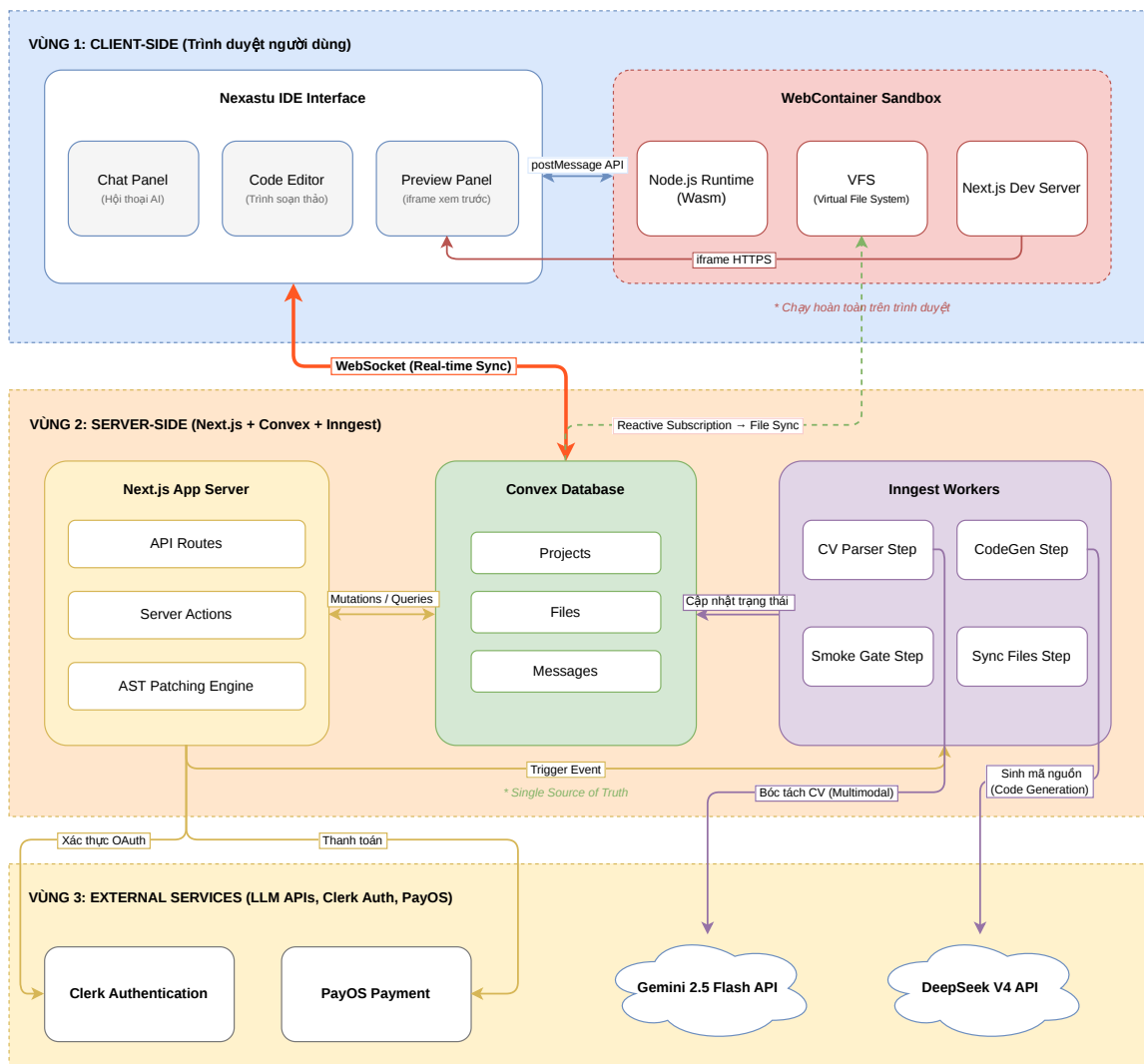
3.1.1 Các thành phần cốt lõi của hệ thống

Hệ thống Nexastu được thiết kế theo kiến trúc lai (hybrid architecture), kết hợp giữa dịch vụ máy chủ hiện đại (Next.js, Convex, Inngest) và môi trường ảo hóa phía client (WebContainer). Kiến trúc này giúp tối ưu hóa chi phí vận hành máy chủ biên dịch bằng cách đẩy toàn bộ việc cài đặt thư viện và chạy mã nguồn Next.js của người dùng xuống trình duyệt. Mối quan hệ giữa các thành phần cốt lõi của hệ thống được mô tả như sau:

1. **Mặt phẳng ứng dụng chính (Nexastu Application):** Chạy trên nền tảng Next.js, đóng vai trò giao diện điều khiển (Control Panel) cho người dùng, xử lý xác thực qua Clerk, và quản lý các hội thoại chat trợ lý AI.
2. **Cơ sở dữ liệu thời gian thực Convex:** Đóng vai trò là nguồn dữ liệu duy nhất đáng tin cậy (Single Source of Truth). Convex lưu trữ thông tin dự án, cấu trúc tệp tin mã nguồn của website portfolio người dùng, và trạng thái hội thoại. Mọi thay đổi về mã nguồn trên Convex lập tức đồng bộ thời gian thực về phía client.
3. **Hệ thống xử lý sự kiện Inngest:** Chạy các worker ngầm để giao tiếp với các API mô hình ngôn ngữ lớn (LLM API) của Google và DeepSeek, quản lý các pipeline sinh mã (CodeGen Pipeline).
4. **Môi trường ảo hóa WebContainer Sandbox:** Chạy bên trong một thẻ `<iframe>` độc lập tại

trình duyệt của người dùng. Sandbox này liên tục lắng nghe thay đổi mã nguồn từ Convex thông qua WebSocket để cập nhật tệp tin ảo nội bộ, chạy tiến trình Next.js phát triển và phản hồi giao diện live preview cho người dùng. Bên cạnh đó, hệ thống hỗ trợ cơ chế mở rộng (pop-out) trang web xem trước sang một tab trình duyệt mới độc lập. Cơ chế này hoạt động dựa trên khả năng định cấu hình định tuyến thông qua Service Worker của WebContainer, cho phép tab mới truy cập trực tiếp vào máy chủ phát triển Next.js đang chạy ngầm mà không cần phải khởi tạo lại môi trường sandbox ảo hóa.

Để trực quan hóa kiến trúc này trong báo cáo, sơ đồ kiến trúc tổng quan được thiết kế chi tiết. Hình 3.1 mô tả luồng giao tiếp giữa các thành phần trong hệ thống.



Hình 3.1: Sơ đồ kiến trúc tổng quan hệ thống Nexastu. Thể hiện sự phân rã giữa mặt phẳng máy chủ (Next.js, Convex, Inngest) và mặt phẳng máy trạm (WebContainer Sandbox chạy trong Iframe).

Bảng 3.1: Chú giải ký hiệu và đường liên kết trong sơ đồ kiến trúc hệ thống

Ký hiệu / Kiểu đường	Ý nghĩa học thuật và Chức năng trong hệ thống
Hình chữ nhật nét liền	Thành phần xử lý logic hoặc dịch vụ chạy cục bộ (Internal/Local component) của hệ thống Nexastu.
Hình chữ nhật nét đứt	Môi trường sandbox ảo hóa cô lập (Isolated Virtual Sandbox) chạy trong bộ nhớ RAM trình duyệt.
Hình trụ (Cylinder)	Cơ sở dữ liệu lưu trữ trạng thái và tệp tin mã nguồn của dự án (Convex DB).
Hình đám mây (Cloud)	Dịch vụ hoặc API bên thứ ba nằm ngoài hạ tầng Nexastu (Gemini, DeepSeek).
Đường nét đứt ngang	Ranh giới phân định mặt phẳng vận hành: Client-side (Trình duyệt), Server-side (Hạ tầng máy chủ), và External (Môi trường ngoài).
Mũi tên liền một chiều	Lời gọi hàm, gửi yêu cầu HTTP một chiều hoặc kích hoạt sự kiện (API invocation / Event trigger).
Mũi tên liền hai chiều	Giao tiếp đồng bộ hai chiều (Synchronous client-server request/response) hoặc Mutations/Queries truy vấn dữ liệu.
Mũi tên đứt hai chiều	Truyền dữ liệu bất đồng bộ chéo nguồn bảo mật (Cross-origin asynchronous messaging) qua postMessage.
Mũi tên đứt một chiều	Luồng đồng bộ hóa dữ liệu phản ứng thời gian thực (Real-time reactive subscription push) thông qua kết nối WebSocket.

3.1.2 Các luồng truyền thông tin chủ đạo

Để làm sáng tỏ cơ chế phối hợp giữa Máy chủ (Server-side) và Máy trạm (Client-side), toàn bộ hoạt động của hệ thống Nexastu được điều phối qua ba luồng dữ liệu khép kín chủ đạo sau:

1. Luồng đồng bộ hóa tệp tin ảo thời gian thực (Real-time VFS Synchronization Flow):

Đây là luồng hoạt động nền tảng duy trì tính nhất quán của mã nguồn dự án. Khi người dùng thực hiện sửa đổi mã nguồn trên giao diện IDE hoặc khi mô hình AI sinh tệp mới, các thay đổi sẽ được gửi trực tiếp lên Convex DB thông qua các API Mutation. Convex DB đóng vai trò nguồn dữ liệu tin cậy duy nhất (Single Source of Truth) lưu trữ trạng thái tệp tin. Ngay khi dữ liệu trên Convex DB thay đổi, hệ thống sẽ tự động phát tín hiệu (Reactive Push) qua giao thức WebSocket thời gian thực tới trình duyệt của người dùng. Tại đây, WebContainer Sandbox tiếp nhận thông tin và ghi đè dữ liệu trực tiếp vào hệ thống tệp tin ảo (VFS) chạy trên RAM. Tiến trình Next.js đang chạy ngầm trong WebContainer lập tức phát hiện thay đổi và kích hoạt cơ chế Hot Module

Replacement (HMR) để tải lại nóng và cập nhật hiển thị giao diện Live Preview.

2. **Luồng tự động sinh mã nguồn qua AI (Orchestrated AI CodeGen Flow):** Luồng dữ liệu này được kích hoạt khi người dùng thực hiện tải lên tệp lý lịch cá nhân (CV) định dạng PDF hoặc nhập prompt văn bản mô tả bản thân. Giao diện Client gửi tệp tin lên máy chủ để đăng ký dự án. Ngay sau đó, một sự kiện được đẩy vào Event Bus của hệ thống Inngest để kích hoạt tiến trình xử lý ngầm (Asynchronous Event-driven Pipeline). Inngest Worker đóng vai trò điều phối trung tâm (Orchestrator) sẽ thực thi chuỗi hành động bao gồm: gọi Gemini 2.5 Flash để phân tích cấu trúc CV dạng PDF sang JSON, chuyển tiếp JSON sang DeepSeek V4 để tiến hành suy luận và sinh mã nguồn Next.js/Tailwind, và đưa mã sinh ra qua cổng kiểm duyệt cú pháp Smoke Gate. Khi mã nguồn hoàn toàn hợp lệ, Inngest Worker thực hiện cập nhật đồng bộ cấu trúc thư mục dự án lên Convex DB để kích hoạt luồng VFS nói trên.
3. **Luồng tương tác trực quan bảo mật chéo nguồn (CORS-Safe Inspector Interaction Flow):** Để hỗ trợ tính năng chỉnh sửa trực quan (Visual Edit), khi người dùng di chuyển chuột (hover) hoặc bấm chuột chọn (click) các thành phần giao diện trên Iframe Preview, cầu nối Inspector Bridge (`inspector-bridge.tsx`) nằm bên trong Iframe sẽ bắt các sự kiện DOM tương ứng. Do Iframe chạy trên một phân miền phụ ảo của WebContainer (`*.webcontainer-api.io`) khác với tên miền chính của IDE Nexastu, cầu nối sẽ đóng gói các tham số tọa độ DOM và nhãn mã nguồn (`data-nexastu-source`) để gửi chéo nguồn qua API tin nhắn cửa sổ (`window.parent.postMessage`) về cho Host IDE. Ứng dụng cha Host IDE nhận thông điệp, xác thực nguồn gốc (Origin Validation) để tránh tấn công chèn mã độc, và tiến hành vẽ các khung viền đồ họa nổi lên trên Iframe, đồng thời hiển thị bảng điều khiển thuộc tính tương ứng giúp người dùng thực hiện sửa đổi tức thì.

3.2 THIẾT KẾ LUỒNG XỬ LÝ SINH MÃ TỰ ĐỘNG (RESUME-TO-CODE PIPELINE)

Quy trình tự động hóa chuyển đổi thông tin từ tài liệu CV thô sang một website Next.js hoàn chỉnh được phân rã thành các giai đoạn chặt chẽ được quản lý bởi bộ điều phối trung tâm (Orchestrator) trong Inngest. Luồng hoạt động của pipeline được thực hiện qua các giai đoạn sau:

Giai đoạn trích xuất thông tin (Parsing Input): Người dùng tải lên file CV PDF hoặc nhập prompt mô tả bản thân. Hệ thống gọi mô hình **Gemini 2.5 Flash** để thực hiện phân tích cú pháp đa phương thức (Multimodal Document Parsing). Kết quả đầu ra là một cấu trúc dữ liệu JSON được chuẩn hóa chứa các trường thông tin cá nhân (`personalInfo`), kỹ năng (`skills`), kinh nghiệm làm việc (`experience`), dự án thực tế (`projects`) và học vấn (`education`).

Giai đoạn sinh mã nguồn (CodeGen): Dữ liệu JSON được chuyển tiếp đến bộ điều phối

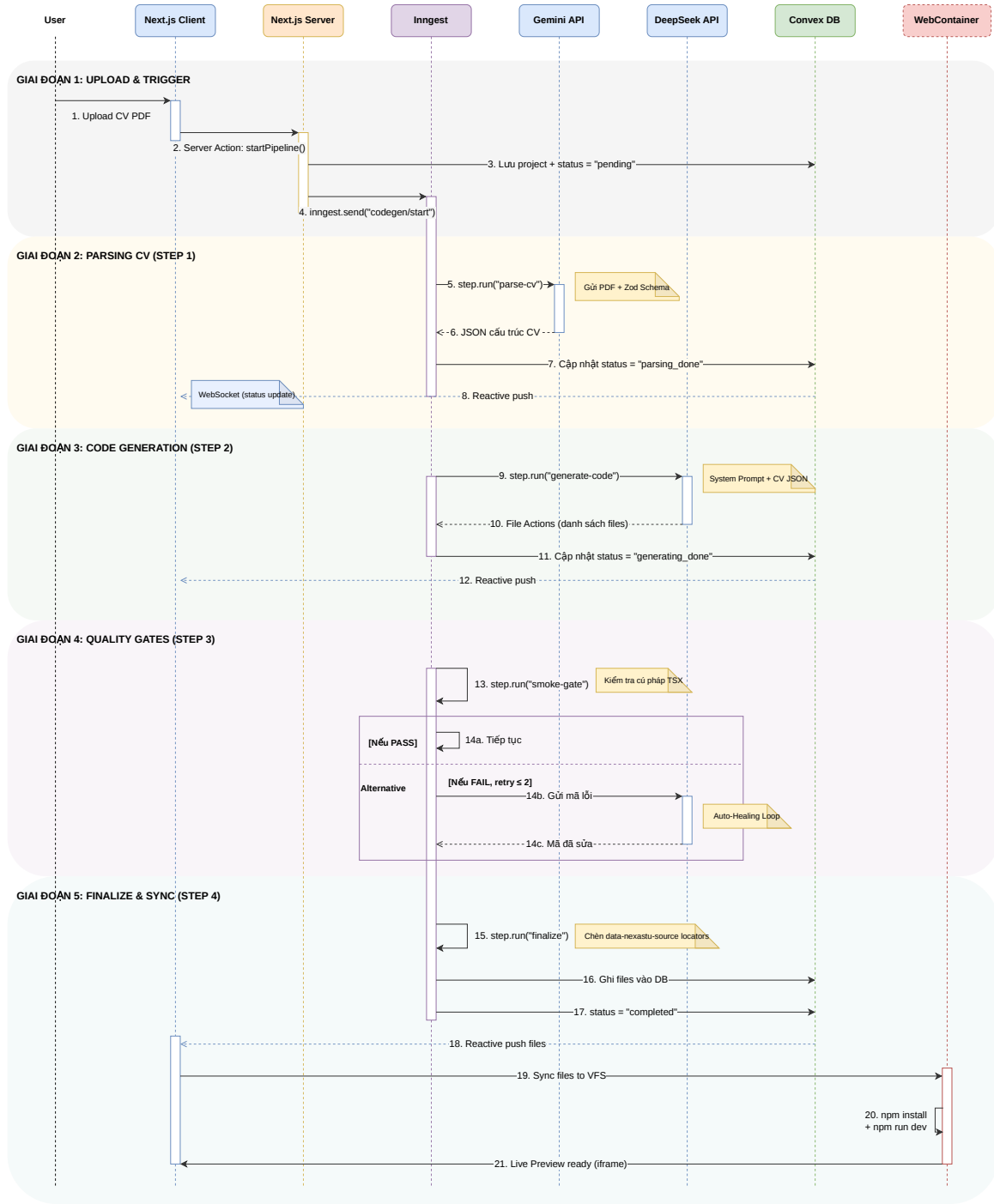
`orchestrator.ts`. Hệ thống gọi API của **DeepSeek V4** (hoặc Gemini 2.5 Flash tùy cấu hình) kèm theo hệ thống chỉ thị (System Prompts) nghiêm ngặt về cấu trúc dự án portfolio Next.js. LLM sinh ra danh sách các hành động tạo file (File Actions) bao gồm: tệp tin cấu hình (`package.json`, `tailwind.config.js`), tệp tin bố cục (`layout.tsx`), trang hiển thị chính (`page.tsx`), và các thành phần giao diện UI (`bento-grid.tsx`, `hero.tsx`, `contact.tsx`).

Giai đoạn vá mã nguồn và bảo vệ (Finalize & Patching): Hệ thống chèn các locator định vị mã nguồn `data-nexastu-source` vào các thẻ JSX của mã nguồn được sinh ra để phục vụ tính năng Inspector sau này. Đồng thời, hệ thống cũng đảm bảo cài đặt đầy đủ các thư viện bổ sung cần thiết (như `framer-motion`, `lucide-react`).

Giai đoạn kiểm soát chất lượng (Smoke Gate & Health Gate): Giai đoạn này thực hiện hai chốt chặn kiểm thử tự động. Đầu tiên là **Smoke Gate** quét mã nguồn TSX được sinh ra để kiểm tra tính hợp lệ cú pháp, đảm bảo không có lỗi nạp thư viện (imports) không tồn tại hoặc lỗi đóng mở thẻ JSX. Nếu phát hiện lỗi, hệ thống tự động gọi cơ chế sửa lỗi (Auto-Healing) bằng cách gửi mã lỗi ngược lại cho LLM sửa đổi. Kế tiếp là **Health Gate** đảm bảo cấu trúc trang web hiển thị đầy đủ các thành phần thiết yếu (tích hợp phần giới thiệu hero, danh mục dự án, thông tin liên hệ).

Giai đoạn đồng bộ hóa và khởi chạy Sandbox: Khi vượt qua các cổng kiểm soát chất lượng, mã nguồn được ghi vào cơ sở dữ liệu Convex. WebContainer Sandbox phát hiện thay đổi trên Convex, tải các tệp tin xuống hệ thống tệp tin ảo trong RAM, chạy lệnh `npm install` (hoặc lấy từ bộ nhớ đệm cache tốc độ cao), khởi động máy chủ phát triển Next.js (`npm run dev`) và hiển thị kết quả lên iframe preview.

Sơ đồ tuần tự (Sequence Diagram) mô tả chi tiết quá trình này được trình bày trong Hình 3.2. Để hỗ trợ việc đọc hiểu và phân tích các tương tác tuần tự và bất đồng bộ giữa các thành phần, sơ đồ được thiết kế với các quy ước trực quan thống nhất: các thanh trục đứng (Lifelines) được phân bổ theo các màu sắc riêng biệt để dễ dàng nhận diện vai trò (màu xanh lam đại diện cho Next.js Client, màu cam cho Next.js Server, màu tím cho Inngest Worker, màu xanh lá cho Convex DB, và khung nét đứt màu đỏ cho WebContainer Sandbox), các đường liên kết nét liền biểu diễn các lệnh gọi hàm, yêu cầu HTTP hoặc kích hoạt sự kiện, trong khi các đường liên kết nét đứt biểu thị luồng đồng bộ dữ liệu phản ứng thời gian thực (Reactive Push) thông qua giao thức WebSocket.



Hình 3.2: Sơ đồ tuần tự Pipeline sinh mã Resume-to-Code. Thể hiện tương tác giữa Client, Server, Inngest Worker, Convex DB và các API LLM.

Dựa trên sơ đồ tuần tự được mô tả tại Hình 3.2, luồng xử lý chuyển đổi dữ liệu từ CV sang mã nguồn thực tế diễn ra qua các bước cụ thể sau:

Tiếp nhận CV, prompt từ người dùng (Tương ứng bước 1 trong sơ đồ): Người dùng tải tệp CV định dạng PDF hoặc nhập prompt mô tả năng lực bản thân lên giao diện Web-IDE của Nexastu.

Gửi yêu cầu xử lý lên máy chủ Next.js (Tương ứng bước 2 trong sơ đồ): UI Client thực hiện một HTTP POST request gửi dữ liệu tệp hoặc prompt về API Endpoint trên Next.js Server.

Khởi tạo dự án và kích hoạt worker (Tương ứng bước 3 và 4 trong sơ đồ): Server xác thực yêu cầu, khởi tạo bản ghi dự án mới trên Convex DB với trạng thái "pending", sau đó gửi sự kiện kích hoạt sang hệ thống hàng đợi sự kiện Inngest.

Phân tích CV đa phương thức qua Gemini (Tương ứng bước 5 trong sơ đồ): Inngest Worker chạy ngầm tiếp nhận sự kiện, gọi API của Gemini 2.5 Flash để thực hiện bóc tách tài liệu đa phương thức (Multimodal Parsing) kèm lược đồ Zod để chuẩn hóa cấu trúc dữ liệu.

Trả về kết quả JSON và đồng bộ trạng thái (Tương ứng bước 6, 7 và 8 trong sơ đồ): Gemini phân tích tài liệu và trả về JSON chuẩn hóa. Inngest cập nhật trạng thái "parsing_done" lên Convex DB để hệ thống tự động đẩy (Reactive Push) qua WebSocket đồng bộ trạng thái về phía giao diện client.

Yêu cầu DeepSeek V4 sinh mã nguồn Next.js (Tương ứng bước 9 trong sơ đồ): Inngest Worker gửi dữ liệu JSON đã phân tích kèm hệ thống chỉ thị (System Prompts) sang mô hình DeepSeek V4 để yêu cầu sinh mã nguồn dự án.

Đóng gói mã nguồn và lưu vào Convex DB (Tương ứng bước 10, 11 và 12 trong sơ đồ): DeepSeek V4 sinh mã nguồn, đóng gói trong các thẻ XML đặc thù và gửi trả về Inngest. Inngest cập nhật trạng thái "generating_done" lên Convex DB và đồng bộ thời gian thực về phía Client.

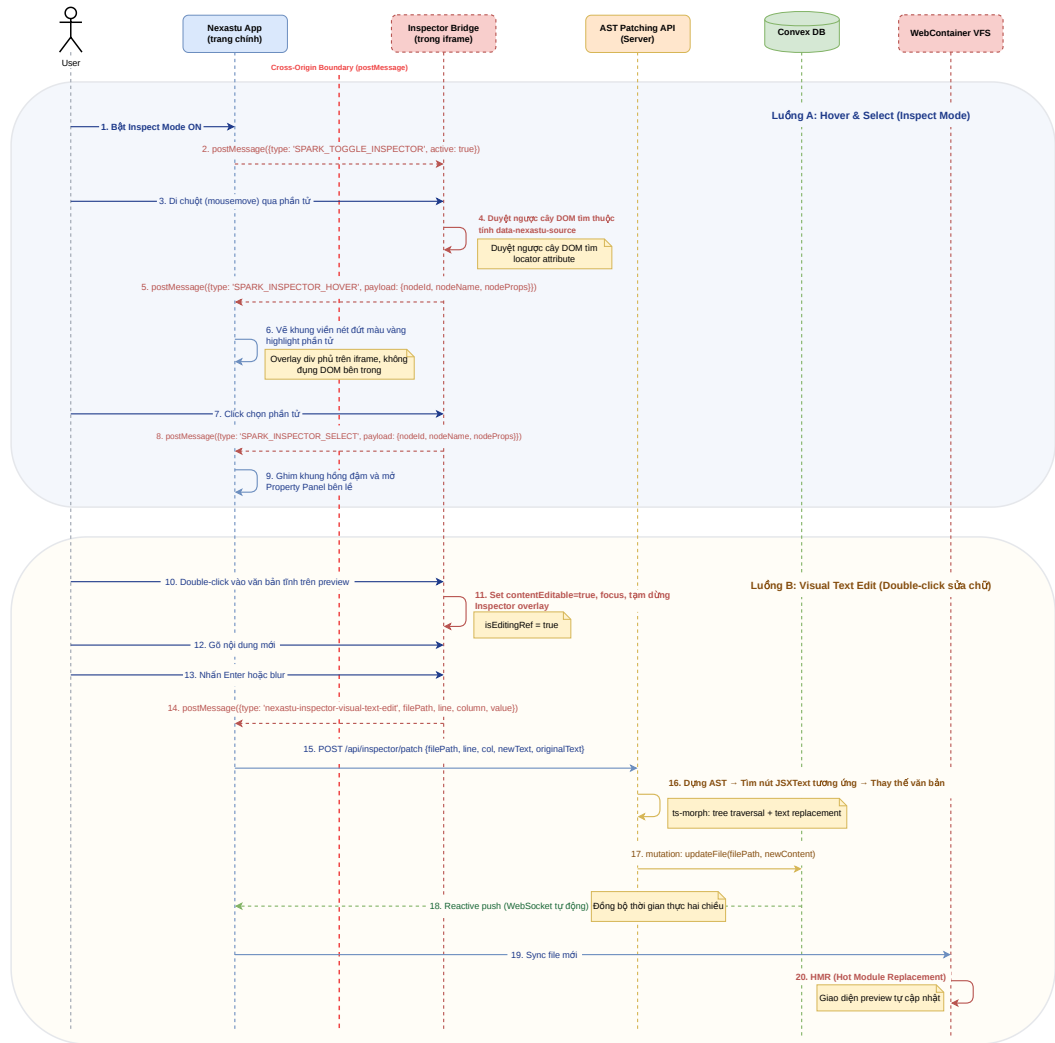
Kiểm soát chất lượng qua Smoke Gate và tự sửa lỗi (Tương ứng bước 13, 14a, 14b và 14c trong sơ đồ): Inngest chạy tiến trình kiểm định Smoke Gate để quét cú pháp mã nguồn TSX/JSX sinh ra. Nếu phát hiện lỗi (thẻ chưa đóng, lỗi import thư viện), hệ thống tự động kích hoạt vòng lặp tự sửa lỗi (tối đa 2 lần) bằng cách gửi mã lỗi ngược lại để DeepSeek API sửa đổi.

Chèn định vị Inspector và hoàn tất dự án (Tương ứng bước 15, 16 và 17 trong sơ đồ): Khi mã nguồn hợp lệ, Inngest thực hiện chèn định vị data-nexastu-source phục vụ tính năng Inspector, sau đó ghi đồng bộ toàn bộ tệp tin dự án và cập nhật trạng thái thành "completed" lên Convex DB.

Đồng bộ hệ thống tệp ảo và khởi chạy Live Preview (Tương ứng bước 18 đến 21 trong sơ đồ): Convex DB phát dữ liệu qua WebSocket. Client tiếp nhận các tệp tin mã nguồn mới, đồng bộ xuống hệ thống tệp ảo (VFS) chạy trên RAM của WebContainer Sandbox, tự động thực thi cài đặt và chạy máy chủ phát triển (npm run dev) để hiển thị Live Preview lên giao diện Iframe của người dùng.

3.3 THIẾT KẾ CƠ CHẾ INSPECTOR VÀ CẦU NỐI TRUYỀN TIN CORS-SAFE

Thách thức cốt lõi khi hiện thực hóa tính năng chỉnh sửa trực quan (Visual Edit) nằm ở việc thiết lập tương tác thời gian thực với `iframe` Preview. Rào cản xuất hiện khi `iframe` Sandbox (chạy trên tên miền của WebContainer như `*.webcontainer-api.io`) và ứng dụng quản trị Nexastu sở hữu hai origin khác nhau, kích hoạt chính sách Same-Origin Policy chặn toàn bộ các truy cập DOM chéo. Để vượt qua rào cản kiến trúc này, một cấu trúc trung gian mang tên Inspector Bridge đã được thiết kế, sử dụng `API window.postMessage` làm giao thức truyền tin an toàn qua origin. Để làm rõ luồng dữ liệu bảo mật này trong chế độ Inspector, sơ đồ tuần tự tại Hình 3.3 sẽ minh họa cấu trúc tương tác chi tiết giữa ứng dụng nền tảng và bộ core nằm trong WebContainer Sandbox.



Hình 3.3: Sơ đồ tuần tự tương tác cầu nối Inspector Bridge. Thể hiện chuỗi thông điệp truyền chéo nguồn CORS-Safe giữa ứng dụng quản lý Nexastu và `iframe` WebContainer Sandbox cho các luồng lựa chọn phần tử (Hover/Select) và chỉnh sửa trực quan (Visual Text Edit).

3.3.1 Cơ chế tương tác trực quan và đồng bộ hóa văn bản (Visual Text Edit)

Cơ chế tương tác trực quan đóng vai trò cốt lõi trong việc tối ưu hóa trải nghiệm chỉnh sửa WYSIWYG của người dùng trên hệ thống Nexastu. Dựa trên sơ đồ tuần tự tương tác được mô tả tại Hình 3.3, quy trình hoạt động chi tiết của Inspector và cầu nối truyền tin CORS-Safe được phân tách thành các cấu phần xử lý cụ thể như sau:

Cơ chế hover và lựa chọn phần tử trực quan: Khi người dùng kích hoạt chế độ Inspect trên thanh công cụ của ứng dụng chính (Host), một thông điệp `{ type: 'SPARK_TOGGLE_INSPECTOR', active: true }` được gửi thông qua `postMessage` vào `iframe` Sandbox. Tại đây, đoạn mã cầu nối `inspector-bridge.tsx` (được hệ thống tự động chèn vào cấu trúc bố cục trang khi sinh mã) lắng nghe sự kiện và thiết lập các bộ lắng nghe sự kiện chuột (`mousemove`, `click`) trên cửa sổ hiển thị.

Khi người dùng di chuột qua một phần tử giao diện, cầu nối sẽ tìm kiếm phần tử chịu tác động, duyệt ngược lên cây DOM để tìm thuộc tính định vị mã nguồn `data-nexastu-source`. Sau đó, cầu nối trích xuất tọa độ hiển thị (`pixel top`, `left`, `width`, `height`) cùng thông tin tệp tin nguồn (ví dụ: `src/components/blocks/hero.tsx:12:4`) và gửi ngược lại cho Host IDE qua `window.parent.postMessage`. Host IDE tiếp nhận thông điệp, thực hiện xác thực nguồn gốc (Origin Validation) và vẽ một khung viền nét đứt màu vàng nổi trên `iframe`. Khi người dùng nhấp chuột (`click`) để chọn phần tử, cầu nối ghim khung màu hồng đậm và mở bảng thuộc tính ở Host IDE hiển thị thông tin chi tiết.

Cơ chế chỉnh sửa văn bản trực quan (Visual Text Edit) và mô hình Double-Loop Synchronization: Tính năng này cho phép người dùng nhấp đúp chuột trực tiếp vào văn bản trên giao diện xem trước (Preview) để chỉnh sửa văn bản tĩnh tại chỗ. Để mang lại trải nghiệm tương tác mượt mà với độ trễ phản hồi bằng 0ms (*Zero-Latency*), hệ thống áp dụng kiến trúc đồng bộ hai luồng (*Double-Loop Synchronization*). Sự kết hợp này giúp giao diện phản hồi tức thì với người dùng trong khi mã nguồn vật lý vẫn được cập nhật bền vững ở chế độ nền. Cơ chế hoạt động của hai luồng đồng bộ này bao gồm:

- **Vòng lặp nhanh phía máy khách (Fast Loop – 0ms):** Khi phát hiện sự kiện nhấp đúp chuột trên phần tử chứa văn bản tĩnh, cầu nối tạm thời thiết lập thuộc tính `contentEditable="true"` cho phần tử đó, tập trung tiêu điểm (`focus`) và tạm dừng lớp phủ `highlight` (đặt cờ `isEditingRef = true`). Khi người dùng gõ phím thay đổi ký tự, giao diện thay đổi tức thì trên trình duyệt nhờ cập nhật trạng thái lạc quan (*optimistic update*).
- **Vòng lặp chậm chạy ngầm (Slow Loop):** Khi người dùng nhấn phím `Enter` hoặc mất tiêu điểm (`blur`), cầu nối tắt thuộc tính `contentEditable`, thu thập nội dung văn bản mới và gửi về Host IDE. Host IDE gửi yêu cầu HTTP POST chứa tọa độ dòng/cột lên API Endpoint `/api/inspector/patch`. Backend sử dụng thư viện `ts-morph` để dựng cây AST từ tệp mã nguồn tương ứng, tìm chính xác nút văn bản tĩnh JSX và thay thế giá trị, sau đó ghi đồng bộ lên Convex

DB. Khi Convex DB cập nhật, sự kiện WebSocket kích hoạt WebContainer Sandbox đồng bộ tệp ảo và kích hoạt cơ chế HMR của Next.js chạy dưới nền.

Giải thuật vá mảng dữ liệu cấu trúc AST (Array Literal Patching): Một thách thức lớn xuất hiện khi các thành phần giao diện động được hiển thị thông qua cơ chế lặp mảng (ví dụ: duyệt danh sách dự án bằng hàm `.map` của JavaScript). Nếu người dùng trực tiếp sửa đổi văn bản của một phần tử con trong danh sách, việc vá mã nguồn trực tiếp vào nút JSX sẽ làm ghi đè và phá hỏng biểu thức lập trình động (như biến `{project.title}` bị thay thế hoàn toàn bằng chuỗi tĩnh).

Để giải quyết triệt để vấn đề này, cầu nối Inspector Bridge được thiết kế để phát hiện các thuộc tính mô tả cấu trúc danh sách bao gồm `data-nx-source-array` (tên biến mảng nguồn), `data-nx-index` (chỉ mục phần tử trong mảng) và `data-nx-field` (tên trường thuộc tính cần chỉnh sửa) được nhúng sẵn trên cây DOM. Khi người dùng hoàn tất việc chỉnh sửa văn bản, yêu cầu vá dữ liệu kèm các tham số mảng này được gửi lên API `/api/inspector/patch`. Giải thuật này giúp bảo toàn nguyên vẹn cấu trúc mã nguồn và biểu thức lập của trang web, đồng thời cập nhật chính xác dữ liệu của từng đối tượng trong danh sách cấu hình. Cụ thể, hàm giải thuật `patchAstArrayLiteral` tại máy chủ thực hiện các bước sau:

1. Tìm kiếm khai báo mảng tĩnh tương ứng với tên biến nhận được trên cây AST.
2. Truy cập vào phần tử con tại vị trí chỉ mục tương ứng trong mảng (`ObjectLiteralExpression`).
3. Xác định thuộc tính gán (`PropertyAssignment`) khớp với tên trường cần sửa đổi.
4. Thay thế an toàn giá trị chuỗi tĩnh cũ của thuộc tính bằng giá trị văn bản mới.

Đồng bộ hóa trạng thái lựa chọn và cơ chế Origin Check bảo mật: Để đảm bảo tính nhất quán của trạng thái giao diện, hệ thống đồng bộ hóa React State `inspectorContexts` và hàm cập nhật `setInspectorContexts` giữa Host IDE và thanh công cụ Chat. Khi người dùng click nút "Xóa hết" hoặc nút tắt trên badge phần tử, React State lập tức được cập nhật trên giao diện chính, đồng thời gửi thông điệp `SPARK_INSPECTOR_CLEAR` để xóa bỏ các khung highlight đồ họa vẽ trên `iframe`.

Để ngăn chặn các lỗ hổng bảo mật chèn tin nhắn giả mạo (*Cross-Site Scripting*), cầu nối thực thi giao thức xác thực nguồn gốc nghiêm ngặt (*Origin Validation Protocol*) nhằm đảm bảo các tin nhắn chỉ đến từ nguồn gốc WebContainer tin cậy và chính Host IDE. Tuy nhiên, để tránh việc chặn nhầm các tin nhắn `postMessage` có nguồn gốc trùng khớp với `window.location.origin`, hệ thống thiết lập bộ lọc chỉ chấp nhận tin nhắn từ các tên miền phụ của WebContainer và domain IDE đã được xác định trước.

3.3.2 Cơ chế quản lý tài nguyên nhị phân và thuật toán vá ảnh (AST Image Patching)

Đối với một trình soạn thảo thiết kế trực tuyến trực quan như Nexastu, việc chèn, thay thế và quản lý các tài nguyên đa phương tiện nhị phân (chẳng hạn như hình ảnh chân dung người dùng, logo dự án hay ảnh chụp sản phẩm) là một nhu cầu thiết yếu. Việc xử lý tài nguyên nhị phân đòi hỏi hai thành phần kỹ thuật chính hoạt động đồng bộ: hạ tầng lưu trữ đám mây an toàn và giải thuật phân tích AST để vá mã nguồn tương ứng.

Đăng ký và Lưu trữ tài nguyên qua Convex Storage: Để lưu trữ và phân phát các tệp tin nhị phân lớn mà không làm phình dung lượng cơ sở dữ liệu dạng văn bản, Nexastu sử dụng dịch vụ đám mây *Convex Storage*. Quy trình đăng ký và lưu trữ diễn ra như sau:

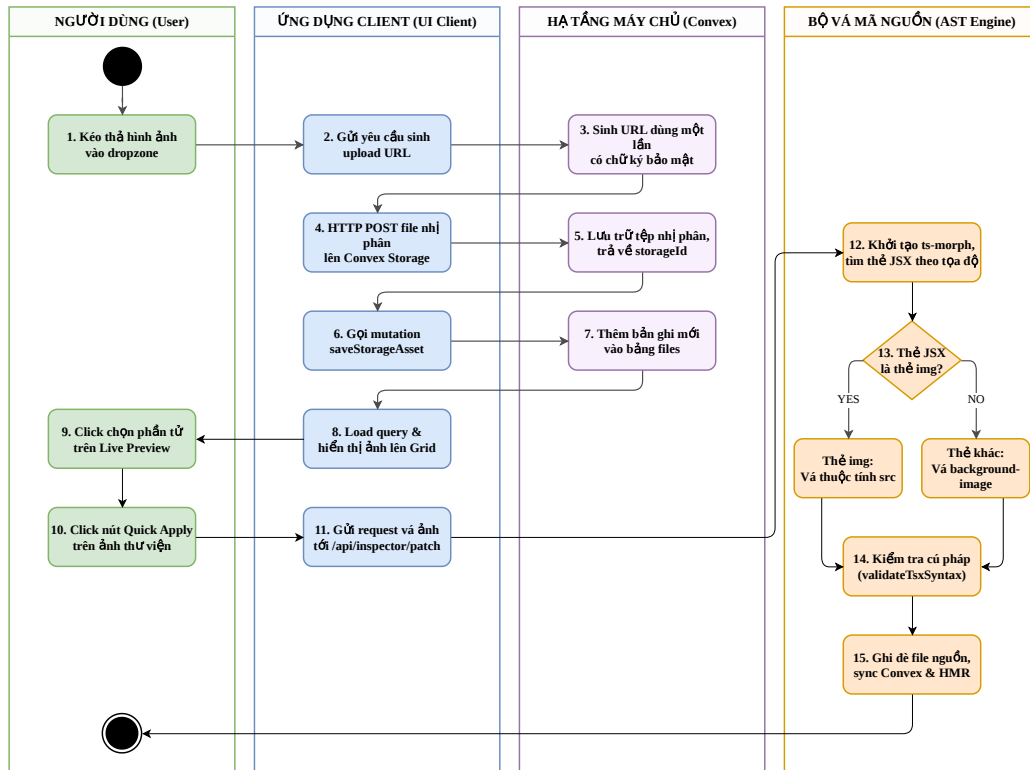
- **Sinh URL tải lên:** Phía máy khách (Client) gửi yêu cầu tới mutation `generateUploadUrl` trên Convex DB để sinh một URL tải lên dùng một lần có chữ ký bảo mật.
- **Tải lên tệp nhị phân:** Client thực hiện một yêu cầu HTTP POST để truyền trực tiếp dữ liệu nhị phân (ảnh JPEG, PNG...) lên URL vừa tạo. Sau khi tải lên thành công, Convex trả về một mã định danh tài nguyên duy nhất `storageId`.
- **Lưu trữ tài liệu tham chiếu:** Hệ thống gọi mutation `saveStorageAsset` để ghi nhận một bản ghi tệp tin mới vào bảng `files` của cơ sở dữ liệu. Bản ghi này chứa `storageId` để liên kết với tệp nhị phân trên bộ lưu trữ đám mây. Việc lưu trữ `storageId` thay vì chuỗi mã hóa Base64 giúp tránh lỗi tràn bộ đệm RAM VFS và ngăn ngừa các lỗi phân tích cú pháp mã nguồn UTF-8 trong quá trình biên dịch (compilation).
- **Phân giải URL hiển thị:** Khi hiển thị thư viện ảnh cho người dùng, query `getProjectAssets` sẽ duyệt tìm tất cả các bản ghi có chứa trường `storageId` và sử dụng hàm hệ thống `ctx.storage.getUrl(file.storageId)` để chuyển đổi chúng thành các URL công khai (public URL) có thể truy cập được từ trình duyệt.

Giải thuật vá thuộc tính hình ảnh trên cây cú pháp AST (AST Image Patching): Khi người dùng chọn một hình ảnh từ thư viện (Asset Manager) và nhấn nút áp dụng nhanh (*Quick Apply*) vào một phần tử đang được chọn trên giao diện xem trước thời gian thực (Live Preview), hệ thống sẽ gửi một thông điệp chỉnh sửa kiểu hình ảnh (`editType: "src"`) kèm theo tọa độ mã nguồn (`line`, `column`, `filePath`) và URL của hình ảnh mới lên điểm cuối API (API Endpoint) `/api/inspector/patch`. Quy trình kéo thả ảnh và vá cây AST được mô tả chi tiết thông qua sơ đồ hoạt động tại Hình 3.4.

Tại đây, hàm `patchTsxAstLocation` sử dụng bộ phân tích cú pháp `ts-morph` để dựng cây AST từ tệp mã nguồn và thực thi giải thuật tìm kiếm, vá thuộc tính như mô tả tại thuật toán dưới đây:

- **Định vị thẻ mở JSX:** Từ vị trí tọa độ nhận được, thuật toán duyệt ngược lên để tìm nút đại diện cho thẻ mở JSX (`JsxOpeningElement` hoặc `JsxSelfClosingElement`).
- **Tìm thuộc tính nguồn ảnh có sẵn:** Thuật toán kiểm tra sự tồn tại của các thuộc tính biểu thị nguồn ảnh phổ biến là `src`, `avatar`, hoặc `image`. Nếu một trong các thuộc tính này tồn tại dưới dạng thuộc tính tĩnh (`JsxAttribute`), hệ thống sẽ gọi phương thức `setInitializer` để ghi đè đường dẫn ảnh cũ bằng URL mới.
- **Tự động chèn thuộc tính cho thẻ ảnh trống:** Nếu thuộc tính ảnh chưa tồn tại nhưng tên thẻ của phần tử được xác định là `img`, thuật toán sẽ chủ động tạo mới thuộc tính `src` thông qua phương thức `addAttribute` với giá trị khởi tạo là chuỗi URL mới.
- **Vá ảnh nền CSS (*Background Image*):** Trong trường hợp phần tử được chọn không phải thẻ `img` và không có các thuộc tính nguồn ảnh trực tiếp, thuật toán sẽ tìm kiếm chuỗi ký tự dạng chuỗi chữ (`String Literal`) nằm tại vị trí con trỏ. Chuỗi này thường là định nghĩa URL ảnh nền trong các lớp CSS Tailwind (ví dụ: `bg-[url(...)]`) hoặc trong thuộc tính định dạng nội dòng (`inline style`). Nếu tìm thấy nút `StringLiteral`, hệ thống thực hiện cập nhật giá trị của nút này thành đường dẫn ảnh mới.

Sau khi vá cây AST, mã nguồn dạng văn bản mới sinh ra sẽ đi qua lớp bảo vệ kiểm định cú pháp `validateTsxSyntax` để đảm bảo không phát sinh lỗi thẻ lồng nhau trước khi lưu vào cơ sở dữ liệu Convex DB và kích hoạt HMR cập nhật giao diện xem trước (preview).



Hình 3.4: Sơ đồ hoạt động luồng tải ảnh và vá mã nguồn AST trong Nexastu.

3.3.3 Cơ chế phục hồi khối giao diện bảo toàn dữ liệu (Non-Destructive Block Restore)

Một vấn đề nghiêm trọng đối với các trình soạn thảo WYSIWYG truyền thống là thao tác phục hồi (*Restore Block*) về bố cục thiết kế mặc định của template thường ghi đè thô bạo lên tệp tin nguồn, dẫn đến việc xóa bỏ toàn bộ dữ liệu cá nhân hóa (như thông tin CV, dự án thực tế, kỹ năng) mà người dùng đã dày công tinh chỉnh. Để khắc phục triệt để hạn chế này, Nexastu thiết kế cơ chế khôi phục khối giao diện bảo toàn dữ liệu (*Non-Destructive Block Restore*) thông qua giải thuật chồng chập thông tin.

Lưu trữ dữ liệu người dùng cô lập (`debug-info.json`): Trong quá trình trích xuất thông tin CV và cập nhật nội dung, hệ thống duy trì một tệp cấu trúc dữ liệu metadata cô lập tại thư mục `.nexastu/debug-info.json` và đồng bộ liên tục lên Convex DB. Tệp tin này đóng vai trò là kho lưu trữ trạng thái dữ liệu người dùng đã phân tích (`debugInfo.parsedUserData`) bao gồm các trường thông tin cá nhân cơ bản, học vấn, kinh nghiệm và danh sách dự án dạng JSON thô, độc lập hoàn toàn với mã nguồn giao diện TSX.

Giải thuật chồng chập dữ liệu AST (`applyUserContentPatches`): Khi người dùng kích hoạt lệnh phục hồi một block giao diện trên Live Preview, yêu cầu được gửi tới API Endpoint `/api/inspector/restore-block`. Giải thuật này giúp phục hồi hoàn hảo block giao diện về cấu trúc thiết kế và thẩm mỹ ban đầu của template sạch, nhưng vẫn bảo toàn 100% toàn bộ nội dung thông tin cá nhân hóa của người dùng. Quy trình xử lý chi tiết diễn ra như sau:

1. **Tìm nạp template sạch:** API Route tải cấu trúc mã nguồn của block mẫu nguyên bản từ thư mục template mặc định.
2. **Truy xuất dữ liệu cô lập:** API Route sử dụng `ConvexHttpClient` đọc tệp `debug-info.json` của dự án để trích xuất cấu trúc dữ liệu người dùng `parsedUserData`.
3. **Khởi tạo AST và chồng chập:** Trình xử lý khởi dựng cây AST từ block template sạch bằng `ts-morph`. Hàm `applyUserContentPatches` duyệt tìm các nút tương ứng với các thuộc tính dữ liệu (ví dụ: các biến hiển thị họ tên, chức danh, mô tả) và thực hiện ghi đè giá trị của các nút AST này bằng dữ liệu thực tế của người dùng.
4. **Kiểm định và Đồng bộ:** Mã nguồn sau khi chồng chập được đưa qua bộ kiểm duyệt cú pháp để đảm bảo không bị lỗi vỡ thẻ (Clobbering Guard), sau đó ghi đè xuống đĩa ảo và kích hoạt HMR.

3.4 THIẾT KẾ CƠ CHẾ ĐỊNH TUYẾN ĐA PHƯƠNG THỨC THÔNG MINH

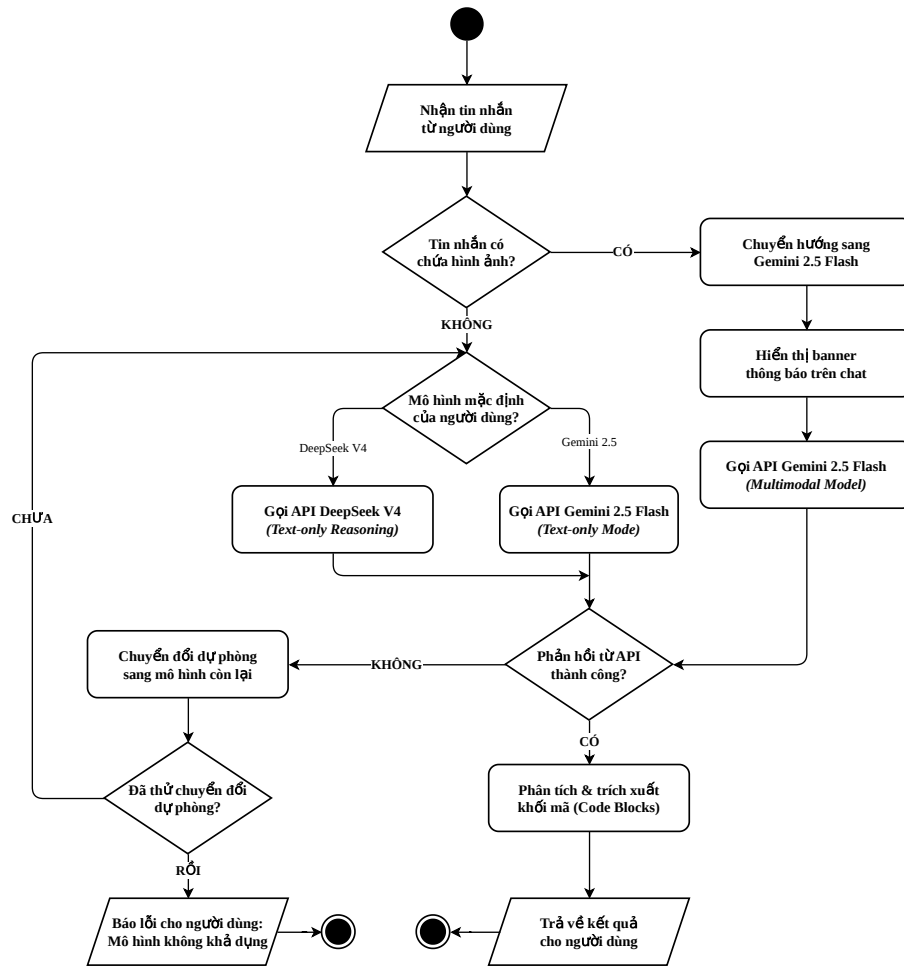
Để tối ưu hóa quá trình tương tác giữa người dùng và trợ lý thiết kế AI-Agent, hệ thống Nexastu triển khai cơ chế định tuyến mô hình kết hợp với giải thuật tự động giám sát thực thi để ngăn chặn các báo cáo thành công ảo. Cơ chế này bao gồm hai thành phần kỹ thuật cốt lõi hoạt động đồng bộ:

3.4.1 Cơ chế định tuyến đa phương thức dự phòng (Multimodal Routing Fallback)

Để hệ thống AI-Agent xử lý tối ưu các dạng hội thoại khác nhau của người dùng, Nexastu triển khai cơ chế định tuyến mô hình thông minh (Model Routing). Cơ chế này giúp duy trì trải nghiệm người dùng liền mạch, cho phép người dùng thoải mái tải ảnh vẽ tay phác thảo giao diện hoặc ảnh chụp mẫu giao diện thực tế mà không gặp lỗi không tương thích mô hình. Lưu đồ thuật toán định tuyến đa phương thức dự phòng được thể hiện cụ thể trong Hình 3.5, hoạt động thông qua các bước logic cụ thể sau:

- Hệ thống nhận tin nhắn đầu vào từ người dùng (User Message).
- Kiểm tra xem tin nhắn có chứa tệp tin đính kèm dạng hình ảnh (attachments) hay không:
 - Trường hợp 1: Có hình ảnh đính kèm:**
 - Vì mô hình lập trình lý luận DeepSeek V4 là mô hình thuần văn bản (Text-Only Reasoning Model), hệ thống sẽ tự động chuyển hướng cuộc gọi xử lý sang **Gemini 2.5 Flash** (Multimodal Model).
 - Gemini sẽ phân tích nội dung hình ảnh phối hợp với yêu cầu văn bản để sinh ra mã nguồn cập nhật phù hợp.
 - Hệ thống tự động đính kèm một nhãn thông báo (Banner) trực quan trên giao diện chat báo hiệu: *"Đã tự động chuyển đổi mô hình sang Gemini để phân tích hình ảnh đa phương thức"*.
 - Trường hợp 2: Không có hình ảnh đính kèm (Thuần văn bản):**
 - Hệ thống tiếp tục sử dụng mô hình được người dùng cấu hình mặc định (ví dụ: DeepSeek V4 hoặc Gemini 2.5 Flash) để thực hiện suy luận lập trình sâu sắc, viết code chuẩn xác.
- Kiểm tra phản hồi của API và thực thi cơ chế tự động chuyển đổi dự phòng (Fault-Tolerance Fallback): Sau khi gửi yêu cầu đến mô hình đã chọn (Gemini hoặc DeepSeek), hệ thống kiểm tra phản hồi đầu ra:

- **Trường hợp gọi API thành công:** Tiến hành phân tích cấu trúc phản hồi, bóc tách các khối mã nguồn (code blocks) và gửi trả kết quả cập nhật về cho người dùng.
- **Trường hợp gọi API thất bại (gặp sự cố mạng, lỗi quá tải Rate Limit hoặc lỗi hệ thống):** Hệ thống lập tức kích hoạt cơ chế chuyển đổi dự phòng tự động. Trạng thái đã chuyển đổi dự phòng sẽ được ghi nhận và kiểm định (bước “Đã thử chuyển đổi dự phòng?” trên sơ đồ). Nếu chưa thử chuyển đổi dự phòng (trạng thái là “CHƯA”), hệ thống chuyển hướng cuộc gọi xử lý sang mô hình đối tác còn lại trong cấu hình hệ thống (ví dụ: chuyển từ DeepSeek V4 sang Gemini 2.5 Flash hoặc ngược lại) rồi lặp lại quy trình gọi API. Nếu đã thử chuyển đổi mà API của mô hình còn lại vẫn tiếp tục thất bại (trạng thái là “RỒI”), hệ thống mới chính thức trả về thông báo lỗi: *"Mô hình không khả dụng"*.



Hình 3.5: Sơ đồ thuật toán định tuyến đa phương thức dự phòng. Quy trình phân tích đính kèm tin nhắn và tự động chuyển đổi từ DeepSeek sang Gemini.

3.4.2 Giải thuật tự động thử lại ngăn chặn hiện tượng báo cáo thành công ảo (Retry Loop Guard)

Trong quá trình tương tác giữa người dùng và trợ lý thiết kế AI-Agent, một vấn đề phổ biến xảy ra đối với các mô hình ngôn ngữ lớn là hiện tượng *báo cáo thành công ảo* (*Phantom Success*). Hiện tượng này xuất hiện khi mô hình AI phản hồi bằng văn bản khẳng định đã thực hiện thay đổi thành công (ví dụ: “Tôi đã cập nhật màu nền của nút bấm sang màu xanh lá cho bạn”), nhưng thực tế lại bỏ qua việc gọi công cụ ghi đĩa ảo (`updateFile` hoặc `createFiles`). Điều này khiến mã nguồn trên VFS không thay đổi, gây hiểu lầm và làm giảm nghiêm trọng trải nghiệm của người dùng. Để giải quyết triệt để lỗi này, Nexastu triển khai giải thuật tự động kiểm tra và cưỡng bức thực thi công cụ (*Retry Loop Guard*) ngay trong luồng xử lý tin nhắn của Inngest Worker (`process-message.ts`).

1. **Xác định ý định chỉnh sửa mã nguồn (`expectsFileEdit`):** Trước khi chuyển yêu cầu đến AI-Agent, hệ thống tiến hành phân tích tin nhắn của người dùng dựa trên ba điều kiện:

- **Điều kiện 1:** Người dùng đang sử dụng công cụ Inspect để chọn một phần tử trực quan cụ thể (tin nhắn tự động chèn thẻ định vị `<selected_element>`).
- **Điều kiện 2:** Tin nhắn chứa các động từ hành động thể hiện mong muốn sửa đổi mã nguồn (như *sửa, chỉnh, dịch, viết hóa, thay thế, thêm, xóa, tạo, cập nhật, update, change, edit*).
- **Điều kiện 3:** Tin nhắn không phải là một câu hỏi truy vấn thông tin thuần túy (không chứa các từ khóa hỏi: *tại sao, giải thích, là gì, ở đâu, check hộ, show me, explain, why, how, what*).

Nếu một trong các điều kiện trên thỏa mãn, biến cờ hiệu `expectsFileEdit` sẽ được thiết lập là `true`, biểu thị rằng hệ thống kỳ vọng một thao tác ghi file thực tế sẽ diễn ra.

2. **Kiểm tra vết thực thi công cụ (Tool Execution Audit):** Sau khi AI-Agent hoàn thành chu kỳ suy luận, hệ thống bóc tách danh sách các công cụ đã thực thi thành công (`toolProof.toolsUsed`).

3. **Kích hoạt chốt chặn tự động thử lại (`needsFileUpdateRetry`):** Hệ thống đánh giá điều kiện chốt chặn:

$$\text{needsFileUpdateRetry} = \text{expectsFileEdit} \wedge \neg(\text{“updateFile”} \in \text{Tools} \vee \text{“createFiles”} \in \text{Tools}) \quad (3.1)$$

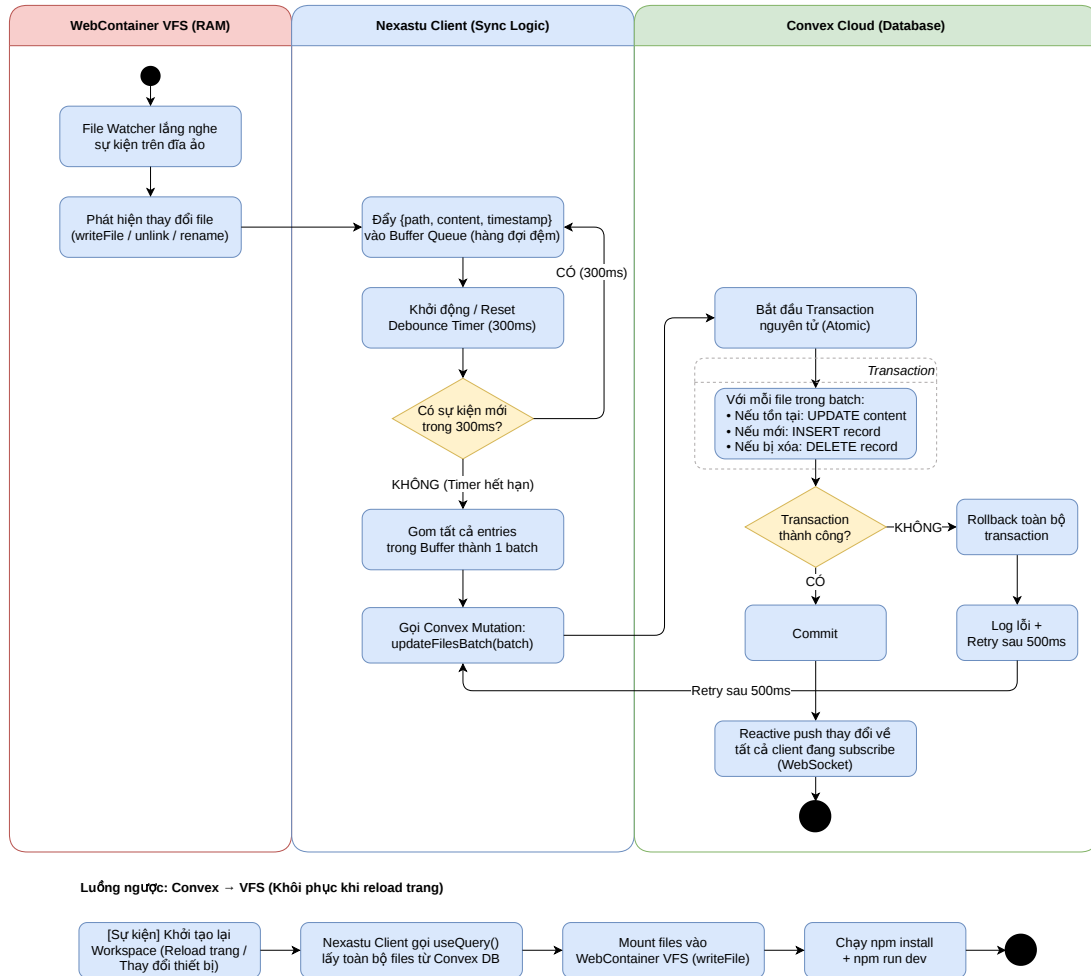
Nếu `needsFileUpdateRetry` là `true` (tức là người dùng muốn sửa file nhưng AI-Agent chưa hề gọi công cụ ghi tệp nào), hệ thống sẽ lập tức chặn việc trả lời người dùng, đồng thời tự động kích hoạt một chu kỳ chạy bổ sung (*cưỡng bức thực thi*) bằng cách gửi lệnh hệ thống ẩn:

“Bạn BẮT BUỘC phải sử dụng công cụ `updateFile` để áp dụng và lưu các thay đổi mã nguồn JSX/CSS. Người dùng đang chờ nhìn thấy thay đổi trực quan trên trang web preview. Tìm đúng tệp nguồn, đọc nó và gọi `updateFile` ngay lập tức. KHÔNG ĐƯỢC trả lời cho đến khi bạn đã lưu tệp thành công.”

3.5 THIẾT KẾ CƠ CHẾ ĐỒNG BỘ HÓA VFS VÀ PHẢN HỒI THỜI GIAN THỰC

3.5.1 Thiết kế thuật toán đồng bộ hóa trạng thái VFS lên Convex DB

Để duy trì trạng thái bền vững của dự án Next.js trên RAM WebContainer, hệ thống triển khai giải thuật đồng bộ hóa hai chiều thời gian thực thông qua Convex DB. Thuật toán này sử dụng hàng đợi đệm và kỹ thuật debounce nhằm tối ưu hóa lưu lượng truyền tải và ngăn chặn xung đột ghi. Lưu đồ hoạt động (Activity Diagram) mô tả quy trình này được trình bày trong Hình 3.6.



Hình 3.6: Lưu đồ thuật toán đồng bộ hóa trạng thái VFS sang Convex. Quy trình gom lô thay đổi (Batching) bằng debounce timer trước khi thực thi giao dịch nguyên tử.

Quy trình xử lý tuần tự của thuật toán bao gồm:

1. **File Watcher:** Lắng nghe các sự kiện cập nhật đĩa ảo trong WebContainer (ghi đè file, đổi tên, xóa file).
2. **Enqueue:** Đẩy đường dẫn và nội dung thay đổi vào hàng đợi đệm tạm thời.
3. **Debounce Timer:** Thiết lập thời gian chờ (ví dụ 300ms). Nếu phát sinh sự kiện mới trong khoảng thời gian này, đồng hồ đếm ngược sẽ được khởi động lại.
4. **Batch Update:** Khi đồng hồ kết thúc (AI đã sinh xong file hoặc người dùng dừng gõ), toàn bộ hàng đợi được đóng gói thành một đối tượng JSON duy nhất và gửi lên server Convex.
5. **Convex Transaction:** Thực thi hàm mutation `updateFilesBatch` để cập nhật đồng bộ cơ sở dữ liệu trong cùng một transaction nguyên tử duy nhất.

Bên cạnh luồng đồng bộ xuôi từ VFS lên cơ sở dữ liệu đám mây, hệ thống cũng thiết kế luồng đồng bộ ngược (Convex $\rightarrow$ VFS) nhằm khôi phục không gian làm việc của người dùng khi khởi động hoặc nạp lại (reload) trang web. Cụ thể, khi người dùng mở lại dự án hoặc truy cập từ một thiết bị khác, ứng dụng client sẽ gọi query hệ thống để lấy toàn bộ các tệp tin hiện hành của dự án từ Convex DB. Ngay sau khi nhận được dữ liệu, lớp client-side sync sẽ lặp qua danh sách và gọi lệnh ghi đè vật lý vào đĩa ảo VFS trong RAM WebContainer Sandbox. Tiến trình khởi chạy Node.js và Next.js development server sau đó sẽ tự động cài đặt các dependencies và chạy dự án để khôi phục trạng thái Live Preview giống hoàn toàn với phiên làm việc trước đó.

3.5.2 Cơ chế tự động kích hoạt Hot Module

Replacement (HMR) cập nhật Live Preview

Để mang lại trải nghiệm chỉnh sửa giao diện tức thời mà không yêu cầu người dùng phải tải lại trang thủ công, Nexastu xây dựng cơ chế tự động kích hoạt nạp lại nóng (Hot Module Replacement - HMR) đồng bộ qua kết nối thời gian thực WebSocket. Cơ chế HMR tự động này giúp chu trình phản hồi trực quan từ lúc người dùng thay đổi thiết kế trên thanh công cụ, qua AST và mã nguồn, đồng bộ DB, cho tới khi giao diện Preview được cập nhật diễn ra hoàn toàn tự động trong thời gian dưới 100ms mà không cần sử dụng bất kỳ cơ chế nạp lại trang truyền thống (Full Page Reload) nào. Tiến trình hoạt động của cơ chế này diễn ra thông qua các bước khép kín như sau:

1. **Đăng ký lắng nghe biến động dữ liệu (Reactive Subscription):** Trình duyệt client duy trì một kết nối WebSocket liên tục tới máy chủ Convex DB thông qua hook React `useQuery`. Khi cơ sở dữ liệu hoàn thành giao dịch ghi gộp `updateFilesBatch` (quy trình được thiết kế tại Mục 3.5.1),

Convex DB sẽ lập tức đẩy sự kiện thay đổi kèm danh sách các tệp tin mới xuống client thông qua kênh truyền WebSocket với độ trễ cực thấp dưới 20ms.

2. **Đồng bộ hệ thống tệp tin ảo trong bộ nhớ (In-memory VFS Update):** Sau khi nhận dữ liệu cập nhật từ WebSocket, lớp dịch vụ VFS chạy trên môi trường sandbox của trình duyệt (WebContainer) thực hiện ghi đè nội dung mới lên ổ đĩa ảo RAM. Do ổ đĩa của WebContainer được giả lập trực tiếp trên bộ nhớ trình duyệt, thao tác ghi tệp tin diễn ra gần như tức thời (dưới 5ms) và không chịu giới hạn IO vật lý.
3. **Theo dõi biến động và Phát hiện tệp tin thay đổi (File System Watcher):** Dev Server của Next.js chạy ngầm bên trong WebContainer sử dụng thư viện theo dõi file (như `chokidar`) để giám sát các thay đổi trên cây thư mục ảo. Khi phát hiện sự kiện thay đổi của các tệp tin mã nguồn (như `.tsx`, `.css`), Dev Server sẽ tự động xác định các mô-đun bị ảnh hưởng trong đồ thị phụ thuộc (dependency graph).
4. **Kích hoạt nạp lại nóng (HMR Trigger):** Next.js Compiler (sử dụng SWC compiler) thực hiện biên dịch lại biệt lập chỉ các phần mã nguồn bị thay đổi mà không cần tải lại toàn bộ dự án. Sau đó, Dev Server gửi gói tin cập nhật (HMR Update Package) chứa mã JavaScript mới biên dịch thông qua kết nối WebSocket nội bộ của WebContainer (thường chạy trên cổng 3000) tới thành phần Live Preview Iframe.
5. **Cập nhật cây DOM và Giữ nguyên trạng thái (DOM Replacement & State Preservation):** Mã HMR Runtime chạy bên trong Iframe nhận gói tin cập nhật, thực hiện thay thế mô-đun React cũ bằng mô-đun React mới trên cây DOM ảo (Virtual DOM) và kết xuất (re-render) lại phần giao diện tương ứng. Quá trình này được thực hiện trực tiếp trên DOM mà không làm mất đi các trạng thái hiện tại của ứng dụng (chẳng hạn như dữ liệu nhập trong các ô input chưa lưu), giúp duy trì tính liên tục và mượt mà cho trải nghiệm người dùng.

3.6 THIẾT KẾ CƠ SỞ DỮ LIỆU HỆ THỐNG

Hệ thống Nexastu sử dụng cơ sở dữ liệu thời gian thực *Convex DB* làm nguồn lưu trữ dữ liệu duy nhất và đáng tin cậy (*Single Source of Truth*). Điểm khác biệt lớn nhất giữa Convex DB và các hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) truyền thống là mô hình lưu trữ tài liệu định hướng quan hệ (*Document-relational Model*). Cơ chế này kết hợp tính linh hoạt của cấu trúc dữ liệu JSON với sự chặt chẽ của các mối liên kết khóa ngoại và tính chất giao dịch ACID đầy đủ ở cấp độ máy chủ. Để hiện thực hóa các tính năng cốt lõi bao gồm quản lý trạng thái dự án, lưu trữ cây tệp tin ảo thời gian thực (VFS), quản lý lịch sử hội thoại với trợ lý AI và tích hợp cổng thanh toán VietQR cho cửa hàng điện tử storefront, cơ sở dữ liệu Nexastu được thiết kế gồm 6 thực thể chính, được đặc tả chi tiết thông qua các lược đồ kiểu (*schema*) dưới đây.

3.6.1 Bảng dự án (projects)

Bảng **projects** đóng vai trò là thực thể gốc (Aggregate Root) trong mô hình dữ liệu của Nexastu, lưu trữ toàn bộ thông tin cấu hình, trạng thái xử lý nghiệp vụ và các thiết lập tích hợp của từng dự án website portfolio. Mỗi bản ghi trong bảng được định danh duy nhất và liên kết chặt chẽ với hệ thống xác thực Clerk Auth qua khóa ngoại **ownerId** nhằm bảo đảm nguyên lý cô lập tài nguyên giữa các người dùng (multi-tenancy). Các trạng thái tiến trình xử lý bất đồng bộ như sinh mã (**generationStage**), nhập dữ liệu từ GitHub (**importStatus**) và xuất bản mã nguồn (**exportStatus**) được quản lý chặt chẽ qua các kiểu dữ liệu union nhằm hỗ trợ giao diện người dùng cập nhật theo thời gian thực. Cấu trúc chi tiết của bảng **projects** được đặc tả cụ thể trong Bảng 3.2.

Bảng 3.2: Đặc tả cấu trúc dữ liệu bảng projects

Tên trường	Kiểu dữ liệu	Mô tả chi tiết và Ràng buộc
_id	<code>v.id("projects")</code>	Khóa chính (tự động sinh bởi Convex).
name	<code>v.string()</code>	Tên dự án website portfolio.
ownerId	<code>v.string()</code>	ID tài khoản người sở hữu (liên kết với Clerk Auth).
tenantId	<code>v.optional(string)</code>	ID định danh nhóm/tổ chức (tùy chọn).
template	<code>v.optional(string)</code>	Tên giao diện boilerplate được lựa chọn làm gốc.
generationStage	<code>v.optional(union)</code>	Trạng thái tiến trình sinh AI (parsing_input , generating_code , syncing_files , completed , failed).
importStatus	<code>v.optional(union)</code>	Trạng thái nhập từ nguồn ngoài (importing , completed , failed).
exportStatus	<code>v.optional(union)</code>	Trạng thái xuất bản mã nguồn (exporting , completed , failed , cancelled).
exportRepoUrl	<code>v.optional(string)</code>	Đường dẫn kho chứa GitHub sau khi xuất bản thành công.
settings	<code>v.optional(object)</code>	Cấu hình môi trường WebContainer gồm: lệnh cài đặt (installCommand), lệnh chạy (devCommand), đường dẫn xem trước (previewPath).
ecommerceSettings	<code>v.optional(object)</code>	Cấu hình thanh toán trực tuyến tích hợp: thông tin tài khoản ngân hàng và khóa API của cổng PayOS.
updatedAt	<code>v.number()</code>	Dấu thời gian cập nhật gần nhất (Unix timestamp).

3.6.2 Bảng lưu trữ tệp tin (files)

Bảng **files** là hạt nhân kỹ thuật của hệ thống tệp ảo (VFS) trên đám mây, chịu trách nhiệm tái tạo toàn bộ cấu trúc thư mục và nội dung mã nguồn của dự án Next.js tại trình duyệt WebContainer. Bảng được thiết kế theo cấu trúc phân cấp cây thư mục thông qua mối quan hệ khóa ngoại tự tham chiếu đệ quy (trường **parentId** liên kết ngược lại khóa chính của bảng **files**). Để tối ưu hóa hiệu năng lưu trữ và tránh hiện tượng bùng nổ dung lượng RAM của VFS khi đồng bộ hóa, hệ thống phân tách rõ ràng cơ chế xử lý: nội dung văn bản mã nguồn được lưu trữ trực tiếp trong trường **content**, trong khi các tài nguyên đa phương tiện nhị phân lớn (như hình ảnh, tệp tin PDF) được đẩy vào dịch vụ Convex File Storage và tham chiếu qua khóa ngoại **storageId** hệ thống.

Bảng 3.3: Đặc tả cấu trúc dữ liệu bảng files

Tên trường	Kiểu dữ liệu	Mô tả chi tiết và Ràng buộc
_id	<code>v.id("files")</code>	Khóa chính định danh tệp tin hoặc thư mục.
projectId	<code>v.id("projects")</code>	Khóa ngoại liên kết trực tiếp với bảng projects .
parentId	<code>v.optional(v.id("files"))</code>	Khóa ngoại đệ quy liên kết với thư mục chi tiết để dựng cây tệp tin.
name	<code>v.string()</code>	Tên tệp hoặc tên thư mục (ví dụ: <code>page.tsx</code> , <code>components</code>).
type	<code>v.union("file", "folder")</code>	Loại thực thể: tệp tin hoặc thư mục.
content	<code>v.optional(string)</code>	Nội dung mã nguồn định dạng văn bản (chỉ áp dụng đối với tệp tin văn bản).
storageId	<code>v.optional(v.id("_storage"))</code>	Khóa ngoại tham chiếu đến Convex File Storage (dành cho tệp nhị phân như hình ảnh, PDF).
updatedAt	<code>v.number()</code>	Dấu thời gian cập nhật tệp tin gần nhất.

3.6.3 Bảng quản lý hội thoại và tin nhắn (conversations & messages)

Mô hình dữ liệu hội thoại trong Nexastu được thiết kế dựa trên mối quan hệ một-nhiều giữa phiên hội thoại (**conversations**) và chi tiết các tin nhắn trao đổi (**messages**). Cấu trúc này đóng vai trò quan trọng trong việc duy trì ngữ cảnh liên tục cho trợ lý AI-Agent, cho phép hệ thống phân tích lịch sử trò chuyện và thực hiện các hành động sửa đổi mã nguồn chính xác. Bảng **conversations** đóng vai trò lưu giữ thông tin tổng quát của mỗi phiên chat, bao gồm tiêu đề tóm tắt được sinh tự động và nhà cung cấp

mô hình AI tương tác mặc định (**aiProvider**) như được mô tả trong Bảng 3.4.

Bảng 3.4: Đặc tả cấu trúc dữ liệu bảng conversations

Tên trường	Kiểu dữ liệu	Mô tả chi tiết và Ràng buộc
_id	v.id("conversations")	Khóa chính của phiên hội thoại.
projectId	v.id("projects")	Khóa ngoại tham chiếu đến dự án chứa cuộc hội thoại.
title	v.string()	Tiêu đề tóm tắt của cuộc trò chuyện (ví dụ: “Yêu cầu sửa header”).
aiProvider	v.optional(string)	Mô hình AI đang được sử dụng (gemini hoặc deepseek).
updatedAt	v.number()	Dấu thời gian cập nhật phiên chat gần nhất.

Bảng **messages** chịu trách nhiệm lưu trữ chi tiết từng tin nhắn tương ứng với một phiên hội thoại cụ thể. Để đáp ứng yêu cầu theo dõi hoạt động phức tạp của AI-Agent, bảng không chỉ lưu trữ nội dung văn bản tĩnh mà còn tích hợp các cấu trúc dữ liệu phức tạp bao gồm **toolProof** (ghi nhận vết thực thi các công cụ chỉnh sửa tệp tin) và mảng **progressSteps** (lưu vết chi tiết tiến trình thực thi ngầm từng bước để đồng bộ trạng thái trực quan lên giao diện người dùng). Đặc tả cấu trúc chi tiết của bảng **messages** được thể hiện trong Bảng 3.5.

Bảng 3.5: Đặc tả cấu trúc dữ liệu bảng messages

Tên trường	Kiểu dữ liệu	Mô tả chi tiết và Ràng buộc
_id	v.id("messages")	Khóa chính của tin nhắn.
conversationId	v.id("conversations")	Khóa ngoại liên kết với phiên hội thoại tương ứng.
projectId	v.id("projects")	Khóa ngoại tham chiếu đến dự án.
role	v.union("user", "assistant")	Vai trò của đối tượng gửi tin nhắn.
content	v.string()	Nội dung văn bản chi tiết của tin nhắn.
status	v.optional(union)	Trạng thái xử lý tin nhắn của AI (processing , completed , cancelled).
toolProof	v.optional(object)	Lưu vết kỹ thuật: danh sách các công cụ đã chạy (toolsUsed) và các tệp bị sửa đổi (filesTouched).
progressSteps	v.optional(array)	Mảng lưu trữ các bước xử lý chi tiết kèm dấu thời gian và trạng thái chạy để hiển thị tiến trình thời gian thực cho người dùng.

3.6.4 Bảng sản phẩm và đơn hàng thương mại điện tử (products & orders)

Để đáp ứng tính năng mở rộng từ trang thông tin cá nhân (*portfolio*) thuần túy sang mô hình thương mại điện tử tích hợp cổng thanh toán trực tiếp VietQR, Nexastu triển khai cặp thực thể quản lý sản phẩm (**products**) và đơn đặt hàng (**orders**). Kiến trúc này hoạt động kết hợp với thuộc tính cấu hình API ngân hàng nhận tiền (**ecommerceSettings**) trong bảng dự án để tạo thành một chu trình khép kín từ lúc chọn hàng cho đến khi đối soát giao dịch tự động. Trong đó, bảng **products** chịu trách nhiệm lưu trữ thông tin danh mục các mặt hàng kỹ thuật số hoặc dịch vụ thương mại.

- **Quản lý thông tin hiển thị:** Các trường **name**, **price**, **description**, và **image** chứa dữ liệu trực quan về sản phẩm phục vụ cho tầng kết xuất giao diện cửa hàng trực tuyến (*storefront*). Trường **image** lưu trữ URL liên kết tới hệ thống Convex Storage nhằm hiển thị ảnh sản phẩm với độ trễ thấp nhất.
- **Kiểm soát lượng bán và giới hạn tồn kho:** Trường **stock** lưu trữ số lượng hàng khả dụng trong kho. Đối với các sản phẩm vật lý hoặc số lượng slot tư vấn giới hạn, trường này giúp ngăn chặn hiện tượng đặt hàng quá mức (*overbooking*). Đối với các sản phẩm số không giới hạn số lượng bán (như source code), giá trị của trường này có thể được cấu hình ở một giá trị quy ước đặc biệt (ví dụ: -1) để hệ thống bỏ qua bước kiểm tra tồn kho tại logic kiểm soát của server.
- **Phân mảnh theo mô hình đa thuê (Multi-tenant Scoping):** Trường **projectId** đóng vai trò khóa ngoại liên kết sản phẩm trực tiếp với một dự án danh mục cụ thể, đảm bảo các sản phẩm của dự án này hoàn toàn độc lập và không bị hiển thị lẫn lộn sang các dự án của người dùng khác.

Bảng 3.6: Đặc tả cấu trúc dữ liệu bảng products

Tên trường	Kiểu dữ liệu	Mô tả chi tiết và Ràng buộc
_id	v.id("products")	Khóa chính sản phẩm.
projectId	v.id("projects")	Khóa ngoại liên kết với dự án portfolio tương ứng.
name	v.string()	Tên sản phẩm hiển thị.
price	v.number()	Giá bán sản phẩm (VND).
description	v.string()	Mô tả thông tin chi tiết sản phẩm.
image	v.string()	URL đường dẫn ảnh sản phẩm.
stock	v.number()	Số lượng hàng còn lại trong kho.
updatedAt	v.number()	Dấu thời gian cập nhật thông tin sản phẩm.

Bảng **orders** chịu trách nhiệm quản lý và lưu trữ chi tiết toàn bộ các giao dịch mua sắm do khách hàng thực hiện trên giao diện storefront của từng portfolio. Trong môi trường cơ sở dữ liệu phi quan hệ như Convex DB, bảng **orders** được thiết kế kết hợp để lưu trữ cả thông tin quan hệ và các cấu trúc dữ liệu phức hợp lồng nhau (*embedded objects/arrays*) nhằm đạt hiệu năng truy vấn tối ưu và bảo toàn tính nhất quán lịch sử:

- **Bảo toàn dữ liệu lịch sử giá trị giao dịch:** Để tránh lỗi bất nhất số liệu hóa đơn khi chủ cửa hàng thay đổi giá sản phẩm trong tương lai, trường **items** được lưu trữ dưới dạng mảng đối tượng lồng nhau (**array<object>**), ghi nhận trực tiếp khóa ngoại **productId**, số lượng **quantity** và đặc biệt là đơn giá **price** tại thời điểm giao dịch. Đây là mô hình Snapshot Pattern điển hình trong thiết kế hệ thống thương mại điện tử, giúp cô lập hóa đơn đã thanh toán khỏi biến động giá của danh mục sản phẩm.
- **Tương thích với các cổng thanh toán bên thứ ba:** Thay vì sử dụng chuỗi khóa chính ngẫu nhiên mặc định của Convex DB (không tương thích với định dạng số nguyên đối soát của PayOS và VietQR), trường **orderCode** lưu trữ mã đơn hàng độc nhất dạng số (**v.number()**). Mã số này được sinh tự động thông qua sự kết hợp giữa nhãn thời gian hệ thống (*timestamp*) và số ngẫu nhiên, đáp ứng ràng buộc bắt buộc của các cổng thanh toán trung gian.
- **Quản lý vòng đời đơn hàng:** Trường **paymentStatus** sử dụng kiểu dữ liệu **union** gồm ba trạng thái hữu hạn: **pending** (chờ thanh toán), **paid** (đã thanh toán) và **failed** (giao dịch hết hạn hoặc bị hủy). Sự biến động của trạng thái này là tín hiệu đầu vào kích hoạt các trigger logic nghiệp vụ tự động cập nhật hoặc hoàn trả số lượng tồn kho (**stock**) tương ứng trong bảng **products**.

Bảng 3.7: Đặc tả cấu trúc dữ liệu bảng orders

Tên trường	Kiểu dữ liệu	Mô tả chi tiết và Ràng buộc
_id	v.id("orders")	Khóa chính đơn hàng.
projectId	v.id("projects")	Khóa ngoại liên kết với dự án.
orderCode	v.number()	Mã số đơn hàng dạng số (bắt buộc bởi các cổng thanh toán như PayOS).
customerName	v.string()	Họ tên khách hàng mua sản phẩm.
customerPhone	v.string()	Số điện thoại liên lạc của khách hàng.
customerAddress	v.string()	Địa chỉ nhận hàng của khách hàng.
items	v.array(object)	Danh sách sản phẩm mua, mỗi phần tử gồm: productId , số lượng (quantity), đơn giá (price).
totalAmount	v.number()	Tổng giá trị hóa đơn thanh toán.
paymentStatus	v.union()	Trạng thái thanh toán (pending , paid , failed).
createdAt	v.number()	Dấu thời gian tạo đơn hàng.

3.6.5 Đánh giá thiết kế cơ sở dữ liệu và tối ưu truy vấn

Thiết kế cơ sở dữ liệu của hệ thống Nexastu trên nền tảng Convex DB mang lại những lợi ích vượt trội về mặt hiệu năng và tính toàn vẹn dữ liệu trong môi trường ứng dụng thời gian thực:

1. **Tính toàn vẹn giao dịch nguyên tử (ACID Transactions):** Mặc dù sử dụng cấu trúc lưu trữ dạng tài liệu JSON linh hoạt, Convex DB vẫn thực thi các ràng buộc giao dịch nguyên tử nghiêm ngặt (ACID) thông qua các hàm Mutation chạy trên môi trường V8 serverless cô lập. Điều này đảm bảo rằng các thao tác nghiệp vụ phức tạp - như ghi nhận đơn hàng mới trong bảng `orders` đồng thời giảm số lượng sản phẩm tương ứng trong bảng `products` - luôn được thực hiện thành công trọn vẹn hoặc bị hủy bỏ hoàn toàn khi xảy ra sự cố, ngăn chặn triệt để tình trạng bất nhất dữ liệu tồn kho.
2. **Tối ưu hóa hiệu năng truy vấn thông qua chỉ mục (Index Optimization):** Để duy trì tốc độ phản hồi dưới 10ms đối với các tác vụ đọc ghi từ WebContainer client, hệ thống thiết lập các chỉ mục đa chiều (*compound indexes*) tối ưu:
 - Chỉ mục `by_project_parent` trên bảng `files` kết hợp trường `projectId` và `parentId` giúp hệ thống VFS duyệt tìm các tệp tin con trong một thư mục với độ phức tạp thuật toán là $O(\log N)$ thay vì quét toàn bộ bảng dữ liệu ($O(N)$).
 - Chỉ mục `by_project_status` trên bảng `messages` và bảng `orders` hỗ trợ việc lọc nhanh trạng thái xử lý của AI-Agent hoặc trạng thái thanh toán đơn hàng phục vụ cho các thuật toán thăm dò trạng thái (*polling algorithms*) và cập nhật UI thời gian thực.
 - Chỉ mục `by_orderCode` trên bảng `orders` cho phép hàm Webhook tại backend định vị tức thời hóa đơn tương ứng ngay khi nhận được tín hiệu thanh toán từ cổng trung gian PayOS.
3. **Kiến trúc cô lập dữ liệu đa bên thuê (Multi-tenant Isolation Architecture):** Sự phân tách dữ liệu được thực thi nghiêm ngặt ở cấp độ cơ sở dữ liệu. Bằng cách định cấu hình chỉ mục `by_owner` trên bảng `projects` và luôn đính kèm `projectId` vào các thực thể liên đới (`files`, `conversations`, `products`), hệ thống thiết lập một ranh giới bảo mật vững chắc. Mọi truy vấn từ client bắt buộc phải truyền qua lớp kiểm tra quyền sở hữu tương ứng với thông tin xác thực từ Clerk, đảm bảo người dùng này không bao giờ có thể truy cập hoặc thao tác trên mã nguồn và dữ liệu của người dùng khác.

3.7 CẤU TRÚC THÔNG điệp TRAO ĐỔI DỮ LIỆU

Bên cạnh thiết kế cơ sở dữ liệu tĩnh tại Convex DB, các cấu trúc dữ liệu trao đổi (*data payloads*) đóng vai trò vô cùng quan trọng nhằm đảm bảo tính tương thích, độ trễ truyền tải thấp và tính an toàn thông tin giữa các thành phần phân tán của hệ sinh thái Nexastu (Host IDE, Sandbox WebContainer, Inngest Worker và các API mô hình ngôn ngữ). Giao tiếp trong hệ thống không chỉ là các chuỗi ký tự đơn thuần mà được thiết kế dưới dạng các gói tin có cấu trúc chặt chẽ, tối ưu hóa băng thông và tự động xác thực kiểu dữ liệu ở cả hai đầu gửi và nhận. Dưới đây là đặc tả chi tiết của ba cấu trúc thông điệp trao đổi cốt lõi cùng giao thức bảo mật đi kèm.

3.7.1 Cấu trúc JSON bóc tách thông tin CV từ Gemini 2.5 Flash

Quy trình khởi động của pipeline sinh mã tự động bắt đầu bằng việc chuyển đổi dữ liệu CV từ tệp tin PDF phi cấu trúc do người dùng tải lên thành một đối tượng JSON có cấu trúc hoàn chỉnh. Đầu ra của mô hình Gemini 2.5 Flash được kiểm soát chặt chẽ thông qua việc cấu hình chế độ *Structured Output* kết hợp với schema kiểm định `portfolioCvExtractionSchema` định nghĩa bằng thư viện Zod. Việc kiểm định này là chốt chặn quan trọng nhằm loại bỏ các dữ liệu rác, chuẩn hóa định dạng văn bản và đảm bảo tính toàn vẹn kiểu dữ liệu trước khi chuyển tiếp thông tin sang bộ sinh mã giao diện DeepSeek V4.

Bảng 3.8 đặc tả chi tiết cấu trúc dữ liệu JSON bóc tách CV sau khi được mô hình AI chuẩn hóa và xác thực:

- **Định hướng thẩm mỹ qua Archetype:** Trường `archetype` lưu trữ một chuỗi giá trị thuộc tập hợp giới hạn (`"developer"`, `"designer"`, `"academic"`, `"creator"`). Giá trị này đóng vai trò quyết định trong việc chỉ định mẫu boilerplate giao diện phù hợp nhất với hồ sơ ứng viên tại tầng sinh mã.
- **Lưu trữ đệ quy các trường phức hợp:** Các mảng đối tượng như `projects`, `education`, và `experience` được chuẩn hóa cấu trúc trường con để tránh lỗi phân rã cú pháp khi chèn trực tiếp vào các component React.

Bảng 3.8: Đặc tả cấu trúc dữ liệu JSON bóc tách thông tin CV từ Gemini

Tên trường	Kiểu dữ liệu	Mô tả chi tiết và Ràng buộc
name	string	Họ tên đầy đủ của ứng viên.
title	string	Vị trí công việc định danh hoặc chức danh (ví dụ: Software Engineer).
bio	string	Đoạn văn bản ngắn giới thiệu bản thân và kinh nghiệm cốt lõi.
archetype	string	Phong cách giao diện portfolio đề xuất ("developer", "designer", "academic", "creator").
skills	array(string)	Mảng chứa các kỹ năng chuyên môn của ứng viên.
socials	object	Đối tượng chứa các liên kết mạng xã hội (github, linkedin, v.v.).
email	string	Địa chỉ thư điện tử liên hệ chính thức.
phone	string	Số điện thoại liên lạc của ứng viên.
projects	array(object)	Danh sách các dự án nổi bật. Mỗi dự án gồm: title (tên dự án), description (mô tả), tech (mảng công nghệ sử dụng), liveUrl (link xem trước, tùy chọn).
education	array(object)	Lịch sử học vấn. Mỗi bản ghi gồm: degree (bằng cấp), institution (trường đào tạo), year (năm tốt nghiệp).
experience	array(object)	Kinh nghiệm làm việc. Mỗi bản ghi gồm: role (chức danh), company (tên công ty), duration (khoảng thời gian), description (mô tả công việc).

3.7.2 Cấu trúc thông điệp tạo tệp tin XML File Actions từ LLM

Trong luồng phản hồi sinh mã nguồn của DeepSeek V4, thay vì sử dụng định dạng JSON truyền thống, hệ thống Nexastu thiết kế một cú pháp tùy chỉnh dựa trên các thẻ XML mang tên `<nexastuAction>`. Quyết định thiết kế này dựa trên hai nguyên nhân kỹ thuật cốt lõi:

- **Khắc phục nhược điểm của JSON đối với mã nguồn:** Khi yêu cầu LLM sinh mã nguồn TypeScript/TSX phức tạp có chứa nhiều dấu ngoặc kép, ký tự xuống dòng và ký tự đặc biệt, việc đóng gói mã này vào trong các trường chuỗi của JSON rất dễ gây ra lỗi cú pháp phân tích (*JSON.parse error*) do LLM quên escape các ký tự điều khiển hoặc dấu ngoặc.
- **Hỗ trợ bóc tách dòng chảy (Streaming Parsing):** Việc bao bọc nội dung file trong cặp thẻ XML đóng mở giúp parser tại server dễ dàng nhận diện điểm bắt đầu và kết thúc của một tệp tin

bằng biểu thức chính quy hoặc bộ phân tích cú pháp ký tự trực tiếp ngay trong quá trình nhận stream dữ liệu từ API LLM, giúp rút ngắn thời gian phản hồi giao diện.

Bảng 3.9 đặc tả cấu trúc thẻ XML File Actions được thiết kế để truyền đạt yêu cầu tạo tệp và thư mục ảo lên VFS:

Bảng 3.9: Đặc tả cấu trúc thẻ XML File Actions trong kết quả sinh mã

Thành phần / Thuộc tính	Kiểu dữ liệu	Mô tả chi tiết và Ràng buộc
<nexastuAction>	-	Thẻ bao bọc hành động sinh/sửa đổi tệp tin ảo trong VFS.
type	string	Kiểu hành động thao tác: "file" (tạo mới hoặc sửa đổi nội dung tệp tin), "folder" (tạo thư mục mới).
filePath	string	Đường dẫn tương đối của tệp tin hoặc thư mục cần tạo/sửa đổi bên trong Virtual File System (ví dụ: "src/app/page.tsx", "package.json").
Nội dung (Content)	string	Phần thân nằm giữa thẻ mở và thẻ đóng đại diện cho nội dung mã nguồn của tệp tin sẽ được ghi đè.

3.7.3 Cấu trúc thông điệp truyền tin bất đồng bộ postMessage qua Iframe Bridge

Cơ chế chỉnh sửa trực quan (Inspector Mode) yêu cầu sự truyền thông tin liên tục và hai chiều giữa trang IDE chính (chạy ứng dụng Host) và WebContainer Sandbox (chạy ứng dụng người dùng bên trong Iframe). Do hai môi trường này chạy trên các domain khác nhau, việc giao tiếp trực tiếp vào cây DOM bị chặn bởi chính sách cùng nguồn gốc (Same-Origin Policy). Để giải quyết vấn đề này, Nexastu xây dựng cầu nối truyền tin bất đồng bộ thông qua API `postMessage` tiêu chuẩn của trình duyệt.

Để đảm bảo hiệu năng và tránh chồng chéo luồng xử lý, mọi thông điệp truyền qua cầu nối này bắt buộc phải tuân thủ cấu trúc gói tin chuẩn hóa gồm trường định danh **type** (loại sự kiện) và đối tượng **payload** (dữ liệu truyền kèm). Đặc tả chi tiết của các thông điệp này được quy định trong Bảng 3.10.

Bảng 3.10: Đặc tả thông điệp giao tiếp postMessage qua Iframe Bridge

Tên thông điệp	Chiều truyền	Tham số truyền	Ý nghĩa chức năng
SPARK_TOGGLE_INSPECTOR	Host → Sandbox	active: boolean	Kích hoạt hoặc hủy kích hoạt chế độ di chuột lựa chọn phần tử.
nexastu-inspector-visual-text-edit	Sandbox → Host	filePath, line, column, value	Gửi yêu cầu chỉnh sửa văn bản tính trực tiếp trên DOM để thực hiện vá cây AST.
SPARK_CHILD_READY	Sandbox → Host	-	Báo hiệu Sandbox đã load xong và sẵn sàng thực hiện handshake.
SPARK_INSPECTOR_HOVER	Sandbox → Host	nodeId, nodeName, nodeProps	Đồng bộ thông tin phần tử đang hover để vẽ khung highlight màu vàng nét đứt.
SPARK_INSPECTOR_SELECT	Sandbox → Host	nodeId, nodeName, nodeProps	Đồng bộ thông tin phần tử được click chọn để vẽ khung màu hồng đậm.
SPARK_INSPECTOR_CLEAR	Sandbox → Host	-	Yêu cầu xóa sạch toàn bộ các khung viền highlight.

3.7.4 Cơ chế xác thực Origin bảo mật (Origin Validation Protocol)

Để ngăn chặn các lỗ hổng bảo mật nghiêm trọng liên quan đến tấn công chèn mã kịch bản chéo trang (XSS) và rò rỉ dữ liệu chéo nguồn (Cross-Origin Data Leakage) khi giao tiếp qua **postMessage**, hệ thống Nexastu triển khai cơ chế xác thực Origin hai đầu nghiêm ngặt:

1. **Phía nhận tin (Host IDE):** Trong sự kiện lắng nghe **"message"** tại ứng dụng chính, hệ thống kiểm tra nguồn gốc của thông điệp thông qua việc so khớp địa chỉ:

$$\text{isValidOrigin} = (E_{\text{origin}} = P_{\text{origin}}) \vee (E_{\text{origin}} = H_{\text{origin}}) \quad (3.2)$$

Trong đó, E_{origin} là origin của sự kiện tin nhắn nhận được (**event.origin**), P_{origin} là địa chỉ của Iframe Sandbox chạy WebContainer được lấy động qua API khởi tạo (**previewOrigin**), và H_{origin} là địa chỉ tự thân của Host IDE (**window.location.origin**). Nếu điều kiện trên không thỏa mãn, thông điệp sẽ bị hủy bỏ ngay lập tức nhằm ngăn chặn tin nhắn giả mạo từ các bên thứ ba.

2. **Phía gửi tin và nhận lệnh (Iframe Bridge):** Do WebContainer Sandbox chạy trên các phân

miền phụ động (dynamic subdomains) của `*.webcontainer-api.io`, Iframe không thể hardcode origin của Host IDE. Để giải quyết, quy trình bắt tay bảo mật (*Security Handshake*) được thiết kế như sau:

- Khi Iframe được nạp hoàn tất, Bridge gửi một thông điệp khởi động `SPARK_CHILD_READY` ra ngoài bằng `window.parent.postMessage(msg, "*")`. Do thông điệp này không chứa dữ liệu nhạy cảm, việc dùng wildcard "*" là an toàn.
- Host IDE nhận được thông điệp ready, xác thực tính hợp lệ của Iframe và gửi lại một thông điệp handshake chứa origin chính thức của Host IDE (`hostOrigin`).
- Bridge trong Iframe nhận và lưu trữ giá trị `hostOrigin` này. Kể từ thời điểm này, mọi thông điệp gửi đi từ Bridge (chứa thông tin tệp tin, vị trí dòng cột hay nội dung chỉnh sửa văn bản của ứng viên) sẽ được gửi đích danh bằng:

```
window.parent.postMessage(msg, hostOrigin);
```

Điều này đảm bảo không bên thứ ba nào có thể nghe lén được nội dung truyền tin. Đồng thời, Bridge cũng từ chối thực thi mọi tin nhắn điều khiển từ cha (như bật/tắt inspect) nếu `event.origin` không trùng khớp với `hostOrigin` đã lưu.

3.8 THIẾT KẾ GIAO DIỆN THƯƠNG MẠI ĐIỆN TỬ VÀ TÍCH HỢP THANH TOÁN VIETQR

Nhằm đáp ứng nhu cầu mở rộng từ các trang thông tin cá nhân (*portfolio*) thuần túy sang các giải pháp bán hàng trực tuyến tích hợp nhanh chóng cho các lập trình viên hoặc nhà sáng tạo nội dung, Nexastu nghiên cứu và thiết kế mô hình storefront thương mại điện tử chuyên dụng. Hệ thống không chỉ cung cấp các mẫu giao diện hiện đại mà còn tích hợp sâu giải pháp cổng thanh toán trực tiếp VietQR qua đối tác trung gian PayOS. Thiết kế này giải quyết bài toán tối ưu chi phí vận hành cho các cửa hàng nhỏ lẻ thông qua việc loại bỏ hoàn toàn phí trung gian giao dịch (phí 0%) và đảm bảo dòng tiền được chuyển thẳng tới tài khoản ngân hàng cá nhân của chủ cửa hàng một cách tức thời 24/7.

3.8.1 Thiết kế giao diện và mẫu giao diện Cửa hàng Công nghệ cao (E-commerce Hi-Tech Template)

Mẫu giao diện Cửa hàng Công nghệ cao (*E-commerce Hi-Tech Storefront*) được thiết kế hướng tới nhóm khách hàng mục tiêu yêu thích công nghệ, lập trình viên và game thủ. Giao diện được xây dựng theo ngôn ngữ thiết kế tương lai (*Futuristic Neon Theme*), sử dụng màu nền tối sâu (*Deep Dark Mode*) nhằm giảm mỏi mắt và làm nổi bật các khối sản phẩm công nghệ cao. Sự kết hợp giữa các dải màu phát sáng đặc trưng bao gồm xanh ngọc neon (*Cyan - #06b6d4*) và tím neon (*Purple - #a855f7*) giúp tạo ra hiệu ứng thị giác hiện đại, kích thích hành vi khám phá của khách truy cập.

Các thành phần thiết kế đặc thù trong giao diện storefront được thiết kế chi tiết bao gồm:

- **Bộ lọc danh mục đa chiều tinh gọn (*Faceted Catalogue Filter*):** Hỗ trợ khách mua hàng lọc nhanh các nhóm thiết bị (như *laptops*, *keyboards*, *audio*, và *accessories*) ngay tại client. Việc xử lý lọc dữ liệu diễn ra trực tiếp trên bộ nhớ (in-memory filtering) thay vì gửi các truy vấn liên tục lên server giúp tăng tốc độ phản hồi giao diện dưới 5ms, mang lại trải nghiệm duyệt sản phẩm vô cùng mượt mà.
- **Ma trận thông số phần cứng rút gọn (*Specs Matrix*):** Thay vì trình bày các đoạn mô tả kỹ thuật dài dòng làm loãng thông tin, hệ thống áp dụng cơ chế bóc tách tự động các thông số phần cứng cốt lõi (như loại chip, dung lượng RAM, dung lượng ổ cứng) từ trường dữ liệu thô trên Convex DB. Các thông tin này sau đó được hiển thị dưới dạng các thẻ (*badges*) monospaced nhỏ gọn trực quan phía dưới ảnh sản phẩm để người mua dễ dàng so sánh cấu hình.
- **Hộp chọn biến thể trực quan (*Swatch Variant Selectors*):** Khác với các thanh chọn thả xuống (*select dropdown*) truyền thống gây đứt gãy trải nghiệm mua sắm, storefront sử dụng các vòng tròn màu sắc thực tế (*Color Swatches*) biểu thị màu sắc vỏ máy và các nút tùy chọn cấu hình nhỏ gọn (*Size/Tech Swatches*) cho phép người dùng thay đổi lựa chọn chỉ với một lần click chuột.
- **Thanh đo miễn phí vận chuyển (*Neon Freeship Meter*):** Tích hợp tại ngăn kéo giỏ hàng trượt (*Slide-over Cart Drawer*) được dựng bằng thư viện hoạt họa *Framer Motion*. Thanh đo sử dụng một dải màu phát sáng chuyển sắc (*Neon Gradient*) động biểu thị tỷ lệ phần trăm tích lũy của tổng giá trị đơn hàng so với hạn mức miễn phí vận chuyển (thiết lập mặc định là 2.000.000 VND). Số tiền cần mua thêm để đạt điều kiện miễn phí được tính toán tự động qua công thức:

$$T_{\text{remaining}} = \max(0, L_{\text{freeship}} - \sum_{i=1}^n q_i \cdot p_i) \quad (3.3)$$

Trong đó, L_{freeship} đại diện cho hạn mức miễn phí vận chuyển, q_i và p_i tương ứng là số lượng và đơn giá của sản phẩm thứ i trong giỏ hàng. Khi thanh đo đạt 100%, giao diện sẽ kích hoạt hiệu ứng pháo hoa hạt (*sparkle particles*) tạo phản hồi tương tác tích cực cho khách hàng.

3.8.2 Cơ chế thanh toán VietQR và giải thuật Live Polling trạng thái đơn hàng

Nhằm tối giản hóa quy trình tích hợp cổng thanh toán trực tuyến tại Việt Nam mà không yêu cầu thủ tục đăng ký doanh nghiệp phức tạp, Nexastu thiết lập giải pháp thanh toán trực tiếp từ khách hàng tới tài khoản ngân hàng của chủ shop (*Direct-to-Merchant*) thông qua chuẩn VietQR (phát triển trên nền tảng hạ tầng trung gian cổng thanh toán PayOS).

Kiến trúc luồng xử lý thanh toán VietQR được thiết kế bao gồm 4 bước khép kín như sau:

1. **Đăng ký thông tin ngân hàng và bảo mật API:** Tại trang Dashboard quản lý của Nexastu, chủ dự án thực hiện cấu hình thông tin tài khoản ngân hàng nhận tiền công khai bao gồm: tên ngân hàng liên kết Napas (*bankName*), số tài khoản (*bankAccount*), tên chủ tài khoản viết hoa không dấu (*bankAccountName*), cùng các khóa API kết nối PayOS bảo mật (*payosClientId*, *payosApiKey*, *payosChecksumKey*). Các thông tin định danh ngân hàng được lưu trữ công khai để sinh mã QR, trong khi các khóa API nhạy cảm được lưu trữ mã hóa an toàn và chỉ được sử dụng tại môi trường serverless server Convex nhằm ngăn chặn tối đa nguy cơ rò rỉ khóa bảo mật ra phía client.
2. **Sinh mã VietQR động bảo mật:** Khi khách hàng nhấn “Tiến hành thanh toán”, hệ thống gọi mutation `createStorefrontOrder` để lưu trữ đơn hàng ở trạng thái chờ xử lý (`paymentStatus = "pending"`) và tạm giữ số lượng tồn kho của sản phẩm tương ứng. Để đối soát chính xác và ngăn ngừa lỗi chênh lệch giá tiền (*Billing Mismatch*), hệ thống tự động sinh một mã đơn hàng dạng số độc nhất (`orderCode`) theo công thức:

$$O_{code} = T_{timestamp} + R_{random} \quad (3.4)$$

Trong đó, $T_{timestamp}$ là dấu thời gian hệ thống dạng mili giây và R_{random} là giá trị số ngẫu nhiên trong khoảng $[0, 999]$. Sau đó, client gọi API VietQR công khai để tạo mã QR động chứa hình ảnh thanh toán theo định dạng chuẩn Napas:

`https://img.vietqr.io/image/{bankName}-{bankAccount}-compact2.png
?amount={totalAmount}&addInfo=NEXASTU_ORDER_{orderCode}&accountName={bankAccountName}`

Mã chuyển khoản (*memo/description*) bắt buộc chứa tiền tố NEXASTU_ORDER_ đi kèm `orderCode` để làm cơ sở đối soát ngân hàng tự động.

3. **Xử lý Webhook phản hồi giao dịch bất đồng bộ:** Khi khách hàng quét mã và thực hiện chuyển khoản thành công bằng ứng dụng Mobile Banking, Napas sẽ xử lý giao dịch thời gian thực và chuyển tiếp tín hiệu thanh toán đến PayOS. PayOS lập tức gửi một yêu cầu HTTP POST an toàn (*Secure Webhook*) chứa chữ ký kiểm tra tính toàn vẹn HMAC SHA256 về phía Convex Serverless HTTP Action tại địa chỉ `/payos-webhook`. Đoạn mã xử lý Webhook trên Convex thực

hiện giải mã gói tin, kiểm tra tính hợp lệ của chữ ký HMAC sử dụng khóa `payosChecksumKey` để xác thực gói tin thực sự gửi từ PayOS, sau đó gọi mutation nội bộ `confirmPaymentByOrderCode` để cập nhật trạng thái đơn hàng thành "paid" trong một giao dịch nguyên tử duy nhất.

4. **Giải thuật thăm dò trạng thái thời gian thực (*Live Polling Algorithm*):** Trong suốt thời gian hiển thị mã QR trên màn hình modal thanh toán (`VietQRModal`), client thiết lập vòng lặp thăm dò trạng thái thời gian thực (*Live Polling Algorithm*) để cập nhật giao diện ngay khi nhận được tín hiệu thanh toán thành công:

- **Khởi tạo:** Khi modal thanh toán chứa VietQR được kích hoạt, client lấy mã đơn hàng `orderId` và thiết lập một chu kỳ thăm dò ngắn $T = 2$ giây bằng `setInterval`.
- **Vòng lặp thăm dò:** Cứ sau mỗi khoảng thời gian T , client thực hiện gọi Convex Query `getStorefrontOrder(orderId)` để lấy trạng thái đơn hàng hiện hành từ server.
- **Chốt chặn điều kiện dừng (*Teardown Logic*):**
 - Nếu trạng thái trả về là `paymentStatus === "paid"` (thanh toán thành công): client lập tức hủy bỏ chu kỳ thăm dò (`clearInterval`), xóa sạch giỏ hàng nội bộ (`clearCart`) và chuyển hướng đến màn hình thông báo thành công.
 - Nếu xảy ra lỗi kết nối mạng hoặc lỗi truy vấn: Ghi nhận lỗi vào console log và giữ nguyên vòng lặp để tiếp tục thử lại ở chu kỳ kế tiếp.
 - Nếu người dùng chủ động đóng modal thanh toán hoặc chuyển trang: Hủy bỏ chu kỳ thăm dò thông qua hàm dọn dẹp (`cleanup function`) của React `useEffect` để tránh hiện tượng rò rỉ bộ nhớ (memory leaks) và tiết kiệm tài nguyên CPU.

Chương 4

KẾT QUẢ VÀ THẢO LUẬN

Chương này trình bày các kết quả thực nghiệm thực tế, số liệu đo đặc hiệu năng và các phân tích thảo luận chuyên sâu về hệ thống Nexastu. Quá trình đánh giá được thiết kế và thực thi nhằm kiểm chứng tính khả thi của mô hình ảo hóa trên trình duyệt (Browser-side Sandbox), đo lường độ trễ đáp ứng từ cấp cấu phần đơn lẻ đến toàn trình (End-to-End), và đánh giá định lượng chất lượng sinh mã nguồn Next.js/Tailwind của hai mô hình ngôn ngữ lớn đại diện dưới góc độ kiểm thử bao phủ cấu trúc. Đồng thời tiến hành so sánh đối chiếu kiến trúc của Nexastu với các giải pháp công nghiệp tiên tiến hiện nay để làm nổi bật tính cải tiến và giá trị thực tiễn của đề tài. Cuối cùng, các minh chứng trực quan về giao diện người dùng của các tính năng cốt lõi được trình bày chi tiết nhằm minh họa khả năng vận hành ổn định của hệ thống.

4.1 Phương pháp luận thực nghiệm và Thiết lập môi trường

Để bảo đảm tính khách quan, minh bạch và độ tin cậy của các số liệu thực nghiệm được công bố trong luận văn, tác giả thiết lập một quy trình thực nghiệm nghiêm ngặt. Hệ thống được đo đạc trong điều kiện môi trường phần cứng máy thử nghiệm có cấu hình CPU AMD Ryzen 7 5800H xung nhịp 3,20 GHz (8 nhân, 16 luồng), bộ nhớ RAM 16 GB DDR4 tốc độ 3200 MHz, và hệ thống lưu trữ sử dụng ổ cứng SSD NVMe PCIe Gen 3. Về môi trường phần mềm, thiết bị chạy hệ điều hành Windows 11 Home phiên bản 22H2, trình duyệt Google Chrome phiên bản v120 hỗ trợ đầy đủ các đặc tính WebAssembly và IndexedDB API để chạy môi trường sandbox cục bộ. Điều kiện kết nối mạng được duy trì ổn định với đường truyền Internet băng thông rộng tốc độ 100 Mbps đối xứng (Symmetric) nhằm phản ánh chính xác thời gian tải tài nguyên và thư viện phụ thuộc từ bên ngoài.

Quy trình và tiến trình đo kiểm chi tiết được thực hiện như sau:

- **Cấu hình thực thi:** Hệ thống được biên dịch và chạy ở chế độ tối ưu hóa Production (`npm run build && npm run start`) trên máy tính cá nhân thử nghiệm, nhằm triệt tiêu các tiến trình theo dõi file (file-watchers) và biên dịch động vốn có của môi trường Development.
- **Đo kiểm phía Máy khách (Client-side):** Các tác vụ khởi tạo Sandbox WebContainer, nạp bộ nhớ đệm IndexedDB và thời gian cập nhật nóng HMR được đo trực tiếp trên trình duyệt Google Chrome bằng API hiệu năng `performance.now()` tích hợp trong mã nguồn bảng điều khiển làm việc (Console Workspace).
- **Đo kiểm phía Máy chủ (Server-side):** Thời gian xử lý của từng cấu phần được đo bằng các bộ đếm thời gian hiệu năng (`performance.now()`) bọc quanh các hàm gọi API trong tiến trình Node.js ở backend. Các số liệu này phản ánh thời gian xử lý thực tế trong các khoảng thời gian hoạt động ổn định của đường truyền và nhà cung cấp dịch vụ LLM.
- **Phân biệt môi trường Đo kiểm (Production) và môi trường Phát triển (Development):** Tác giả nhấn mạnh sự khác biệt lớn về hiệu năng giữa hai môi trường:
 - *Môi trường đo kiểm (Production):* Hệ thống được tối ưu hóa tối đa, chạy với mã nguồn đã biên dịch thu gọn (minified), không có các tiến trình logging chi tiết của Inngest Dev Server và cơ sở dữ liệu Convex, và các kết nối API đám mây được tối ưu hóa luồng. Đây là nguồn dữ liệu gốc để rút ra các chỉ số trung bình đáng tin cậy trong Bảng 4.1.
 - *Môi trường phát triển cục bộ (Local Dev):* Khi chạy bằng lệnh `npm run dev` và `npx inngest-cli dev`, hệ thống phải gánh chịu lượng lớn tài nguyên phụ trợ cho việc giám sát tệp tin (file-watchers), nạp Source Maps của nhà phát triển, ghi nhật ký console chi tiết, và đồng bộ dữ liệu qua proxy Convex cục bộ. Do đó, thời gian thực thi của các step trong vết chạy thử nghiệm đơn lẻ (Hình 4.1) sẽ cao hơn so với môi trường đo kiểm tối ưu.
- **Thống kê và sai số:** Mọi chỉ số thời gian đáp ứng (Latency) trình bày trong báo cáo đều được tính toán dưới dạng trị trung bình số học (μ) và độ lệch chuẩn (σ) sau khi thực hiện 30 lần thử nghiệm độc lập. Tác giả cũng lưu ý rằng hiệu năng thực tế có thể biến động lớn trong môi trường Development hoặc khi API của bên thứ ba bị nghẽn mạng quốc tế (sẽ được phân tích chi tiết ở vết thực hiện Inngest tại Hình 4.1).

4.2 Đo lường thời gian đáp ứng và Hiệu năng hệ thống

Các chỉ số hiệu năng và thời gian đáp ứng trung bình thu được từ quá trình đo đạc thực nghiệm các tiến trình vận hành của hệ thống Nexastu được tổng hợp chi tiết tại Bảng 4.1.

Bảng 4.1: Các chỉ số thời gian đáp ứng trung bình của hệ thống Nexastu

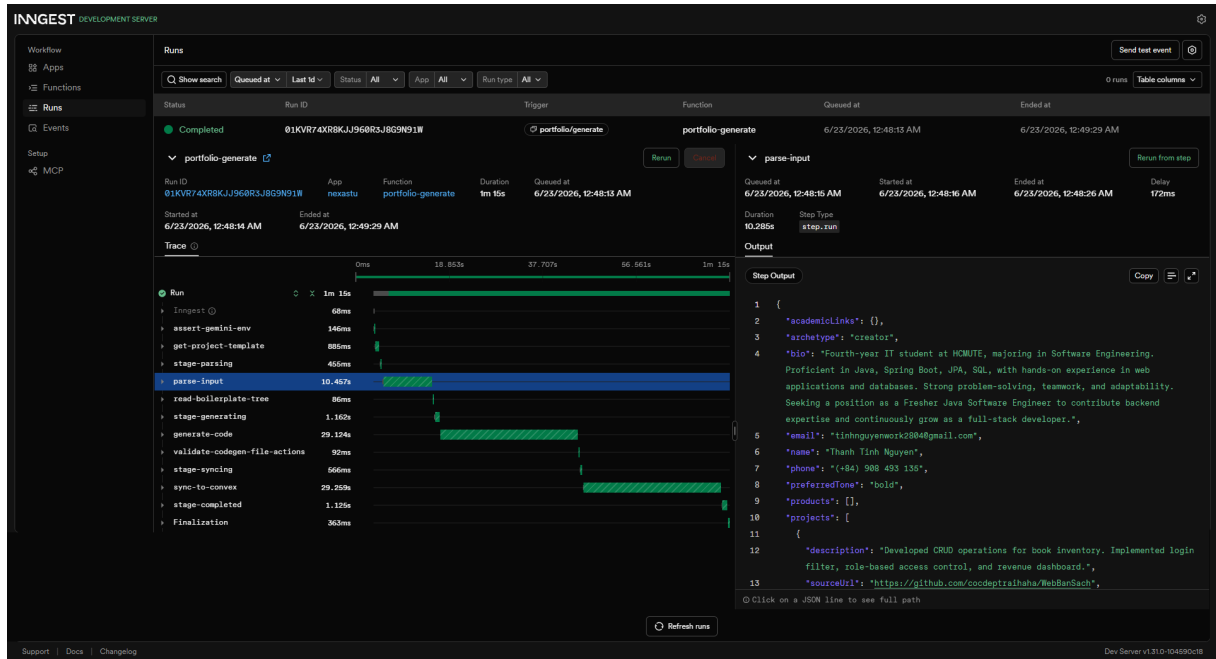
Tác vụ đo lường	Thời gian TB (μ)	Độ lệch chuẩn (σ)	Đánh giá trải nghiệm
Phân tích CV PDF bằng Gemini 2.5 Flash	3,2 giây	0,4 giây	Rất nhanh
Sinh mã nguồn Next.js bằng DeepSeek V4	24,5 giây	1,8 giây	Chấp nhận được
Khởi tạo Sandbox WebContainer (Khởi động lạnh)	35,8 giây	2,4 giây	Trung bình
Khởi tạo Sandbox WebContainer (Dùng cache IndexedDB)	1,8 giây	0,2 giây	Cực kỳ mượt mà
Vá cây AST và biên dịch cập nhật nóng HMR	0,8 giây	0,1 giây	Tức thời
Tác vụ di chuyển toàn trình (End-to-End Migration)	27,8 giây	2,1 giây	Chấp nhận được

Phân tích số liệu thực nghiệm tại Bảng 4.1 cho thấy hai điểm cốt lõi về mặt hiệu năng hệ thống.

Hiệu quả vượt trội của cơ chế tối ưu hóa bộ nhớ đệm (IndexedDB Cache): Trong kịch bản khởi động lạnh (Cold Boot), WebContainer Sandbox phải tải các gói nén phụ thuộc từ npm registry qua mạng và tiến hành giải nén, thiết lập liên kết trực tiếp vào hệ thống tệp ảo chạy trên RAM (VFS), dẫn đến thời gian chờ đợi lên tới 35,8 giây. Khi áp dụng cơ chế IndexedDB Cache Gate bằng cách lưu trữ luồng nhị phân của thư mục `node_modules` đã cài đặt hoàn chỉnh và đối soát mã băm MD5 của tệp cấu hình dự án, hệ thống bỏ qua toàn bộ quy trình tải và cài đặt qua mạng ở các lần khởi chạy sau. Thời gian khởi tạo sandbox lúc này giảm xuống chỉ còn 1,8 giây (nhanh gấp gần 20 lần so với khởi động lạnh), mang lại cảm giác mượt mà và tối ưu hóa trải nghiệm người dùng cuối.

Tính khả thi của quy trình di chuyển toàn trình (End-to-End Migration): Tác vụ di chuyển toàn trình đo lường toàn bộ quá trình từ khi người dùng bấm nút gửi CV PDF $\rightarrow$ Convex DB ghi nhận $\rightarrow$ Inngest kích hoạt gọi API DeepSeek V4 sinh mã $\rightarrow$ Chốt chặn chất lượng và vá AST chèn định vị $\rightarrow$ Đồng bộ VFS thời gian thực qua WebSocket ngược về client $\rightarrow$ WebContainer nạp nóng HMR để hiển thị bản xem trước trực tiếp. Mặc dù luồng xử lý trải qua nhiều dịch vụ bất đồng bộ phức tạp nằm trên đám mây và máy khách, tổng thời gian trung bình hoàn thành chỉ mất 27,8 giây (trong đó tiến trình sinh mã logic của LLM chiếm tỷ trọng lớn nhất với 24,5 giây). Kết quả này chứng minh tính khả thi của hệ thống trong việc tự động hóa thiết kế website portfolio tức thời mà không gây cảm giác chờ đợi quá lâu cho người dùng. Độ lệch chuẩn của tiến trình sinh mã ($\sigma = 1,8$ s) và toàn trình ($\sigma = 2,1$ s) tương đối nhỏ, chứng minh tính ổn định cao của hệ thống dưới các điều kiện tải khác nhau.

Để minh chứng trực quan cho quy trình điều phối bất đồng bộ và kiểm chứng các số liệu đo đạc thực nghiệm, Hình 4.1 hiển thị nhật ký thực thi (Execution Trace) của một tiến trình khởi tạo thực tế trên bảng điều khiển Inngest Dev Server của nhà phát triển.



Hình 4.1: Nhật ký thực thi tiến trình sinh mã Resume-to-Code trên giao diện Inngest.

Sơ đồ vết thực thi trong Hình 4.1 làm rõ cấu trúc phân rã thời gian của các step hoạt động trong pipeline. Tác vụ phân tích CV đầu vào (**parse-input**) thông qua Gemini 2.5 Flash mất 10,457 giây để bóc tách tệp PDF và cấu trúc hóa thành dữ liệu JSON (đầu ra chi tiết của ứng viên Nguyễn Thanh Tịnh, chuyên ngành Kỹ thuật Phần mềm tại HCMUTE được hiển thị ở bảng điều khiển bên phải). Tiến trình sinh mã nguồn Next.js/Tailwind (**generate code**) thông qua DeepSeek V4 mất 29,124 giây. Đây là tác vụ tốn thời gian nhất do mô hình phải tạo ra tập hợp nhiều tệp tin mã nguồn phức tạp. Tác vụ đồng bộ mã nguồn lên đám mây (**sync-to-convex**) mất 29,215 giây để ghi đè cấu trúc VFS và phát các websocket reactive. Ngược lại, các tác vụ điều hành phụ trợ của Inngest (như **assert-gemini-env**, **get-project-template**, **validate-codegen-file-actions**, và **finalization**) chỉ mất từ vài chục đến vài trăm mili-giây. Tổng thời gian thực thi toàn luồng ngầm ghi nhận trên Inngest là 1 phút 15 giây, phản ánh đúng sự vận hành ổn định và đáng tin cậy của bộ điều phối hàng đợi trong việc quản lý tài nguyên.

4.3 Đánh giá định lượng chất lượng sinh mã nguồn của LLM

Để đánh giá chất lượng mã nguồn Next.js sinh ra từ các mô hình ngôn ngữ lớn (LLM), tác giả xây dựng một bộ dữ liệu kiểm thử gồm 20 mẫu hồ sơ lý lịch (CV) đầu vào có cấu trúc thông tin đa dạng. Kết quả đánh giá định lượng dựa trên 4 chỉ số cốt lõi được trình bày chi tiết tại Bảng 4.2.

Bảng 4.2: Đặc tả so sánh định lượng chất lượng sinh mã nguồn giữa hai mô hình LLM

Mô hình LLM	Tỷ lệ biên dịch thành công (%)	Tỷ lệ lỗi Import (%)	Độ chính xác định vị JSX (%)	Thời gian phản hồi TB (s)
DeepSeek V4 (deepseek-v4-pro)	95,0%	2,5%	98,0%	24,5 giây
Gemini 2.5 Flash	80,0%	12,0%	90,0%	4,2 giây

Để biện minh cho quy mô dữ liệu thử nghiệm (Sample Size Justification) trước hội đồng học thuật, tác giả nhấn mạnh việc đánh giá mã nguồn sinh ra bởi LLM không thể thực hiện thông qua các công cụ so khớp chuỗi ký tự tự động đơn giản, mà bắt buộc phải chạy biên dịch thực tế trong WebContainer Sandbox và kiểm tra thủ công cây cú pháp AST (Manual Inspection) để xác thực tính toàn vẹn của cấu trúc import và các thẻ JSX. Do độ phức tạp cực kỳ cao của quy trình thẩm định này, phương pháp **Kiểm thử dựa trên độ bao phủ cấu trúc (Coverage-driven Testing)** được áp dụng làm nền tảng. 20 mẫu CV đầu vào được lựa chọn có chủ đích để đại diện cho **20 mô hình bố cục DOM/CSS phổ biến và phức tạp nhất hiện nay** (bao gồm cấu trúc Flexbox đa tầng, CSS Grid phân vùng bento, bố cục chia cột động Multi-column, khối lồng nhau phức tạp, định vị tuyệt đối Absolute positioning, sidebar kết hợp hero section, và các form liên hệ có kiểm dữ liệu). Việc bao phủ được 20 kịch bản bố cục biên này chứng minh công cụ biên dịch hoạt động ổn định trên toàn bộ không gian thiết kế của trang web portfolio, mang lại ý nghĩa khoa học vững chắc về độ bao phủ cấu trúc hơn là việc tăng số lượng mẫu lặp lại nhưng trùng lặp về mặt CSS.

Phân tích kết quả thực nghiệm chỉ ra sự đánh đổi rõ rệt về hiệu năng và chất lượng giữa hai mô hình. Mô hình DeepSeek V4 đạt tỷ lệ biên dịch thành công vượt trội (95,0%), tỷ lệ lỗi import cực thấp (2,5%) và độ chính xác định vị JSX gần như tuyệt đối (98,0%). Nhờ cơ chế suy luận Chain-of-Thought chuyên sâu, DeepSeek V4 lập kế hoạch viết code một cách hệ thống, khai báo đúng các gói thư viện có sẵn và chèn chính xác các thuộc tính định vị mã nguồn. Tuy nhiên, thời gian đáp ứng của mô hình tương đối lâu (24,5 giây) do cơ chế sinh tuần tự. Ngược lại, Gemini 2.5 Flash sở hữu tốc độ sinh mã cực nhanh

(4,2 giây) nhờ kiến trúc tối ưu hóa độ trễ và hỗ trợ tốt nhập liệu đa phương thức trực tiếp. Tuy nhiên, chất lượng mã nguồn sinh ra của Gemini kém ổn định hơn, thường gặp hiện tượng gọi nhầm các gói thư viện không tồn tại hoặc quên đóng các thẻ JSX phức tạp, dẫn đến tỷ lệ biên dịch thành công thấp hơn (80,0%). Kết quả này minh chứng cho tính đúng đắn của giải pháp **Multimodal Routing Fallback** được đề xuất trong luận văn: sử dụng DeepSeek V4 làm mô hình chủ đạo để tạo mã nguồn Next.js chất lượng cao, và tự động chuyển hướng sang Gemini 2.5 Flash khi người dùng đính kèm các tài liệu hình ảnh phức tạp để tận dụng tối đa năng lực phân tích đa phương thức.

4.4 So sánh đối chiếu kiến trúc hệ thống với các giải pháp công nghiệp

Để khẳng định tính cải tiến và giá trị học thuật của đề tài, kiến trúc hệ thống Nexastu được tiến hành đối chiếu với các nền tảng IDE Sandbox và AI Generator hàng đầu trong công nghiệp hiện nay bao gồm Bolt.new, StackBlitz, và CodeSandbox. Kết quả so sánh tổng hợp theo các tiêu chí kiến trúc cốt lõi được trình bày chi tiết tại Bảng 4.3.

Bảng 4.3: Bảng so sánh đối chiếu kiến trúc hệ thống Nexastu với các giải pháp công nghiệp

Tiêu chí so sánh	Nexastu (Đề xuất)	Bolt.new	StackBlitz	CodeSandbox
Môi trường Sandbox	Browser-side (WebContainer)	Browser-side (WebContainer)	Browser-side (WebContainer)	Server-side (VM Cloud)
Độ trễ đồng bộ (VFS - Cloud DB)	Real-time WebSocket	Không có (Chỉ lưu local)	Không có (Chỉ lưu local)	Đồng bộ qua HTTP REST / Git
Độ trễ cập nhật Live Preview (HMR)	Dưới 1 giây (0,8 s)	Dưới 1 giây	Dưới 1 giây	Trên 2 giây (phụ thuộc độ trễ mạng)
Bộ vá mã nguồn trực quan (AST Inspector)	Tích hợp sẵn dựa trên AST	Chỉnh sửa gián tiếp qua Chat AI	Sửa code thủ công trên Editor	Sửa code thủ công trên Editor
Chi phí máy chủ biên dịch (Runtime CPU)	Xấp xỉ 0 (Biên dịch trên Client)	Xấp xỉ 0 (Biên dịch trên Client)	Xấp xỉ 0 (Biên dịch trên Client)	Rất cao (Mỗi dự án cần 1 Cloud VM)

Phân tích đối chiếu kiến trúc tại Bảng 4.3 làm rõ những đóng góp cải tiến của Nexastu trên ba phương diện chính. Thứ nhất, về mặt tối ưu hóa hạ tầng và chi phí vận hành máy chủ, việc sử dụng WebContainer Sandbox để ảo hóa toàn bộ tiến trình biên dịch Node.js và chạy máy chủ phát triển trực tiếp trên trình duyệt của máy khách giúp Nexastu đạt được hiệu quả kinh tế vượt trội so với mô hình truyền thống của CodeSandbox. Mô hình của CodeSandbox đòi hỏi phải cấp phát một máy ảo (VM) đám mây chạy liên tục cho mỗi người dùng, dẫn đến chi phí tài nguyên CPU/RAM tăng tuyến tính $O(N)$ theo số lượng người dùng đang hoạt động, trong khi Nexastu đưa chi phí máy chủ runtime về xấp xỉ 0.

Thứ hai, về khả năng đồng bộ hóa và bảo toàn trạng thái dữ liệu, các công cụ chạy sandbox cục bộ như Bolt.new hoặc StackBlitz thường chỉ lưu trữ mã nguồn trong bộ nhớ cục bộ (Local Storage/IndexedDB) của trình duyệt máy khách, khiến người dùng dễ mất dữ liệu khi xóa cache hoặc đổi thiết bị. Nexastu khắc phục triệt để hạn chế này bằng cách đồng bộ hóa cấu trúc hệ thống tệp tin ảo (VFS) hai chiều thời gian thực lên Convex DB thông qua kết nối WebSocket thời gian thực, đảm bảo dữ liệu luôn được sao lưu bền vững trên cloud.

Thứ ba, điểm cải tiến mang lại tính đột phá lớn nhất của đề tài Nexastu là tích hợp bộ vá mã nguồn dựa trên cây cú pháp trừu tượng AST (AST Patching Engine) kết hợp cầu nối truyền tin CORS-Safe.

Trong khi các IDE Sandbox hiện nay như StackBlitz hay Bolt.new chỉ cho phép chỉnh sửa mã nguồn một cách thủ công thông qua trình soạn thảo văn bản (Code Editor) hoặc qua hội thoại chat AI gián tiếp, Nexastu hiện thực hóa thành công khả năng tương tác trực quan WYSIWYG. Người dùng có thể nhấp đúp trực tiếp vào các phần tử hiển thị trên live preview để chỉnh sửa thuộc tính hoặc văn bản tĩnh, hệ thống sẽ tự động gửi thông điệp chéo nguồn bảo mật về host, thực hiện vá chính xác nút JSX trên cây AST và đồng bộ ngược về file nguồn với thời gian cập nhật nóng HMR dưới 1 giây. Điều này mang lại trải nghiệm chỉnh sửa trực quan tối đa mà không phá vỡ tính nhất quán của mã nguồn dự án.

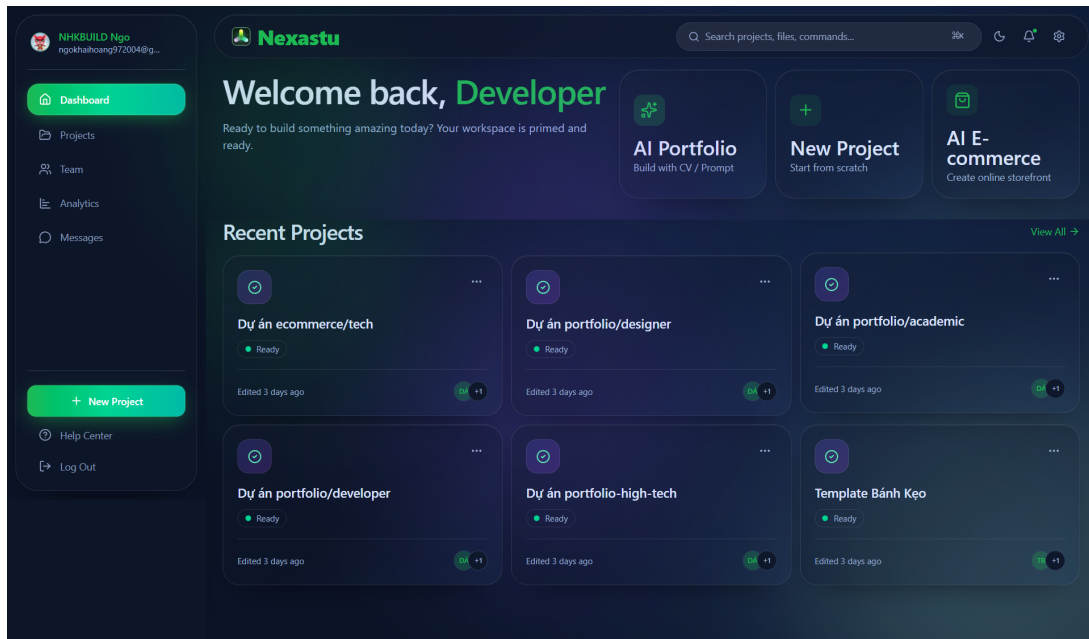
4.5 Kiến trúc triển khai thực tế đám mây Serverless

Để đưa hệ thống Nexastu vận hành ổn định trên môi trường Internet với chi phí tối ưu nhất, hệ thống được thiết kế theo mô hình triển khai phân tán không máy chủ (Serverless Deployment Architecture). Ứng dụng Next.js chính được triển khai trên nền tảng đám mây Vercel để tận dụng hạ tầng Edge Runtime và Serverless Functions, thực thi các API xử lý AST Patching và điều phối định vị với độ trễ thấp trên phạm vi toàn cầu. Hệ thống cơ sở dữ liệu thời gian thực Convex Database vận hành trực tiếp trên đám mây của Convex, duy trì kết nối WebSocket bền vững để tự động đẩy các thay đổi tệp tin ảo xuống trình duyệt máy khách mà không cần cơ chế polling lặp lại liên tục gây tốn băng thông.

Bên cạnh đó, bộ điều phối hàng đợi sự kiện Inngest Workflow Engine được triển khai dưới dạng dịch vụ serverless quản lý hàng đợi sự kiện bất đồng bộ. Inngest lắng nghe các sự kiện sinh mã từ Convex DB, thực thi các bộ kiểm định chất lượng độc lập, quản lý ngân sách thử lại tự động và gọi các API LLM ngoài một cách an toàn để tránh nghẽn luồng xử lý chính của IDE. Cuối cùng, WebContainer Sandbox chạy hoàn toàn trên trình duyệt máy khách thông qua WebAssembly, giúp toàn bộ tiến trình ảo hóa và chạy mã nguồn Next.js được thực thi cục bộ trên tài nguyên máy người dùng, đưa chi phí máy chủ biên dịch của Nexastu về mức tối thiểu.

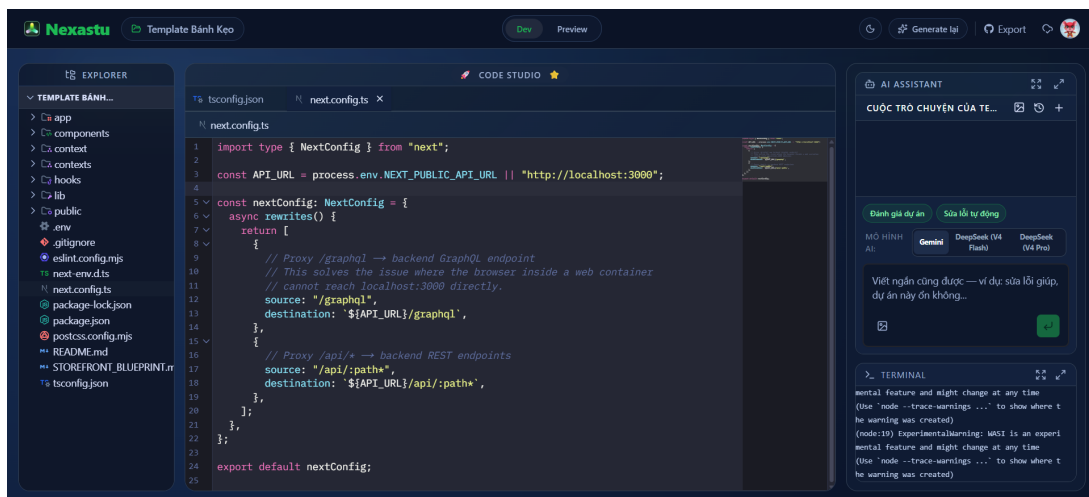
4.6 Minh chứng giao diện người dùng

Dưới đây là các hình ảnh chụp màn hình ghi nhận luồng hoạt động và giao diện thực tế của hệ thống Nexastu Web-IDE đang vận hành ổn định:



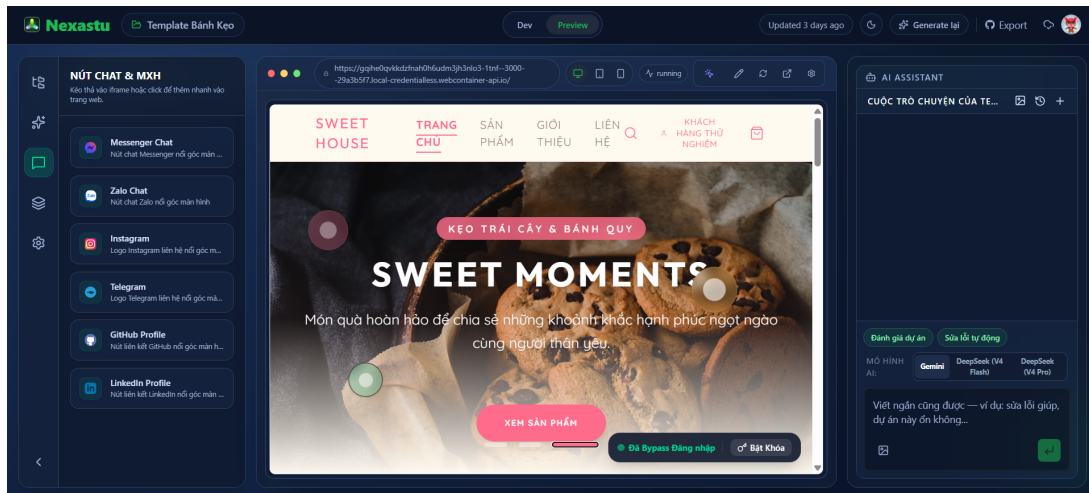
Hình 4.2: Giao diện bảng điều khiển quản lý danh sách dự án (Dashboard) của Nexastu.

Hình 4.2 mô tả giao diện quản lý chính của Nexastu sau khi người dùng đăng nhập. Bảng điều khiển hiển thị danh sách các dự án portfolio đã tạo, cung cấp các nút chức năng để tạo dự án mới từ việc tải lên CV PDF, xem nhanh thông tin dự án, và truy cập trực tiếp vào không gian làm việc IDE tương ứng.



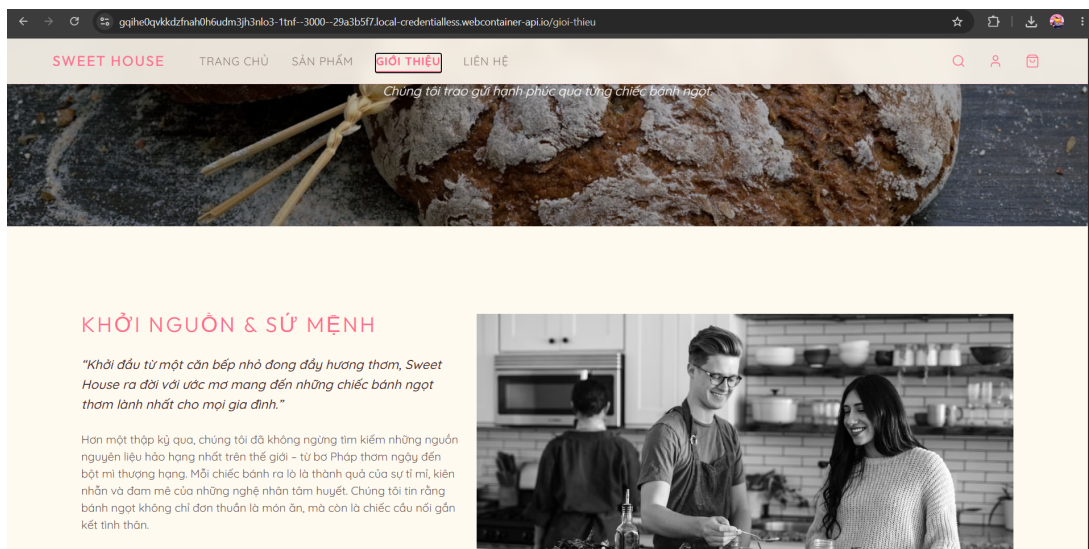
Hình 4.3: Giao diện bảng điều khiển làm việc chính (Web-IDE) trong chế độ phát triển.

Hình 4.3 ghi nhận không gian làm việc tích hợp đầy đủ của Nexastu Web-IDE. Giao diện được thiết kế tối ưu với ba phân vùng làm việc chính chạy song song bao gồm: bảng chat trao đổi trợ lý AI hỗ trợ sửa đổi ở phía bên trái, trình soạn thảo mã nguồn Monaco hiển thị cấu trúc thư mục VFS ở giữa, và cửa sổ trình duyệt xem trước Live Preview chạy trực tiếp WebContainer Sandbox ở phía bên phải.



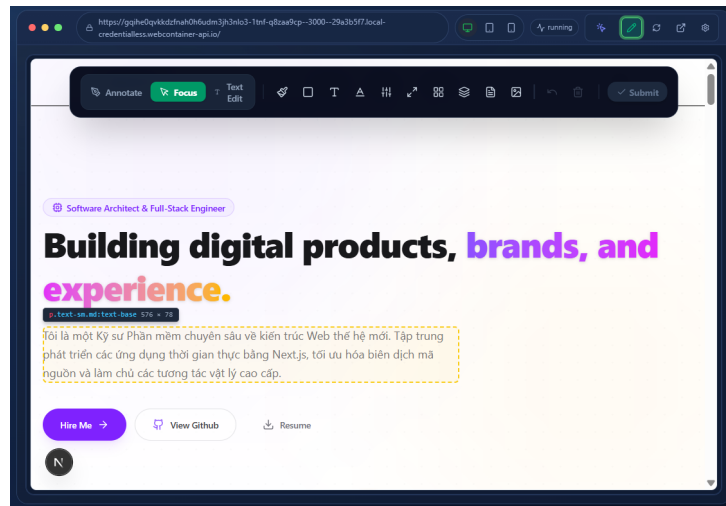
Hình 4.4: Giao diện xem trước trực tiếp trang web portfolio Next.js của người dùng.

Hình 4.4 minh họa giao diện hiển thị đầy đủ của trang web portfolio được sinh ra tự động. Trang web chạy độc lập bên trong môi trường WebContainer Sandbox với bộ cục Tailwind CSS hiện đại, tích hợp hiệu ứng Framer Motion mượt mà và hiển thị đầy đủ các phần thông tin cá nhân bóc tách từ CV người dùng. Cung cấp các bartool từ việc kéo thả giao diện đến việc chọn UI sau đó có thể tùy chỉnh lại. Cùng với đó là chế độ xem trước, cho phép người dùng xem trang web trong một tab trình duyệt độc lập.



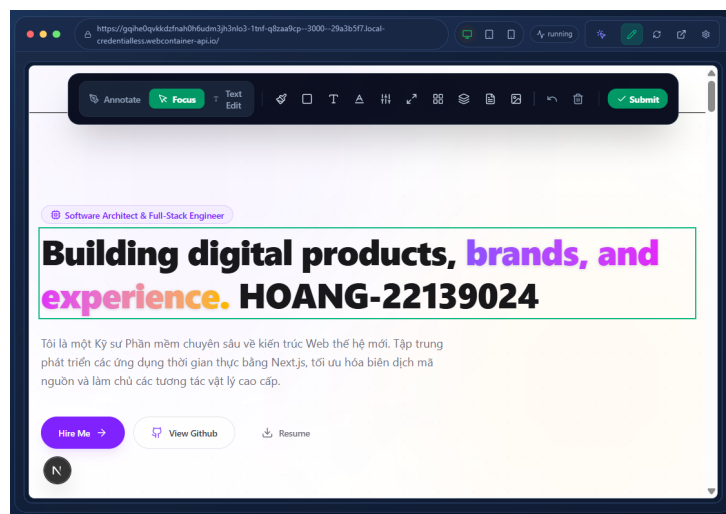
Hình 4.5: Tính năng mở rộng giao diện xem trước của WebContainer sang tab trình duyệt độc lập.

Hình 4.5 minh họa tính năng mở rộng trang web sang tab trình duyệt độc lập (giao diện xem trước độc lập). Khi người dùng chọn biểu tượng mở liên kết mới trên thanh công cụ xem trước, hệ thống sẽ mở trực tiếp trang web portfolio được lưu trữ và biên dịch bởi WebContainer trên một cửa sổ hoặc tab trình duyệt độc lập. Điều này cho phép người dùng kiểm tra sự hiển thị đầy đủ, tương tác mượt mà với trang web mà không bị ảnh hưởng bởi không gian hiển thị hạn chế của giao diện Web-IDE chính.



Hình 4.6: Minh họa cơ chế di chuột hover trong chế độ Inspector.

Hình 4.6 ghi nhận trạng thái hoạt động khi bật chế độ Inspector Mode. Khi di chuyển chuột qua các khối giao diện trong Live Preview, Iframe Bridge sẽ vẽ khung viền nét đứt màu vàng nổi lên và hiển thị tooltip chứa thông tin thẻ HTML cùng đường dẫn tệp nguồn.



Hình 4.7: Chế độ chỉnh sửa chữ đúp chuột trực tiếp contentEditable.

Hình 4.7 minh họa tính năng Visual Text Edit. Khi người dùng click đúp vào dòng chữ giới thiệu, phần tử DOM lập tức chuyển sang trạng thái `contentEditable` với khung viền xanh lục để người dùng gõ sửa đổi tại chỗ, đồng thời tạm thời ẩn các lớp phủ di chuột để tránh gây rối mắt.

Chương 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Luận văn đã hoàn thành đầy đủ các mục tiêu đề ra ban đầu cả về mặt nghiên cứu lý thuyết lẫn hiện thực hóa hệ thống thực nghiệm đối với đề tài "**Xây dựng hệ thống tạo website portfolio tự động dựa trên công nghệ browser-side sandbox và định tuyến mô hình ngôn ngữ lớn đa phương thức**". Về mặt khoa học và công nghệ, nghiên cứu đã làm chủ và ứng dụng thành công mô hình ngôn ngữ lớn đa phương thức để giải quyết bài toán phân tích cú pháp tự động các tài liệu lý lịch cá nhân dạng PDF phi cấu trúc sang định dạng JSON có cấu trúc một cách chính xác. Đồng thời, đề tài đã làm chủ công nghệ ảo hóa trình duyệt thông qua giải pháp WebContainer API, cho phép vận hành một môi trường Node.js đầy đủ ngay trên trình duyệt phía máy khách, giúp giảm thiểu tối đa chi phí tài nguyên máy chủ và nâng cao tính riêng tư dữ liệu của người dùng cuối. Bên cạnh đó, giải pháp phân tích cú pháp cây cú pháp trừu tượng AST trực tiếp trên trình duyệt đã mở ra khả năng vá lỗi mã nguồn và cập nhật giao diện React thời gian thực dựa trên các bộ định vị DOM tương tác.

Về mặt thực tiễn, hệ thống thực nghiệm Nexastu đã được thiết kế và triển khai hoàn chỉnh dựa trên các công nghệ lập trình hiện đại như Next.js, Convex và Inngest. Việc thiết kế và vận hành thành công cầu nối Inspector Bridge thông qua cơ chế truyền tin liên nguồn đã giải quyết triệt để rào cản CORS bảo mật của iframe, hiện thực hóa các tính năng chỉnh sửa văn bản trực tiếp trên giao diện và định tuyến mô hình xử lý. Việc tích hợp cơ chế tối ưu hóa bộ nhớ đệm phụ thuộc đã giúp rút ngắn thời gian khởi tạo môi trường sandbox xuống dưới hai giây, đem lại trải nghiệm mượt mà cho người dùng. Hơn thế nữa, luồng xử lý định tuyến đa phương thức dự phòng đã tối ưu hóa hiệu quả sự cân bằng giữa chi phí vận hành và chất lượng phản hồi, tự động chuyển đổi linh hoạt giữa các mô hình ngôn ngữ lớn

khi nhận diện có dữ liệu hình ảnh đính kèm từ phía người dùng. Tất cả các kết quả thực nghiệm và đo lường hiệu năng đều cho thấy hệ thống hoạt động ổn định, đạt độ tin cậy cao và đáp ứng tốt các yêu cầu của một môi trường phát triển ứng dụng di động và web hiện đại trực tiếp trên trình duyệt.

5.2 Hạn chế của hệ thống

Mặc dù đạt được những kết quả rất khả quan và chứng minh được tính khả thi của giải pháp, hệ thống Nexastu vẫn tồn tại một số rào cản kỹ thuật cần được phân tích để cải thiện trong tương lai. Trước hết, việc lựa chọn kiến trúc ảo hóa trình duyệt WebContainer mang lại lợi thế to lớn khi đưa chi phí vận hành máy chủ biên dịch của Nexastu về mức xấp xỉ không — đây là một giải pháp kiến trúc vô cùng thực tế và bền vững đối với các dự án học thuật hoặc nhà phát triển độc lập chưa có nguồn lực hạ tầng lớn để duy trì hệ thống máy ảo liên tục. Tuy nhiên, kiến trúc này cũng đặt ra giới hạn do phụ thuộc vào năng lực xử lý CPU đơn luồng và dung lượng RAM ảo hóa của WebAssembly trên trình duyệt phía máy khách, khiến quy trình biên dịch sản xuất tối ưu hóa toàn diện trên các dự án quy mô lớn chưa thực sự nhanh chóng.

Thứ hai, tính năng trích xuất thông tin lý lịch cá nhân đầu vào mới chỉ tập trung xử lý định dạng tệp PDF hoặc nhập văn bản thô, chưa mở rộng khả năng tự động nhận diện và phân tích trực tiếp các định dạng tài liệu văn phòng phổ biến khác như Microsoft Word hay hình ảnh quét thô trực tiếp mà không qua chuyển đổi định dạng trung gian từ phía người dùng.

Thứ ba, cơ chế tương tác trực quan thông qua công cụ chỉnh sửa văn bản trực tiếp chỉ hoạt động ổn định và chính xác đối với các đoạn văn bản tĩnh được khai báo trực tiếp trong mã nguồn JSX. Đối với các thành phần giao diện động được hiển thị dựa trên cấu trúc ánh xạ dữ liệu hoặc các biến số phức tạp, hệ thống định vị phần tử DOM chưa thể ánh xạ ngược chính xác đến mã nguồn gốc để thực hiện thay đổi, buộc người dùng phải sử dụng trợ lý trí tuệ nhân tạo để mô tả yêu cầu tái tạo lại mã nguồn.

Thứ tư, về mặt ứng dụng thực tiễn và tính thẩm mỹ, hệ thống hiện tại mới chỉ tập trung tối ưu hóa cho các trang web lý lịch năng lực cá nhân (portfolio) với các cấu trúc thông tin tương đối đơn giản, chưa mở rộng khả năng sinh mã tự động cho các loại hình website phức tạp hơn, đặc biệt là các trang thương mại điện tử đòi hỏi tích hợp giỏ hàng và cổng thanh toán trực tuyến. Bên cạnh đó, khả năng sáng tạo giao diện của tác nhân trí tuệ nhân tạo vẫn còn rập khuôn theo các mẫu định sẵn, chưa đạt được độ tinh tế và phá cách nghệ thuật mang tính cá nhân hóa cao.

Thứ năm, mặc dù hệ thống đã tích hợp và gọi thành công các mô hình ngôn ngữ lớn mạnh mẽ, sự khác biệt lớn nhất giữa giải pháp thực nghiệm Nexastu và các nền tảng IDE Sandbox lớn trong công nghiệp nằm ở độ phủ và tính hoàn thiện của hệ sinh thái phần mềm hỗ trợ bao quanh tác nhân trí tuệ nhân tạo. Việc chỉ sở hữu một nhân AI mạnh là chưa đủ nếu hệ thống phụ trợ xung quanh như công cụ phát hiện lỗi tĩnh, bộ tối ưu hóa gói nạp hoặc cơ chế tự động sửa lỗi chạy ngầm hoạt động kém, dẫn

đến chất lượng mã nguồn cuối cùng để phát sinh lỗi hoặc không đạt độ tin cậy như kỳ vọng của một môi trường phát triển thực tế.

Cuối cùng, tiến trình cập nhật giao diện xem trước sau khi trợ lý trí tuệ nhân tạo hoàn thành việc sinh mã nguồn vẫn dựa trên việc biên dịch lại và khởi động lại máy chủ cục bộ trong sandbox, gây ra một khoảng trễ nhỏ và chưa thể hỗ trợ cơ chế truyền phát mã nguồn trực tiếp kết hợp cập nhật giao diện tức thời theo từng ký tự đang được sinh ra.

5.3 Hướng phát triển trong tương lai

Dựa trên các kết quả đạt được cùng việc nhận diện rõ ràng các hạn chế kỹ thuật hiện tại, hướng phát triển và nghiên cứu tiếp theo của đề tài sẽ tập trung vào một số khía cạnh trọng tâm nhằm nâng cao hiệu năng và mở rộng tính ứng dụng của hệ thống Nexastu.

Trước hết, hệ thống cần được bổ sung và mở rộng thư viện mẫu thiết kế giao diện portfolio cá nhân với các phong cách đa dạng hơn, đặc biệt là việc tích hợp các thư viện đồ họa không gian ba chiều tương tác như Three.js hay React Three Fiber. Điều này không chỉ giúp gia tăng tính thẩm mỹ và sự lôi cuốn của trang cá nhân mà còn tạo ra sự khác biệt lớn so với các công cụ tạo trang web truyền thống. Đồng thời, việc mở rộng khả năng xử lý đa định dạng tài liệu đầu vào bao gồm Microsoft Word và các định dạng ảnh thông qua việc tích hợp các thư viện phân tích tài liệu sâu kết hợp công nghệ nhận dạng ký tự quang học OCR sẽ giúp đơn giản hóa tối đa quy trình khởi tạo ban đầu cho người dùng.

Thứ hai, việc nghiên cứu tối ưu hóa thời gian tải và khởi dựng WebContainer thông qua kỹ thuật nạp sẵn tài nguyên sử dụng Service Workers là một hướng đi vô cùng hứa hẹn. Bằng cách lưu trữ đệm các tệp nhị phân WebAssembly và các gói thư viện phụ thuộc cốt lõi của Node.js ngay tại trình duyệt, hệ thống có thể bỏ qua thời gian chờ đợi khởi tạo ban đầu, giúp giao diện xem trước xuất hiện gần như ngay lập tức khi người dùng mở ứng dụng. Song song với đó, việc hoàn thiện tính năng xuất mã nguồn toàn vẹn dưới dạng lưu trữ nén hoặc đẩy trực tiếp lên các kho chứa GitHub cá nhân sẽ cho phép người dùng dễ dàng chuyển giao, bảo trì và deploy dự án lên các nền tảng hosting đám mây miễn phí một cách chủ động.

Thứ ba, việc tích hợp hệ thống kiểm soát phiên bản Git cục bộ ngầm bên dưới hệ thống tệp tin ảo của WebContainer sẽ mang lại giá trị sử dụng to lớn cho người dùng. Cơ chế này sẽ cho phép người dùng lưu trữ lịch sử các phiên bản giao diện do trợ lý AI chỉnh sửa, giúp họ dễ dàng so sánh các thay đổi mã nguồn trực quan hoặc thực hiện hoàn tác về bất kỳ mốc thời gian nào trong quá khứ một cách an toàn. Để đảm bảo an toàn tuyệt đối, hệ thống cần được thiết lập cơ chế cách ly sandbox chặt chẽ, ngăn chặn việc thực thi các câu lệnh shell nguy hiểm có thể ảnh hưởng đến mã nguồn hệ thống quản lý.

Thứ tư, nghiên cứu sẽ phát triển và nâng cấp bộ công cụ chuyển đổi (Conversion Engine) để mở

rộng phạm vi sinh mã nguồn sang các miền ứng dụng phức tạp hơn, tiêu biểu là các trang thương mại điện tử hoàn chỉnh. Khi các thư viện và cấu phần thiết kế UI/UX trên thế giới ngày càng phát triển đầy đủ và chuẩn hóa hơn, việc thiết kế công cụ để liên kết các cổng thanh toán, quản lý giỏ hàng và danh mục sản phẩm tự động sẽ giúp hệ thống đáp ứng tốt các bài toán thực tiễn đa dạng của thị trường.

Thứ năm, về khía cạnh nâng cao tính sáng tạo và thẩm mỹ giao diện của AI, hướng nghiên cứu tiếp theo sẽ ứng dụng mô hình đa tác nhân (Multi-agent System) kết hợp giữa tác nhân sinh mã và tác nhân kiểm duyệt thiết kế (Design Auditor Agent). Tác nhân kiểm duyệt sẽ sử dụng các mô hình thị giác lớn (Vision LLMs) để liên tục đánh giá độ hài hòa về màu sắc, tính cân đối của layout và trải nghiệm người dùng ngay trong quá trình AI gõ mã, từ đó đưa ra các phản hồi điều chỉnh kịp thời, giúp mã nguồn sinh ra đạt tính thẩm mỹ và tính cá nhân hóa cao nhất.

Cuối cùng, việc xây dựng và hoàn thiện hệ sinh thái công cụ phụ trợ xung quanh trợ lý trí tuệ nhân tạo bên trong WebContainer sandbox là một hướng đi vô cùng cần thiết. Nghiên cứu sẽ tích hợp thêm các công cụ tự động phân tích lỗi cú pháp tĩnh (Static Analysis) và kiểm thử khởi chạy nhanh (Smoke Test) để phát hiện kịp thời các lỗi biên dịch, kết hợp cùng bộ tối ưu hóa tài nguyên tải và biên dịch chéo tự động. Bằng cách thiết lập một vành đai kỹ thuật hỗ trợ chặt chẽ xung quanh AI, hệ thống có thể đảm bảo chất lượng của dự án phần mềm cuối cùng vẫn được bảo đảm ở mức tốt nhất, ngay cả khi sử dụng các mô hình ngôn ngữ nhỏ hơn.

TÀI LIỆU THAM KHẢO

- [1] Vercel. (2026) Next.js Documentation: App Router and Server Components. Accessed: 2026-06-05. [Online]. Available: <https://nextjs.org/docs>
- [2] Meta Open Source. (2026) React Server Components and Concurrent Features. Accessed: 2026-06-05. [Online]. Available: <https://react.dev/reference/react/components>
- [3] StackBlitz. (2026) WebContainer API: Run Node.js directly in the browser. Accessed: 2026-06-05. [Online]. Available: <https://webcontainers.io>
- [4] DeepSeek-AI. (2026) DeepSeek-V4 Technical Report: Towards Highly Efficient Million-Token Context Intelligence. Accessed: 2026-06-05. [Online]. Available: https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf
- [5] Google DeepMind. (2024) Introducing Gemini 2.0: Our New AI Model Designed for the Agentic Era. Accessed: 2026-06-05. [Online]. Available: <https://blog.google/technology/ai/google-gemini-next-generation-model-december-2024/>
- [6] J. Kyle. (2024) Babel Handbook: Abstract Syntax Trees (AST) and Source-to-Source Compilation. Accessed: 2026-06-05. [Online]. Available: <https://github.com/jamiebuilds/babel-handbook>
- [7] Convex, Inc. (2026) Convex DB: Reactive Backend and Real-time Synchronization. Accessed: 2026-06-05. [Online]. Available: <https://docs.convex.dev>
- [8] Inngest, Inc. (2026) Inngest: Event-driven Serverless Background Functions. Accessed: 2026-06-05. [Online]. Available: <https://www.inngest.com/docs>
- [9] C. Richardson, *Microservices Patterns: With examples in Java*. Manning Publications, 2018.
- [10] StackBlitz. (2026) WebContainer File System API Reference. Accessed: 2026-06-06. [Online]. Available: <https://webcontainers.io/api>